

HiDiTingV100 多媒体软件

开发指南

文档版本 00B02

发布日期 2025-07-30

前言

概述

本文档介绍媒体子系统相关的方案、使用方法以及注意事项，用于指导用户快速开发上层应用。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B02	2025-07-30	- 在“2.2.2 音频中断策略管理”中更新图 2-2。 - 新增“5.7 编码码率、5.8 上行音量”。
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

1.1 媒体子系统框架..... 1

1.2 模块划分..... 2

1.2.1 Framework 层模块..... 2

1.2.2 Hal 层模块..... 3

2 音频管理.....4

2.1 基本概念..... 4

2.2 功能描述..... 6

2.2.1 音频设备管理..... 6

2.2.2 音频中断策略管理..... 6

2.2.3 音频流管理..... 8

2.2.4 音量管理..... 9

2.2.5 音效管理..... 9

2.2.6 音频管理 DFX..... 9

2.3 API 参考..... 10

2.3.1 AudioManager 类..... 10

2.3.1.1 GetInstance.....11

2.3.1.2 Initialize.....11

2.3.1.3 MakeSessionId..... 12

2.3.1.4 SetVolume..... 13

2.3.1.5 GetVolume..... 14

2.3.1.6 GetMinVolume..... 15

2.3.1.7 GetMaxVolume..... 16

2.3.1.8 Mute..... 16

2.3.1.9 UnMute..... 17

2.3.1.10 IsMute	18
2.3.1.11 SetMasterMute	19
2.3.1.12 IsMasterMute	20
2.3.1.13 SetMicrophoneMute	21
2.3.1.14 IsMicrophoneMute	22
2.3.1.15 SetRingerMode	22
2.3.1.16 GetRingerMode	23
2.3.1.17 GetDevices	24
2.3.1.18 IsSupportOffload	25
2.3.1.19 SetDeviceConnectionState	26
2.3.1.20 ActivateAudioInterrupt	27
2.3.1.21 DeactivateAudioInterrupt	28
2.3.1.22 DumpInfo	29
2.3.2 InterruptListener 类	30
2.3.2.1 OnInterrupt	30
2.3.3 AudioStreamIn 类	31
2.3.3.1 Init	31
2.3.3.2 GetCurrentDeviceId	32
2.3.3.3 GetCurrentChannelId	33
2.3.3.4 AttachBackend	34
2.3.3.5 GetTimestamp	35
2.3.3.6 Start	36
2.3.3.7 Stop	36
2.3.3.8 SetInputDevice	37
2.3.3.9 ObtainBuffer	38
2.3.3.10 Release	39
2.3.4 AudioStreamOut 类	40
2.3.4.1 Init	41
2.3.4.2 Play	42
2.3.4.3 Pause	42
2.3.4.4 Stop	43
2.3.4.5 Release	44
2.3.4.6 SetPlayBackSpeed	45
2.3.4.7 SetVolume	46
2.3.4.8 GetCurrentDeviceId	47
2.3.4.9 GetCurrentChannelId	47
2.3.4.10 AttachFrontend	48
2.3.4.11 StreamWrite	49
2.3.4.12 GetPosition	50
2.3.4.13 GetTimestamp	51
2.3.4.14 Flush	52

2.3.4.15 SetOutputDevice.....	52
2.3.4.16 GetBufferFrameSize	53
2.3.4.17 GetBufferUnderflowTimes	54
2.3.4.18 DumpInfo	55
2.3.4.19 SetParam.....	56
2.4 数据类型、数据结构.....	57
2.4.1 AudioSession	58
2.4.2 DeviceFlag	58
2.4.3 AudioDeviceType	59
2.4.4 AudioDeviceInfo	60
2.4.5 AudioDeviceConnectState	61
2.4.6 AudioRingMode	62
2.4.7 AudioSourceType	63
2.4.8 AudioDeviceDesc.....	65
2.4.9 AudioCodecFormat	66
2.4.10 AudioStreamType	68
2.4.11 AudioLinkDirection	71
2.4.12 AudioStreamFlag	72
2.4.13 AudioChannelMode	73
2.4.14 AudioBitWidth	74
2.4.15 AudioOffloadInfo	75
2.4.16 AudioStrategy	76
2.4.17 enum AudioInterruptType	77
2.4.18 AudioInterruptHintType	78
2.4.19 AudioStreamInfo	80
2.4.20 AudioRendererConfig	81
2.4.21 CapturerInputConfig	82
2.4.22 RendererCallback	83
2.4.23 CapturerCallback	84
2.4.24 CapturerBuffer	84
2.4.25 InterruptDebugInfo.....	85
2.4.26 DeviceDebugInfo	86
2.4.27 StreamDebugInfo.....	86
2.4.28 AudioInterruptDebugInfo	87
2.4.29 AudioStreamDebugInfo	88
2.4.30 AudioDeviceDebugInfo	89
2.4.31 AudioServiceDebugInfo	89
2.4.32 struct AudioInterrupt	90
2.4.33 AudioManagerDebugInfo	91
2.5 错误码	92
2.6 开发指导.....	93

2.6.1 本地音乐播放	93
2.6.2 本地音乐和蓝牙音乐切换	94
2.6.3 蓝牙语音通话	95
2.6.4 音乐播放中途蓝牙语音通话	97
2.6.5 4G VOLTE 本地通话	98
2.6.6 4G VOLTE 蓝牙通话	99
2.6.7 A2DP Sink 蓝牙播放	101
3 音频采集	103
3.1 概述	103
3.2 功能描述	103
3.2.1 机制原理	103
3.2.1.1 音频采集状态机	105
3.2.2 音频采集 DFX	105
3.3 API 参考	106
3.3.1 SetCapturerInfo	106
3.3.2 GetCapturerInfo	107
3.3.3 Start	108
3.3.4 Read	109
3.3.5 GetStatus	110
3.3.6 GetAudioTime	111
3.3.7 GetMinFrameCount	112
3.3.8 GetFrameCount	113
3.3.9 Stop	113
3.3.10 Release	114
3.3.11 DumpInfo	115
3.4 数据类型、数据结构	116
3.4.1 AudioCapturer	116
3.4.2 AudioCapturerInfo	117
3.4.3 AudioSourceType	118
3.4.4 AudioCodecFormat	120
3.4.5 AudioStreamType	122
3.4.6 AudioBitWidth	123
3.4.7 Timestamp	124
3.4.8 Timebase	125
3.4.9 State	126
3.4.10 AudioCapturerDebugInfo	127

3.5 错误码	128
3.6 典型场景开发指导	129
3.6.1 音频采集	129
3.6.1.1 音频采集准备工作	129
3.6.1.2 音频采集启动	132
3.6.1.3 音频采集数据提取	132
3.6.1.4 音频采集停止	133
3.6.1.5 音频采集退出工作	133
4 播放器模块	134
4.1 概述	134
4.1.1 播放器模块层次结构	135
4.1.2 播放器模块处理流程	136
4.1.3 播放器状态图	138
4.1.4 StreamSource 源类型 Buffer 流转	139
4.2 功能描述	140
4.2.1 播控模块	140
4.2.2 播放器模块 DFX	141
4.3 API 参考	141
4.3.1 Player 类	141
4.3.1.1 SetSource	142
4.3.1.2 Prepare	143
4.3.1.3 Play	144
4.3.1.4 IsPlaying	145
4.3.1.5 Pause	145
4.3.1.6 Stop	146
4.3.1.7 Rewind	147
4.3.1.8 SetVolume	148
4.3.1.9 SetVideoSurface	149
4.3.1.10 EnableSingleLooping	150
4.3.1.11 IsSingleLooping	151
4.3.1.12 GetPlayerState	152
4.3.1.13 GetCurrentTime	153
4.3.1.14 GetDuration	154
4.3.1.15 GetVideoWidth	155
4.3.1.16 GetVideoHeight	155

4.3.1.17 SetPlaybackSpeed	156
4.3.1.18 GetPlaybackSpeed	157
4.3.1.19 Reset	158
4.3.1.20 Release	159
4.3.1.21 SetPlayerCallback	160
4.3.1.22 SetParameter	161
4.3.1.23 DumpInfo	162
4.3.2 PlayerCallback 类	163
4.3.2.1 OnPlaybackComplete	163
4.3.2.2 OnError	164
4.3.2.3 OnInfo	164
4.3.2.4 OnVideoSizeChanged	165
4.3.2.5 OnRewindToComplete	166
4.3.3 Source 类	167
4.3.3.1 Source	167
4.3.3.2 GetSourceFd	169
4.3.3.3 GetSourceType	169
4.3.3.4 GetSourceFdOffset	170
4.3.3.5 GetSourceFdLength	171
4.3.3.6 GetSourceUri	172
4.3.3.7 GetSourceHeader	173
4.3.3.8 GetSourceStream	173
4.3.3.9 GetSourceStreamFormat	174
4.3.4 StreamSource 类	175
4.3.4.1 OnBufferAvailable	175
4.3.4.2 SetStreamCallback	176
4.3.5 StreamCallback 类	177
4.3.5.1 GetBuffer	177
4.3.5.2 QueueBuffer	178
4.3.5.3 SetParameters	179
4.4 数据类型及数据结构	179
4.4.1 PlayerSeekMode	180
4.4.2 PlayerStates	180
4.4.3 PlayerInfoType	181
4.4.4 PlayerErrorType	182
4.4.5 SourceType	183
4.4.6 BufferFlags	183
4.4.7 CodecFormat	184
4.4.8 PlayerDebugInfo	186
4.4.9 PlayerFileDebugInfo	187

4.4.10 PlayerVideoDebugInfo	188
4.4.11 PlayerAudioDebugInfo	189
4.4.12 PlayerControlDebugInfo	190
4.5 错误码	192
4.6 使用指导	192
4.6.1 本地文件播放示例	192
4.6.2 StreamSource 播放示例	196
4.6.3 注意事项	197
5 音频配置	198
5.1 内置输入设备	198
5.2 内置输出设备	199
5.3 外挂输入设备	200
5.4 外挂输出设备	201
5.5 TRACK MODE	203
5.6 AEF	203
5.7 编码码率	203
5.8 上行音量	203
6 AT 命令使用	204
6.1 本地播放	204
6.1.1 设置指令	204
6.1.2 查询指令	206
6.2 本地采集	207
6.2.1 设置指令	207
6.2.2 查询指令	208
6.3 音频管理	208
6.3.1 设置指令	208

插图目录

图 1-1 整体框图 1

图 1-2 模块划分 2

图 2-1 音频管理总体框架 5

图 2-2 输出流打断策略关系图 7

图 2-3 输入流打断策略关系图 7

图 3-1 音频采集模块上下文组件关系图 104

图 3-2 音频采集调用和状态迁移 105

图 4-1 播放器模块上下文 135

图 4-2 播放器模块层次结构图 135

图 4-3 播放器模块处理流程 137

图 4-4 播放器模块状态图 139

图 4-5 StreamSource buffer 流转图 140

表格目录

表 2-1 音频管理错误码..... 92

表 3-1 音频采集常用参数配置..... 107

表 3-2 音频采集 API 错误码..... 128

表 6-1 本地播放设置指令一览表..... 204

表 6-2 本地播放查询指令一览表..... 206

表 6-3 本地录制设置指令一览表..... 207

表 6-4 本地录制查询指令一览表..... 208

表 6-5 音频管理设置指令一览表..... 208

1 概述

本章节介绍媒体子系统框架结构及支持的功能，详细见如下分解内容。

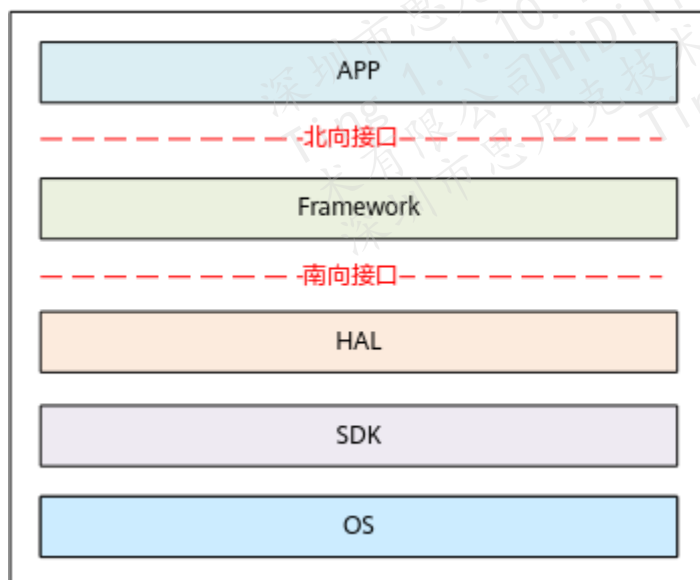
1.1 媒体子系统框架

1.2 模块划分

1.1 媒体子系统框架

媒体子系统框架如图 1-1 所示。

图1-1 整体框图

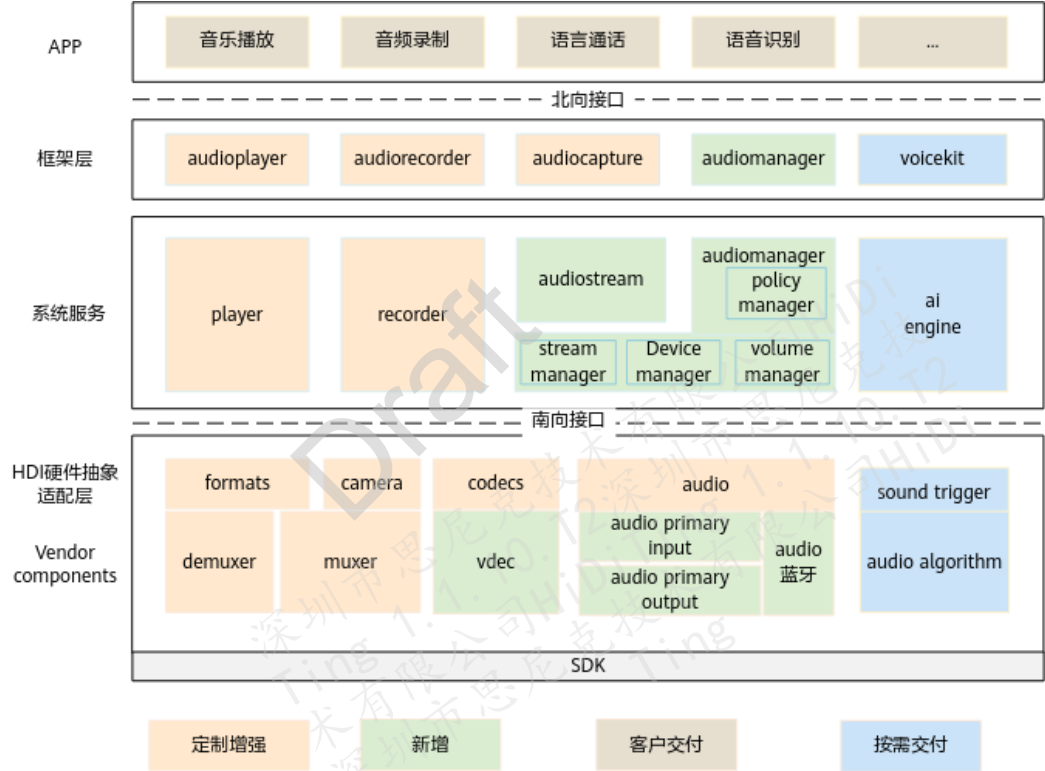


各层次职责概述：

- APP：基于媒体框架提供的接口开发应用，同时向用户提供 UI 使用。
- Framework：为上层开发提供媒体业务相关北向接口。
- HAL：基于 SDK，提供设备管理、数据流控制等功能。
- SDK：提供媒体采集、编解码、图像处理、视频显示等具体功能。

1.2 模块划分

图1-2 模块划分



本文针对媒体子系统主要介绍 AudioManager（音频管理）、AudioCapture（音频采集）、Player（播放器）。

1.2.1 Framework 层模块

- AudioManager：音频管理，对 APP 侧提供音频资源管理接口。
- AudioCapture：音频采集，对 APP 侧提供音频数据采集接口。
- Player：播放器，对 APP 侧提供音视频片源播放接口。

1.2.2 Hal 层模块

- audio：提供音频设备管理、音频采集、渲染管理、音量调节等功能。
- codecs：提供音视频编解码功能。
- camera：提供解码显示功能。
- formats：提供容器封装及解封装功能。

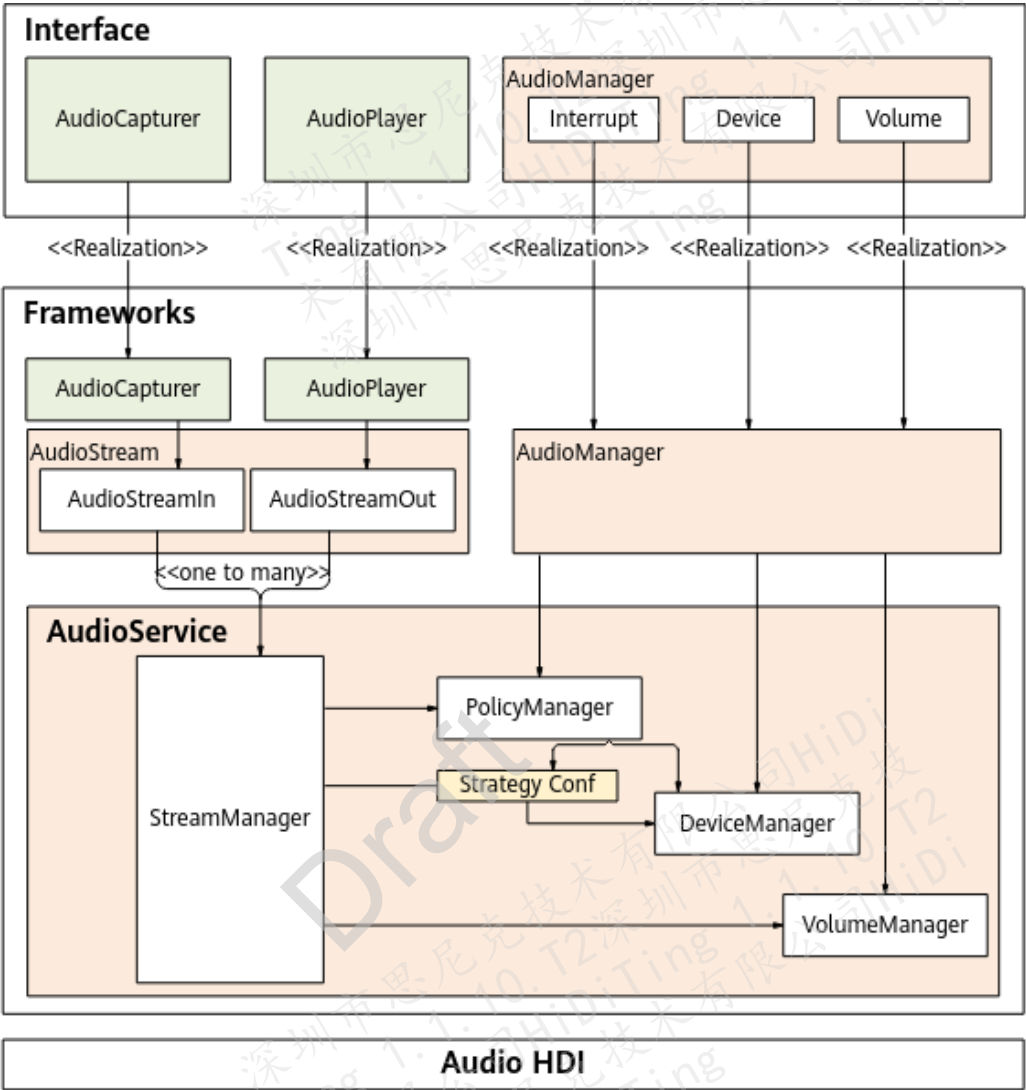
2 音频管理

- 2.1 基本概念
- 2.2 功能描述
- 2.3 API 参考
- 2.4 数据类型、数据结构
- 2.5 错误码
- 2.6 开发指导

2.1 基本概念

音频管理负责管理系统的音频资源，包括音频设备管理、音频策略管理、音频冲突管理、音频流管理、音量管理、音效管理。

图2-1 音频管理总体框架



- `AudioManager` 包括对输入输出设备的管理和输入输出流的管理。
- `AudioManager` 根据功能的不同，将音频设备分为输入和输出设备，使用 `DeviceManager` 模块基于南向 `Audio HDI` 对音频设备进行抽象和统一管理。
- 对应到输入设备上的采集数据流用 `AudioStreamIn` 实现。
- 对应到输出设备上的播放数据流用 `AudioStreamOut` 实现。
- `StreamManager` 模块对不同应用的不同类型数据流进行统一管理。
- `PolicyManager` 模块根据不同产品音频配置策略决策音频流的冲突解决策略和路由设备选择。

2.2 功能描述

2.2.1 音频设备管理

AudioManager 根据功能的不同，将音频设备分为输入和输出设备。当前支持 Mic、Speaker、Modem 以及蓝牙设备的管理，Mic、Speaker 属于 Primary device，是常驻设备；Modem 和蓝牙设备的连接和断开状态，需要用户或者蓝牙服务在蓝牙对应的音频设备状态改变时，通过接口调用反馈给 AudioManager：

- Primary device：Mic、Speaker。
- Bluetooth device：A2DP、SCO IN、SCO OUT。
- Modem device：OUT_MODEM、OUT_MODEM_HEADSET、IN_MODEM、IN_MODEM_HEADSET。

2.2.2 音频中断策略管理

音频管理根据业务场景不同，涉及音频输入、输出策略决策和焦点管理，细分如下：

- 支持如下音频类型：
 - 报警类：健康报警、设备告警。
 - 通话类：语音通话、蓝牙通话、4G VOLTE 通话。
 - 语音交互类：智能语音交互播报（例如：智能音箱）。
 - 提示类：蓝牙电话提示音、系统提示音、闹钟。
 - 操作类：触屏音、按键音。
 - 媒体类：影音。
 - 蓝牙输入媒体音：手机蓝牙连接手表，然后从手机过来的媒体音。

图2-2 输出流打断策略关系图

第二路 \ 第一路	设备报警	蓝牙通话	来电铃声	语音合成	语音助手	闹钟	设备提示音	运动健康提示音	媒体音	健身视频提示音	蓝牙输入媒体音
设备报警		打断退出	打断退出	打断退出	打断退出	打断退出	打断退出	打断退出	打断暂停	打断退出	打断暂停
蓝牙通话/4G VOLTE 通话	不播放		打断退出	打断退出	打断退出	打断退出	打断退出	打断退出	打断暂停	打断退出	打断暂停
来电铃声	不播放	混音		打断退出	打断退出	打断退出	打断退出	打断退出	打断暂停	打断退出	打断暂停
语音合成	不播放	不播放	不播放		打断退出	打断退出	打断退出	打断退出	打断暂停	打断退出	打断暂停
语音助手	不播放	不播放	不播放	不播放		打断退出	打断退出	打断退出	打断暂停	打断退出	打断暂停
闹钟	不播放	不播放	混音	混音	混音		打断退出	打断退出	打断暂停	打断退出	打断暂停
设备提示音	不播放	混音	混音	混音	混音	混音		混音	混音	混音	混音
运动健康提示音	不播放	混音	混音	混音	混音	混音	混音		混音	混音	混音
媒体音	延时	延时	延时	延时	延时	不播放	混音	混音		不播放	不播放
健身视频提示音	不播放	不播放	不播放	不播放	不播放	不播放	混音	混音	打断暂停		打断暂停
蓝牙输入媒体音	延时	延时	延时	延时	延时	延时	混音	混音	打断暂停	延时	

说明

- 目前不支持只能在本地播放的音频和蓝牙提示音的并发场景，用户需要在播放本地音频前，先将蓝牙输出设备断开。
- 有蓝牙输入媒体音时，可以连接蓝牙耳机，但是手表本身的音频都从本地输出，不会从蓝牙耳机输出。
- 如果手表连接手机，手表正在播放音乐，此时从手机侧过来的 A2DP 链路建立会把手表播放的音乐打断，A2DP 链路断开后音乐恢复播放，可能手机只是创建 A2DP 链路，并没有发送数据，所以手表音乐播放过程中会出现短暂的无声，过一会就恢复的现象，实际上是正常的现象。

图2-3 输入流打断策略关系图

第二路 \ 第一路	蓝牙通话	语音助手	语音采集录制	语音识别	蓝牙输入媒体音
蓝牙通话/4G VOLTE 通话		打断退出	打断退出	打断退出	打断暂停
语音助手	不播放		打断退出	打断退出	打断暂停
语音采集录制	不播放	不播放		打断退出	混音
语音识别	不播放	不播放	不播放		打断暂停
蓝牙输入媒体音	延时	延时	混音	延时	

目前不支持音频输入和输出并发场景的打断策略，典型场景：第一路媒体音，第二路语音采集录制、第一路语音采集录制，第二路媒体音等。

- 支持音频输出分类分级处理策略：
 - 支持不同音频类型配置不同优先级，根据产品可以进行不同的配置。

- 支持音频输出根据类型和优先级决策音频冲突的策略：打断退出、打断暂停、不播放、混音，延时。
- 支持音频输入设备分优先级选择：
支持音频输入设备分优先级选择，默认蓝牙优先于本地 Mic。
- 支持音频输出设备分优先级选择：
支持音频输出设备分优先级选择，默认蓝牙优先于本地 Speaker。
- 支持异常焦点恢复：
 - 焦点申请成功后，一直不创建对应流会触发超时，进而删除该异常焦点。
 - 流销毁后，不去激活焦点会触发超时，进而删除该异常焦点。

须知

- 目前 target1 支持的多媒体业务场景只有音乐、提示音和蓝牙通话，因内存的限制，不支持这些业务场景之间的并发策略。
- 因 target1 的内存限制，如果当前只有音乐或提示音业务场景的时候不支持和其他业务场景并发，包括但不限于连接、数传等。

2.2.3 音频流管理

音频管理根据流的方向不同，将流分为输入流和输出流。从流的生命周期、状态、倍速、信息、在线模式等角度实现流管理和控制，细分如下：

- 支持音频流的生命周期管理：
支持根据音频策略决策、参数创建和销毁音频流。
- 支持音频流状态管理：
 - 支持根据用户行为和焦点状态启动、停止、暂停、刷新音频输出流。
 - 支持根据用户行为启动、停止音频输入流。
- 支持音频输出流倍速：
支持音频输出流的倍速播放（依赖 SDK，当前 SDK 未支持）。
- 支持音频流信息获取：
 - 支持获取音频输出流的播放位置信息。
 - 支持获取音频输出流的播放时间戳信息。
 - 支持获取音频输入流的采集时间戳信息。

- 支持获取音频输出流的播放缓冲大小。
- 支持音频流在线模式：
 - 支持获取音频输入流的数据通道 ID，在线采集数据。
 - 支持获取音频输出流的数据通道 ID，在线播放数据。

2.2.4 音量管理

音量管理涉及音量调节和音量记录，细分如下：

- 支持音频流音量调节：
 - 支持增大音量、减少音量、静音、关闭静音、恢复音量。
 - 支持调节不同的音频类型的音量，包含不同的输出设备：本地 speaker 和蓝牙音量。
- 支持音频流音量记录和管理：
 - 支持音量记录，播放时恢复上一次的音量。
 - 支持音量分类型记录管理，不同音频类型可以分别保存。
- 支持多音频场景叠加时音量的淡入淡出效果：
 - 即将到来的音频流会把当前的流的音量压低为即将到来的音频流的一半，这个比例会一直存在，调整新的音频流音量，老的音频流音量也会变化，调整老的音频流音量，新的音频流音量也会变化，但是这个变化的音量值不会被一直记录，音频流停止以后音量就恢复为原来的值。
 - 即将到来的音频流停止以后会恢复当前音频流的音量。

例如：现在正在播放提示音，此时又开始播放音乐，提示音和音乐应该混音，按照策略此时提示音音量会是音乐音量的一半，可能会出现提示音突然听不见或者声音很小，退出音乐又恢复的情况，就是这个策略导致的。

2.2.5 音效管理

音效按照音频场景对应生效，例如：音乐播放、语音通话场景对应不同音效，也可以没有音效，目前媒体配置音效具体效果依赖 SDK。

2.2.6 音频管理 DFX

音频管理 DFX 属于非功能属性，例如：性能、可靠性、可服务性、可测试性，细分如下：

- 音频焦点信息（焦点 SessionID、对应流类型、策略、对应流的创建销毁状态），请参见“2.3.1.22 DumpInfo”小节。
- 支持前后台音频流信息查询（包括音频流类型、对应策略、音频格式、混音比例、流类型音量、流音量、流路由设备），请参见“2.3.1.22 DumpInfo”小节。
- 支持音频设备音量查询。
- 音频设备切换到耳机时，保持音量在安全范围内。

2.3 API 参考

2.3.1 AudioManager 类

音频管理模块主要提供获取输入/输出设备、设备连接/断开、激活/去激活音频中断、音量设置/获取、静音设置/获取、响铃模式设置/获取等功能。提供以下 API：

- GetInstance：获取 AudioManager 实例（单例模式）。
- Initialize：初始化音频资源。
- MakeSessionId：创建会话 ID。
- SetVolume：设置指定流类型音量。
- GetVolume：获取指定流类型音量。
- GetMinVolume：获取指定流类型最小音量。
- GetMaxVolume：获取指定流类型最大音量。
- Mute：静音指定流类型。
- UnMute：取消指定流类型静音。
- IsMute：检查指定流类型是否静音。
- SetMasterMute：静音或取消静音全局音频流。
- IsMasterMute：检查音频流是否全局静音。
- SetMicrophoneMute：设置麦克风静音或取消静音。
- IsMicrophoneMute：检查麦克风是否静音。
- SetRingerMode：设置振铃模式。
- GetRingerMode：获取振铃模式。
- GetDevices：获取音频输入输出设备或所有设备的信息。
- IsSupportOffload：根据音频 Offload 信息查询是否支持 Offload。

- SetDeviceConnectionState：设置设备连接状态。
- ActivateAudioInterrupt：激活音频中断。
- DeactivateAudioInterrupt：去激活音频中断。
- DumpInfo：Dump 信息。

2.3.1.1 GetInstance

【描述】

获取 AudioManager 实例。

【语法】

static AudioManager& GetInstance();

【参数】

无

【返回值】

返回值	描述
AudioManager&	获取 AudioManager 实例，单例模式。

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.1 本地音乐播放](#)”小节。

【相关主题】

无。

2.3.1.2 Initialize

【描述】

初始化音频资源。

【语法】

bool Initialize();

【参数】

无。

【返回值】

返回值	描述	输入/输出
bool	- true：成功。 - false：失败。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.1 本地音乐播放](#)”小节。

【相关主题】

无。

2.3.1.3 MakeSessionId

【描述】

创建会话 ID。

【语法】

AudioSession MakeSessionId();

【参数】

无。

【返回值】

返回值	描述	输入/输出
-----	----	-------

返回值	描述	输入/输出
AudioSession	- 成功：返回会话 ID。 - 失败：返回 0。 详见“ 2.4.1 AudioSession ”小节。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.1 本地音乐播放](#)”小节。

【相关主题】

无。

2.3.1.4 SetVolume

【描述】

设置指定流类型音量。

【语法】

bool SetVolume(AudioStreamType streamType, int32_t volume);

【参数】

参数名称	描述	输入/输出
streamType	流类型。	输入
volume	音量值。	输入

【返回值】

返回值	描述	输入/输出
-----	----	-------

返回值	描述	输入/输出
bool	• true: 成功。 • false: 失败。	输出

【芯片差异】

无。

【注意】

音量值范围，可通过 GetMinVolume/ GetMaxVolume 获取。

【举例】

无。

【相关主题】

无。

2.3.1.5 GetVolume

【描述】

获取指定流类型音量。

【语法】

int32_t GetVolume(AudioStreamType streamType);

【参数】

参数名称	描述	输入/输出
streamType	流类型。	输入

【返回值】

返回值	描述	输入/输出
int32_t	音量值。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.6 GetMinVolume

【描述】

获取指定流类型最小音量。

【语法】

int32_t GetMinVolume(AudioStreamType streamType);

【参数】

参数名称	描述	输入/输出
streamType	流类型。	输入

【返回值】

返回值	描述	输入/输出
int32_t	最小音量值。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.7 GetMaxVolume

【描述】

获取指定流类型最大音量。

【语法】

int32_t GetMaxVolume(AudioStreamType streamType);

【参数】

参数名称	描述	输入/输出
streamType	流类型。	输入

【返回值】

返回值	描述	输入/输出
int32_t	最大音量值。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.8 Mute

【描述】

静音指定流类型。

【语法】

bool Mute(AudioStreamType streamType);

【参数】

参数名称	描述	输入/输出
streamType	流类型。	输入

【返回值】

返回值	描述	输入/输出
bool	- true：成功。 - false：失败。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.9 UnMute

【描述】

取消指定流类型静音。

【语法】

bool UnMute(AudioStreamType streamType) ;

【参数】

参数名称	描述	输入/输出
------	----	-------

参数名称	描述	输入/输出
streamType	流类型。	输入

【返回值】

返回值	描述	输入/输出
bool	- true: 成功。 - false: 失败。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.10 IsMute

【描述】

检查指定流类型是否静音。

【语法】

```
bool IsMute(AudioStreamType streamType);
```

【参数】

参数名称	描述	输入/输出
streamType	流类型。	输入

【返回值】

返回值	描述	输入/输出
bool	• true: 静音。 • false: 非静音。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.11 SetMasterMute

【描述】

静音或取消静音全局音频流。

【语法】

```
bool SetMasterMute(bool isMute);
```

【参数】

参数名称	描述	输入/输出
isMute	• true: 静音。 • false: 取消静音。	输入

【返回值】

返回值	描述	输入/输出
bool	• true: 成功。	输出

返回值	描述	输入/输出
	false: 失败。	

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.12 IsMasterMute

【描述】

检查音频流是否全局静音。

【语法】

bool IsMasterMute();

【参数】

无。

【返回值】

返回值	描述	输入/输出
bool	- true: 静音。 - false: 非静音。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.13 SetMicrophoneMute

【描述】

设置麦克风静音或取消静音。

【语法】

```
bool SetMicrophoneMute(bool isMute);
```

【参数】

参数名称	描述	输入/输出
isMute	• true: 静音。 • false: 取消静音。	输入

【返回值】

返回值	描述	输入/输出
bool	• true: 成功。 • false: 失败。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.14 IsMicrophoneMute

【描述】

检查麦克风是否静音。

【语法】

```
bool IsMicrophoneMute();
```

【参数】

无。

【返回值】

返回值	描述	输入/输出
bool	• true: 静音。 • false: 非静音。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.15 SetRingerMode

【描述】

设置振铃模式。

【语法】

bool SetRingerMode(AudioRingMode mode) ;

【参数】

参数名称	描述	输入/输出
mode	振铃模式，详见“2.4.6 AudioRingMode”小节。	输入

【返回值】

返回值	描述	输入/输出
bool	- true：成功。 - false：失败。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.16 GetRingerMode

【描述】

获取振铃模式。

【语法】

int32_t GetRingerMode();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	当前振铃模式，详见“2.4.6 AudioRingMode”小节。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.17 GetDevices

【描述】

获取音频输入输出设备或所有设备的信息。

【语法】

std::vector<AudioDeviceInfo> GetDevices(DeviceFlag flag) ;

【参数】

参数名称	描述	输入/输出
flag	设备标志位，详见“2.4.2 DeviceFlag”小节。	输入

【返回值】

返回值	描述	输入/输出
std::vector<AudioDeviceInfo>	设备信息列表，详见“2.4.4 AudioDeviceInfo”小节。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.18 IsSupportOffload

【描述】

根据音频 Offload 信息查询是否支持 Offload。

【语法】

bool IsSupportOffload(AudioOffloadInfo &info) ;

【参数】

参数名称	描述	输入/输出
info	Offload 信息，详见“2.4.15 AudioOffloadInfo”小节。	输入

【返回值】

返回值	描述	输入/输出
bool	- true：支持 Offload。 - false：不支持 Offload。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.19 SetDeviceConnectionState

【描述】

设置设备连接状态。

【语法】

int32_t SetDeviceConnectionState(AudioDeviceType device,
AudioDeviceConnectState state);

【参数】

参数名称	描述	输入/输出
device	设备类型，详见“2.4.3 AudioDeviceType”小节。	输入
state	连接状态，详见“2.4.5 AudioDeviceConnectState”小节。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.1.20 ActivateAudioInterrupt

【描述】

激活音频中断，应用在使用音频资源前，向其他应用发出中断请求，根据反馈结果，合理使用音频资源，避免资源冲突。

【语法】

```
int32_t ActivateAudioInterrupt(AudioInterrupt &interrupt);
```

【参数】

参数名称	描述	输入/输出
interrupt	中断数据结构，详见“ 2.4.32 struct AudioInterrupt ”小节。	输入

【返回值】

返回值	描述	输入/输出
int32_t	- INTERRUPT_FAILED：中断拒绝。 - INTERRUPT_GRANTED：中断激活。 - INTERRUPT_DELAYED：中断延时。 - 其他：错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

当前应用激活音频中断成功后，如果有其他应用被打断，用户要等打断策略执行结束再使用音频资源，避免竞争。

典型场景：蓝牙音乐被蓝牙通话打断，要等音乐应用暂停流程结束，再创建通话通路（音乐应用暂停耗时长，通话应用可提示接听失败）。

【举例】

具体示例内容请参见“[2.6.1 本地音乐播放](#)”小节。

【相关主题】

DeactivateAudioInterrupt。

2.3.1.21 DeactivateAudioInterrupt

【描述】

去激活音频中断，表示应用不再使用音频资源，其他应用可激活音频中断，使用音频资源。

【语法】

```
int32_t DeactivateAudioInterrupt(AudioInterrupt &interrupt);
```

【参数】

参数名称	描述	输入/输出
interrupt	中断数据结构，详见“ 2.4.32 struct AudioInterrupt ”小节。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.1 本地音乐播放](#)”小节。

【相关主题】

ActivateAudioInterrupt。

2.3.1.22 DumpInfo

【描述】

Dump 信息。

【语法】

```
int32_t DumpInfo(AudioManagerDebugInfo &audioManagerInfo);
```

【参数】

参数名称	描述	输入/输出
audioManagerInfo	AudioManager 基本信息，详见“ 2.4.33 AudioManagerDebugInfo ”小节。	输出

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

维测类接口，非功能接口。

【举例】

无。

【相关主题】

无。

2.3.2 InterruptListener 类

当应用程序的音频被另一个应用程序中断时，回调以通知应用程序。提供以下 API：

OnInterrupt：中断处理回调函数。

2.3.2.1 OnInterrupt

【描述】

中断处理回调函数（纯虚函数），应用程序继承实现，当被另一个应用程序中断时，在该函数中做相应响应。

【语法】

virtual void OnInterrupt(int32_t type, int32_t hint) = 0;

【参数】

参数名称	描述	输入/输出
type	中断类型，详见“ 2.4.17 enum AudioInterruptType ”小节。	输入
hint	音频中断提示类型，详见“ 2.4.18 AudioInterruptHintType ”小节。	输入

【返回值】

无。

【芯片差异】

无。

【注意】

该函数运行在媒体内部创建的子线程中，应用程序在回调函数中应回到自身线程去做播控操作等。

【举例】

具体示例内容请参见“[2.6.1 本地音乐播放](#)”小节。

【相关主题】

无。

2.3.3 AudioStreamIn 类

音频输入流模块主要提供流控操作和数据获取等功能。提供以下 API：

- Init：初始化流信息。
- GetCurrentDeviceId：获取当前设备 ID。
- GetCurrentChannelId：获取当前通道 ID。
- AttachBackend：绑定后端输出流。
- GetTimestamp：获取时间戳。
- Start：启动流。
- Stop：停止流。
- SetInputDevice：设置输入设备。
- ObtainBuffer：获取数据。
- Release：释放资源。

2.3.3.1 Init

【描述】

初始化流信息。

【语法】

```
int32_t Init(const CapturerInputConfig &config, CapturerCallback cbk = nullptr, const void *caller = nullptr);
```

【参数】

参数名称	描述	输入/输出
config	采集配置，详见“ 2.4.21 CapturerInputConfig ”小节。	输入
cbk	采集回调函数指针，详见“ 2.4.23 CapturerCallback ”小节，默认 nullptr。	输入
caller	调用者指针，默认 nullptr。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.3.2 GetCurrentDeviceId

【描述】

获取当前设备 ID。

【语法】

int32_t GetCurrentDeviceId();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	当前设备 ID，详见“ 2.4.3 AudioDeviceType ”小节。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.3.3 GetCurrentChannelId

【描述】

获取当前通道 ID。

【语法】

```
int32_t GetCurrentChannelId(uint32_t &channelId);
```

【参数】

参数名称	描述	输入/输出
channelId	当前通道 ID。	输出

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

该接口与 AudioStreamOut 的 AttachFrontend 配合使用。

【举例】

具体示例内容请参见“2.6.3 蓝牙语音通话”小节。

【相关主题】

AudioStreamOut 的 AttachFrontend。

2.3.3.4 AttachBackend

【描述】

绑定后端输出流。

【语法】

int32_t AttachBackend(uint32_t &backendId);

【参数】

参数名称	描述	输入/输出
backendId	输出流 ID。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

该接口与 AudioStreamOut 的 GetCurrentChannelId 配合使用。

【举例】

具体示例内容请参见“2.6.3 蓝牙语音通话”小节。

【相关主题】

AudioStreamOut 的 GetCurrentChannelId。

2.3.3.5 GetTimestamp

【描述】

获取时间戳。

【语法】

```
int32_t GetTimestamp(struct timespec &timestamp, Timebase base);
```

【参数】

参数名称	描述	输入/输出
timestamp	C 标准库时间结构。	输出
base	预留参数。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功。 - 非 0: 失败, 错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.3.6 Start

【描述】

启动流。

【语法】

int32_t Start();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功。 - 非 0: 失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.3.7 Stop

【描述】

停止流。

【语法】

int32_t Stop();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.3.8 SetInputDevice

【描述】

设置输入设备。

【语法】

```
int32_t SetInputDevice(int32_t deviceId, AudioRendererConfig &renderConfig);
```

【参数】

参数名称	描述	输入/输出
deviceId	设备 ID，详见“ 2.4.3 AudioDeviceType ”小节。	输入
renderConfig	预留参数。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功; - 非 0: 失败, 错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

设备 ID 必须是输入设备的 ID。

【举例】

无。

【相关主题】

无。

2.3.3.9 ObtainBuffer

【描述】

获取数据。

【语法】

```
int32_t ObtainBuffer(CapturerBuffer &buffer, bool isBlocking);
```

【参数】

参数名称	描述	输入/输出
buffer	采集缓冲数据结构, 详见“ 2.4.24 CapturerBuffer ”小节。	输入
isBlocking	预留参数。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功;	输出

返回值	描述	输入/输出
	非 0：失败，错误码定义在 audio_errors.h。	

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.3.10 Release

【描述】

释放资源。

【语法】

int32_t Release();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.4 AudioStreamOut 类

音频输出流模块主要提供流控操作和数据发送等功能。提供以下 API：

- Init：初始化流信息。
- Play：启动流。
- Pause：暂停流。
- Stop：停止流。
- Release：释放资源。
- SetPlayBackSpeed：设置流速度。
- SetVolume：设置音量。
- GetCurrentDeviceId：获取当前设备 ID。
- GetCurrentChannelId：获取当前通道 ID。
- AttachFrontend：绑定前端输入流。
- StreamWrite：发送数据。
- GetPosition：获取当前流位置。
- GetTimestamp：获取当前流时间戳。
- Flush：刷新缓存。
- SetOutputDevice：设置输出设备。
- GetBufferFrameSize：获取缓冲帧数。
- GetBufferUnderflowTimes：获取缓冲欠载次数。
- DumpInfo：Dump 信息。
- SetParam：设置参数。

2.3.4.1 Init

【描述】

初始化流信息。

【语法】

```
int32_t Init(AudioRendererConfig &config, RendererCallback cbk = nullptr, const void  
*caller = nullptr);
```

【参数】

参数名称	描述	输入/输出
config	渲染配置, 详见“ 2.4.20 AudioRendererConfig ”小节。	输入
cbk	渲染回调函数指针, 详见“ 2.4.22 RendererCallback ”小节, 默认 nullptr。	输入
caller	调用者指针, 默认 nullptr。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功。 - 非 0: 失败, 错误码定义在 <code>audio_errors.h</code>。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.4.2 Play

【描述】

启动流。

【语法】

int32_t Play();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功。 - 非 0: 失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.4.3 Pause

【描述】

暂停流。

【语法】

int32_t Pause();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.4.4 Stop

【描述】

停止流。

【语法】

int32_t Stop();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。	输出

返回值	描述	输入/输出
	非 0：失败，错误码定义在 <code>audio_errors.h</code>。	

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.4.5 Release

【描述】

释放资源。

【语法】

```
int32_t Release();
```

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 <code>audio_errors.h</code>。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“2.6.3 蓝牙语音通话”小节。

【相关主题】

无。

2.3.4.6 SetPlayBackSpeed

【描述】

设置流速度。

【语法】

int32_t SetPlayBackSpeed(float speed);

【参数】

参数名称	描述	输入/输出
speed	播放速度。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.4.7 SetVolume

【描述】

设置音量。

【语法】

int32_t SetVolume(float volume);

【参数】

参数名称	描述	输入/输出
volume	音量值，范围：0.0 ~ 1.0。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.4.8 GetCurrentDeviceId

【描述】

获取当前设备 ID。

【语法】

int32_t GetCurrentDeviceId();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	当前设备 ID，详见“2.4.3 AudioDeviceType”小节。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.4.9 GetCurrentChannelId

【描述】

获取当前通道 ID。

【语法】

int32_t GetCurrentChannelId(uint32_t &channelId);

【参数】

参数名称	描述	输入/输出
channelId	当前通道 ID。	输出

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功。 - 非 0: 失败, 错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

该接口与 AudioStreamIn 的 AttachBackend 配合使用。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

AudioStreamIn 的 AttachBackend。

2.3.4.10 AttachFrontend

【描述】

绑定前端输入流。

【语法】

```
int32_t AttachFrontend(uint32_t &frontendId);
```

【参数】

参数名称	描述	输入/输出
frontendId	输入流 ID。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

该接口与 AudioStreamIn 的 GetCurrentChannelId 配合使用。

【举例】

具体示例内容请参见“2.6.3 蓝牙语音通话”小节。

【相关主题】

AudioStreamIn 的 GetCurrentChannelId。

2.3.4.11 StreamWriter

【描述】

发送数据。

【语法】

int32_t StreamWriter(const uint8_t *buffer, uint64_t userSize, uint64_t &writtenSize, AudioRendererConfig &renderConfig);

【参数】

参数名称	描述	输入/输出
buffer	数据地址。	输入
userSize	数据大小。	输入
writtenSize	发送成功数据大小。	输出
renderConfig	预留参数。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0: 成功。 - 非 0: 失败, 错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[2.6.3 蓝牙语音通话](#)”小节。

【相关主题】

无。

2.3.4.12 GetPosition

【描述】

获取当前流位置。

【语法】

```
int64_t GetPosition();
```

【参数】

无。

【返回值】

返回值	描述	输入/输出
int64_t	0: 播放位置, 单位: ns。 - 非 0: 失败, 错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.4.13 GetTimestamp

【描述】

获取当前流时间戳。

【语法】

```
int32_t GetTimestamp(struct timespec &timeStamp);
```

【参数】

参数名称	描述	输入/输出
timestamp	时间结构，详见 struct timespec。	输出

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.4.14 Flush

【描述】

刷新缓存。

【语法】

int32_t Flush();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.3.4.15 SetOutputDevice

【描述】

设置输出设备。

【语法】

```
int32_t SetOutputDevice(int32_t deviceId, AudioRendererConfig &renderConfig);
```

【参数】

参数名称	描述	输入/输出
deviceId	设备 ID，详见“2.4.3 AudioDeviceType”小节。	输入
renderConfig	预留参数。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

设备 ID 必须是输出设备的 ID。

【举例】

无。

【相关主题】

无。

2.3.4.16 GetBufferFrameSize

【描述】

获取缓冲帧数。

【语法】

int32_t GetBufferFrameSize();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	预留。	输出

【芯片差异】

无。

【注意】

预留接口，功能未实现。

【举例】

无。

【相关主题】

无。

2.3.4.17 GetBufferUnderflowTimes

【描述】

获取缓冲欠载次数。

【语法】

int32_t GetBufferUnderflowTimes();

【参数】

无。

【返回值】

返回值	描述	输入/输出
int32_t	预留。	输出

【芯片差异】

无。

【注意】

预留接口，功能未实现。

【举例】

无。

【相关主题】

无。

2.3.4.18 DumpInfo

【描述】

Dump 信息。

【语法】

StreamDebugInfo DumpInfo();

【参数】

无。

【返回值】

返回值	描述	输入/输出
StreamDebugInfo	流信息，详见“2.4.27 StreamDebugInfo”小节。	输出

【芯片差异】

无。

【注意】

维测类接口，非功能接口。

【举例】

无。

【相关主题】

无。

2.3.4.19 SetParam

【描述】

设置参数。

【语法】

int32_t SetParam(const char *keyValueList, int32_t len);

【参数】

参数名称	描述	输入/输出
keyValueList	参数键值对列表（格式：key = value，参数间用空格分隔）。	输入
len	参数键值对长度。	输入

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

无。

【举例】

无。

【相关主题】

无。

2.4 数据类型、数据结构

- AudioSession: 音频会话 ID 数据类型。
- DeviceFlag: 设备标志位。
- AudioDeviceType: 音频设备类型。
- AudioDeviceInfo: 音频设备信息。
- AudioDeviceConnectState: 音频设备连接状态。
- AudioRingMode: 音频振铃模式。
- AudioSourceType: 音频源类型。
- AudioDeviceDesc: 音频设备描述信息。
- AudioCodecFormat: 音频编解码格式。
- AudioStreamType: 音频流类型。
- AudioLinkDirection: 音频链路方向。
- AudioStreamFlag: 音频流标志位。
- AudioChannelMode: 音频声道模式。
- AudioBitWidth: 音频位宽。
- AudioOffloadInfo: 音频 Offload 信息。
- AudioStrategy: 音频策略。
- enum AudioInterruptType: 音频中断类型。
- AudioInterruptHintType: 音频中断提示类型。
- AudioStreamInfo: 音频流信息。
- AudioRendererConfig: 音频渲染配置。
- CapturerInputConfig: 音频采集配置。
- RendererCallback: 渲染回调函数指针。
- CapturerCallback: 采集回调函数指针。
- CapturerBuffer: 采集 Buffer。
- InterruptDebugInfo: 中断信息。

- DeviceDebugInfo：设备信息。
- StreamDebugInfo：流信息。
- AudioInterruptDebugInfo：音频中断信息汇总。
- AudioStreamDebugInfo：音频流信息汇总。
- AudioDeviceDebugInfo：音频设备信息汇总。
- AudioServiceDebugInfo：音频服务信息。
- struct AudioInterrupt：音频中断。
- AudioManagerDebugInfo：音频管理信息。

2.4.1 AudioSession

【说明】

会话 ID 数据类型。

【定义】

```
typedef uint32_t AudioSession;
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

```
enum {  
    AUDIO_SESSION_ID_NONE    = 0, // 无效会话 ID  
};  
AudioManager::MakeSessionId();  
AudioManager::ActivateAudioInterrupt(AudioInterrupt &interrupt);
```

2.4.2 DeviceFlag

【说明】

设备标志位。

【定义】

```
typedef enum {  
    OUTPUT_DEVICES_FLAG      = 0x1u,
```

```
INPUT_DEVICES_FLAG      = 0x2u,  
} DeviceFlag;
```

【成员】

成员名称	描述
OUTPUT_DEVICES_FLAG	输出设备标志位。
INPUT_DEVICES_FLAG	输入设备标志位。

【注意事项】

无。

【相关数据类型及接口】

```
std::vector<AudioDeviceInfo> GetDevices(DeviceFlag flag);
```

2.4.3 AudioDeviceType

【说明】

音频设备类型。

【定义】

```
typedef enum {  
    UNKNOWN = 0x0,  
    IN_FLAG = 0x80000000,  
    OUT_BUILTIN_SPEAKER = 0x2,  
    OUT_BLUETOOTH_SCO = 0x10,  
    OUT_BLUETOOTH_A2DP = 0x80,  
    OUT_MODEM = 0x100,  
    OUT_MODEM_HEADSET = 0x200,  
    IN_BUILTIN_MIC = IN_FLAG | 0x4,  
    IN_BLUETOOTH_SCO = IN_FLAG | 0x8,  
    IN_BLUETOOTH_A2DP = IN_FLAG | 0x20,  
    IN_MODEM = IN_FLAG | 0x40,  
    IN_MODEM_HEADSET = IN_FLAG | 0x400,  
} AudioDeviceType;
```

【成员】

成员名称	描述
UNKNOWN	未知设备。
IN_FLAG	输入设备标志位。
OUT_BUILTIN_SPEAKER	内置扬声器输出设备。
OUT_BLUETOOTH_SCO	蓝牙 SCO 链路输出设备。
OUT_BLUETOOTH_A2DP	蓝牙 A2DP 协议输出设备。
OUT_MODEM	本地 MODEM 输出设备。
OUT_MODEM_HEADSET	连接蓝牙设备时，MODEM 输出设备。
IN_BUILTIN_MIC	内置麦克风输入设备。
IN_BLUETOOTH_SCO	蓝牙 SCO 链路输入设备。
IN_BLUETOOTH_A2DP	蓝牙 A2DP 协议输入设备。
IN_MODEM	本地 MODEM 输入设备。
IN_MODEM_HEADSET	连接蓝牙设备时，MODEM 输入设备。

【注意事项】

无。

【相关数据类型及接口】

```
int32_t SetDeviceConnectionState(AudioDeviceType device, AudioDeviceConnectState state);
```

2.4.4 AudioDeviceInfo

【说明】

音频设备信息。

【定义】

```
typedef struct {
    DeviceFlag      flag;          /* enum DeviceRole */
    AudioDeviceType device;        /* device type */
    const char      *name;
} AudioDeviceInfo;
```

【成员】

成员名称	描述
flag	设备标志位, 详见“2.4.2 DeviceFlag”小节。
device	音频设备类型, 详见“2.4.3 AudioDeviceType”小节。
name	设备名称。

【注意事项】

无。

【相关数据类型及接口】

```
std::vector<AudioDeviceInfo> GetDevices(DeviceFlag flag);
```

2.4.5 AudioDeviceConnectState

【说明】

音频设备连接状态。

【定义】

```
typedef enum {  
    AUDIO_DEVICE_CONNECTED      = 0x0u,  
    AUDIO_DEVICE_DISCONNECTED   = 0x1u,  
} AudioDeviceConnectState;
```

【成员】

成员名称	描述
AUDIO_DEVICE_CONNECTED	音频设备连接。
AUDIO_DEVICE_DISCONNECTED	音频设备断开。

【注意事项】

无。

【相关数据类型及接口】

```
int32_t SetDeviceConnectionState(AudioDeviceType device, AudioDeviceConnectState state);
```

2.4.6 AudioRingMode

【说明】

音频振铃模式。

【定义】

```
/**
 * @brief Describes ringer modes.
 *
 * @since 1.0
 * @version 1.0
 */
typedef enum {
    /**
     * Indicates that the ringer mode is restored to the previous setting. A device will be audible if
     the previous mode was sound.
     */
    RINGER_MODE_NORMAL = 0,
    /**
     * Indicates that a device is silent and vibrates. For calls, the device will vibrate,
     * and for message notifications, the vibration can be enabled by users.
     */
    RINGER_MODE_VIBRATE,
    /**
     * Indicates that a device is silent and does not vibrate. This mode overrides the vibration mode.
     */
    RINGER_MODE_SILENT,
    /**
     * Invalid value
     */
    RINGER_MODE_INVALID,
} AudioRingMode;
```

【成员】

成员名称	描述
RINGER_MODE_NORMAL	正常模式：恢复到以前的设置，如果上个模式是响铃，可以听到设备的声音。
RINGER_MODE_VIBRATE	振动模式：静音并振动，通话场景，设备将振动；消息通知场景，用户可以启用振动。

成员名称	描述
RINGER_MODE_SILENT	静音模式：静音不振动，此模式会覆盖振动模式。
RINGER_MODE_INVALID	无效模式。

【注意事项】

无。

【相关数据类型及接口】

```
bool SetRingerMode(AudioRingMode mode);
int32_t GetRingerMode();
```

2.4.7 AudioSourceType

【说明】

音频源类型。

【定义】

```
/**
 * @brief Enumerates audio source types.
 *
 * @since 1.0
 * @version 1.0
 */
typedef enum {
    /** Invalid audio source */
    AUDIO_SOURCE_INVALID = -1,
    /** Default audio source */
    AUDIO_SOURCE_DEFAULT = 0,
    /** Microphone */
    AUDIO_MIC = 1,
    /** Uplink voice */
    AUDIO_VOICE_UPLINK = 2,
    /** Downlink voice */
    AUDIO_VOICE_DOWNLINK = 3,
    /** Voice call */
    AUDIO_VOICE_CALL = 4,
    /** Camcorder */
    AUDIO_CAMCORDER = 5,
```

```

/** Voice recognition */
AUDIO_VOICE_RECOGNITION = 6,
/** Voice communication */
AUDIO_VOICE_COMMUNICATION = 7,
/** Remote submix */
AUDIO_REMOTE_SUBMIX = 8,
/** Unprocessed audio */
AUDIO_UNPROCESSED = 9,
/** Voice performance */
AUDIO_VOICE_PERFORMANCE = 10,
/** Echo reference */
AUDIO_ECHO_REFERENCE = 1997,
/** Radio tuner */
AUDIO_RADIO_TUNER = 1998,
/** Hotword */
AUDIO_HOTWORD = 1999,
/** Extended remote submix */
AUDIO_REMOTE_SUBMIX_EXTEND = 10007,
} AudioSourceType;

```

【成员】

成员名称	描述
AUDIO_SOURCE_INVALID	无效类型。
AUDIO_SOURCE_DEFAULT	默认源类型，对应麦克风。
AUDIO_MIC	麦克风。
AUDIO_VOICE_UPLINK	不支持。
AUDIO_VOICE_DOWNLINK	不支持。
AUDIO_VOICE_CALL	不支持。
AUDIO_CAMCORDER	不支持。
AUDIO_VOICE_RECOGNITION	不支持。
AUDIO_VOICE_COMMUNICATION	不支持。
AUDIO_REMOTE_SUBMIX	不支持。
AUDIO_UNPROCESSED	不支持。

成员名称	描述
AUDIO_VOICE_PERFORMANCE	不支持。
AUDIO_ECHO_REFERENCE	不支持。
AUDIO_RADIO_TUNER	不支持。
AUDIO_HOTWORD	不支持。
AUDIO_REMOTE_SUBMIX_EXTEND	不支持。

【注意事项】

该数据结构仅在音频录制场景使用。

【相关数据类型及接口】

无。

2.4.8 AudioDeviceDesc

【说明】

音频设备描述信息。

【定义】

```
/**
 * @brief Defines the audio Device Descriptor.
 *
 * @since 1.0
 * @version 1.0
 */
typedef struct {
    /** Device name */
    uint8_t *deviceName; // todo: std::string --> uint8_t *
    /** Type of the audio input source */
    AudioSourceType inputSourceType;
    /** Bits 31-24: reserved bits; bits 23-16: mode ID; bits 15-8: device ID; bits 7-0: channel ID */
    uint32_t deviceId;
} AudioDeviceDesc;
```

【成员】

成员名称	描述
deviceName	设备名称。
inputSourceType	输入源类型，详见“2.4.7 AudioSourceType”小节。
deviceId	设备 ID。

【注意事项】

该数据结构仅用在音频录制场景。

【相关数据类型及接口】

无。

2.4.9 AudioCodecFormat

【说明】

音频编解码格式。

【定义】

```
/**
 * @brief Enumerates audio codec formats.
 *
 * @since 1.0
 * @version 1.0
 */
typedef enum {
    /** Default format */
    AUDIO_DEFAULT    = 0,
    /** PCM */
    PCM              = 1,
    /** Advanced Audio Coding Low Complexity (AAC-LC) */
    AAC_LC           = 2,
    /** High-Efficiency Advanced Audio Coding (AAC-HE), previously known as AAC+ or aacPlus v1 */
    AAC_HE_V1        = 3,
    /** AAC++ or aacPlus v2 */
    AAC_HE_V2        = 4,
    /** Advanced Audio Coding Low Delay (AAC-LD) */
    AAC_LD           = 5,
    /** Advanced Audio Coding Enhanced Low Delay (AAC-ELD) */
}
```

```

AAC_ELD          = 6,
/** G711 a-law */
G711A            = 7,
/** G711 u-law */
G711U           = 8,
/** G726 */
G726             = 9,
/** Opus */
OPUS             = 10,
/** Flac */
FLAC             = 11,
/** Vorbis */
VORBIS           = 12,
/** Ape */
APE              = 13,
/** Mp3 */
MP3              = 14,
/** mSBC */
mSBC             = 15,
SILK             = 16,
SBC              = 17,
AMR_WB           = 18,
/** Invalid value */
FORMAT_INVALID,
} AudioCodecFormat;

```

【成员】

成员名称	描述	当前是否支持
AUDIO_DEFAULT	默认格式，对应 PCM。	NA
PCM	脉冲编码调制。	是
AAC_LC	高级音频编码低复杂度。	是
AAC_HE_V1	高效高级音频编码以前称为 AAC+或 aacPlus v1，目前不支持。	否
AAC_HE_V2	高效高级音频编码以前称为 AAC+或 aacPlus v2，目前不支持。	否
AAC_LD	高级音频编码低延迟，目前不支持。	否

成员名称	描述	当前是否支持
AAC_ELD	高级音频编码增强低延迟，目前不支持。	否
G711A	G711 a-law。	否
G711U	G711 u-law。	否
G726	G726。	否
OPUS	Opus。	是
FLAC	Flac。	是
VORBIS	Vorbis。	否
APE	Ape。	否
MP3	Mp3。	是
mSBC	msbc。	是
SILK	silk。	是
SBC	sbc。	是
AMR_WB	AMR_WB。	是
FORMAT_INVALID	无效格式。	NA

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.10 AudioStreamType

【说明】

音频流类型。

【定义】

/**

* @brief 音频流类型定义.

*

* @since 1.0

* @version 1.0

*/

// 0xFFFF FFFFu 高8位表示基础类型分类 低8位 子类

typedef enum {

/** Default audio stream type */

TYPE_DEFAULT = -1,

/** Media */

TYPE_MEDIA = 0,

/** Voice call */

TYPE_VOICE_COMMUNICATION = 1,

AUDIO_STREAM_NONE = 0x0u, /**< None */

AUDIO_STREAM_ALARM = 0x00010000u, /**< 报警类 */

AUDIO_STREAM_ALARM_SYSTEM = 0x00010001u, /**< 设备报警:

手表温度过高报警、车机上发动机故障报警、转向系统报警、刹车系统故障报警等,健康报警:心率过高、环境噪声报警 */

AUDIO_STREAM_ALARM_CLOCK = 0x00010002u, /**< 闹钟 */

AUDIO_STREAM_RING = 0x00020000u, /**< 来电铃声 */

AUDIO_STREAM_VOICE_CALL = 0x00040000u, /**< 语音通话 */

AUDIO_STREAM_VOICE_CALL_VOIP = 0x00040001u, /**< 语音通话 */

AUDIO_STREAM_VOICE_CALL_BT_SCO = 0x00040002u, /**< 蓝牙通话 */

AUDIO_STREAM_VOICE_CALL_VOLTE = 0x00040003u, /**< 4G VOLTE

通话 */

AUDIO_STREAM_VOICE_ASSISTANT = 0x00080004u, /**< 语音助手 */

AUDIO_STREAM_TTS = 0x00100000u, /**< 语音合成 */

AUDIO_STREAM_NOTIFICATION = 0x00200000u, /**< 提示音 */

AUDIO_STREAM_NOTIFICATION_SYSTEM = 0x00200001u, /**< 设备提示音:

蓝牙断开、连接提示音 低电量 LOW POWER 消息提示音 MESSAGE*/

AUDIO_STREAM_NOTIFICATION_PROMPT = 0x00200002u, /**< 运动健康提

示音 */

```

AUDIO_STREAM_MUSIC                = 0x00400000u,    /**< 媒体音 */
AUDIO_STREAM_A2DP_MUSIC            = 0x00400001u,    /**< 蓝牙输入媒体
音 */
AUDIO_STREAM_VOICE_RECORD          = 0x00800000u,    /**< 语音采集录
制 */
AUDIO_STREAM_VOICE_RECOGNITION     = 0x01000000u,    /**< 语音识别 */
AUDIO_STREAM_INVALID               = 0x7FFFFFFFu     /**< 无效流 */
} AudioStreamType;

```

【成员】

成员名称	描述
TYPE_DEFAULT	默认类型，按照 TYPE_MEDIA 处理。
TYPE_MEDIA	媒体声，采集场景对应从麦克风接收到的声音。
TYPE_VOICE_COMMUNICATION	语音通话声音。
AUDIO_STREAM_NONE	无。
AUDIO_STREAM_ALARM	报警类。
AUDIO_STREAM_ALARM_SYSTEM	设备报警：手表温度过高报警、车机上发动机故障报警、转向系统报警、刹车系统故障报警等。 健康报警：心率过高、环境噪声报警。
AUDIO_STREAM_ALARM_CLOCK	闹钟。
AUDIO_STREAM_RING	来电铃声。
AUDIO_STREAM_VOICE_CALL	语音通话。
AUDIO_STREAM_VOICE_CALL_VOIP	VOIP 语音通话。
AUDIO_STREAM_VOICE_CALL_BT_SC O	蓝牙通话。
AUDIO_STREAM_VOICE_CALL_VOLTE	4G VOLTE 通话。
AUDIO_STREAM_VOICE_ASSISTANT	语音助手。

成员名称	描述
AUDIO_STREAM_TTS	语音合成。
AUDIO_STREAM_NOTIFICATION	提示音。
AUDIO_STREAM_NOTIFICATION_SYSTEM	设备提示音：蓝牙断开/连接提示音、低电量 LOW POWER 消息提示音 MESSAGE。
AUDIO_STREAM_NOTIFICATION_PROMPT	运动健康提示音。
AUDIO_STREAM_MUSIC	音乐播放。
AUDIO_STREAM_A2DP_MUSIC	蓝牙输入媒体音（A2DP sink）。
AUDIO_STREAM_VOICE_RECORD	语音采集录制。
AUDIO_STREAM_VOICE_RECOGNITION	语音识别。
AUDIO_STREAM_INVALID	无效流。

【注意事项】

下面类型仅用在音频录制场景：

TYPE_DEFAULT、TYPE_MEDIA、TYPE_VOICE_COMMUNICATION。

【相关数据类型及接口】

```
bool SetVolume(AudioStreamType streamType, int32_t volume);
int32_t GetVolume(AudioStreamType streamType);
int32_t GetMinVolume(AudioStreamType streamType);
int32_t GetMaxVolume(AudioStreamType streamType);
bool Mute(AudioStreamType streamType);
bool UnMute(AudioStreamType streamType);
bool IsMute(AudioStreamType streamType);
```

2.4.11 AudioLinkDirection

【说明】

音频链路方向。

【定义】

```
/**
 * Describes audio stream Link Direction.
 *
 * @since 1
 */
typedef enum {
    AUDIO_DOWN_LINK,
    AUDIO_UP_LINK,
    AUDIO_LINK_NONE
} AudioLinkDirection;
```

【成员】

成员名称	描述
AUDIO_DOWN_LINK	音频下行链路。
AUDIO_UP_LINK	音频上行链路。
AUDIO_LINK_NONE	无。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.12 AudioStreamFlag

【说明】

音频流标志位。

【定义】

```
/**
 * Describes audio stream flags.
 *
 * @since 1
 */
typedef enum {
    AUDIO_STREAM_FLAG_NONE = 0x0,
    AUDIO_STREAM_FLAG_LOW_LATENCY = 0x1,
    AUDIO_STREAM_FLAG_SECURE = 0x2,
```

```
} AudioStreamFlag;
```

【成员】

成员名称	描述
AUDIO_STREAM_FLAG_NONE	无。
AUDIO_STREAM_FLAG_LOW_LATENCY	低延迟通路标志。
AUDIO_STREAM_FLAG_SECURE	安全通路标志。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.13 AudioChannelMode

【说明】

音频声道模式。

【定义】

```
/**
 * @brief Enumerates the audio channel mode.
 *
 * A mask describes an audio channel position.
 */
typedef enum {
    /**
     * Indicates an invalid channel mask.
     */
    AUDIO_CHANNEL_MODE_NONE = 0x0u,
    /**
     * Indicates the mask of the front left output channel.
     */
    AUDIO_CHANNEL_MODE_FRONT_LEFT = 0x1u,
    /**
     * Indicates the mask of the front right output channel.
     */
    AUDIO_CHANNEL_MODE_FRONT_RIGHT = 0x2u,
```

```
/**
 * Indicates the single output channel mask.
 */
AUDIO_CHANNEL_MODE_MONO = 0x4u,
/**
 * Indicates the dual output channel mask of
 * {@code AUDIO_CHANNEL_FRONT_LEFT | AUDIO_CHANNEL_RIGHT}.
 */
AUDIO_CHANNEL_MODE_STEREO = 0x8u,
} AudioChannelMode;
```

【成员】

成员名称	描述
AUDIO_CHANNEL_MODE_NONE	无效音频声道掩码。
AUDIO_CHANNEL_MODE_FRONT_LEFT	左前输出声道掩码。
AUDIO_CHANNEL_MODE_FRONT_RIGHT	右前输出声道掩码。
AUDIO_CHANNEL_MODE_MONO	单输出声道掩码。
AUDIO_CHANNEL_MODE_STEREO	双输出声道掩码。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.14 AudioBitWidth

【说明】

音频位宽。

【定义】

```
/**
 * @brief Enumerates audio bit widths.
 *
 * @since 1.0
 * @version 1.0
```

```
*/
typedef enum {
    /** 8-bit width */
    BIT_WIDTH_8    = 8,
    /** 16-bit width */
    BIT_WIDTH_16   = 16,
    /** 24-bit width */
    BIT_WIDTH_24   = 24,
    /** 32-bit width */
    BIT_WIDTH_32   = 32,
    /** Invalid value */
    BIT_WIDTH_BUTT,
} AudioBitWidth;
```

【成员】

成员名称	描述
BIT_WIDTH_8	8bit 位宽。
BIT_WIDTH_16	16bit 位宽。
BIT_WIDTH_24	24bit 位宽。
BIT_WIDTH_32	32bit 位宽。
BIT_WIDTH_BUTT	无效位宽。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.15 AudioOffloadInfo

【说明】

音频 Offload 信息。

【定义】

```
typedef struct {
    AudioStreamType streamType;
```

```
uint32_t    samplerate;  
AudioCodecFormat format;  
uint32_t    channel;  
uint32_t    bitwidth;  
} AudioOffloadInfo;
```

【成员】

成员名称	描述
streamType	音频流类型，详见“ 2.4.10 AudioStreamType ”小节。
samplerate	采样率。
format	音频编解码格式，详见“ 2.4.9 AudioCodecFormat ”小节。
channel	声道数。
bitwidth	位宽，详见“ 2.4.14 AudioBitWidth ”小节。

【注意事项】

无。

【相关数据类型及接口】

```
AudioManager::IsSupportOffload(AudioOffloadInfo &info);
```

2.4.16 AudioStrategy

【说明】

音频策略。

【定义】

```
/**  
 * @brief 音频策略定义。  
 *  
 * @since 1.0  
 * @version 1.0  
 */  
typedef enum {  
    /* NA同级别流策略，或者不存在此种流组合 */  
    STRATEGY_NONE        = 0x0u,
```

```
/* 打断、排他，当返回该策略时需要给使用者回调INTERRUPT的事件并停止接收之前流的数据 */
STRATEGY_EXCLUSIVE = 0x1u,
/* 混音（第一条流权重100%，每次增加需要混音的流权重为50%，之前的每条流按比例共享剩下的50%） */
STRATEGY_MIX = 0x2u,
/* 延时，当前流延时获取音频焦点 */
STRATEGY_DELAY = 0x3u,
/* 不播放，当前无法获取音频焦点 */
STRATEGY_REJECT = 0x4u
} AudioStrategy;
```

【成员】

成员名称	描述
STRATEGY_NONE	同级别流策略，或者不存在此种流组合。
STRATEGY_EXCLUSIVE	打断、排他，当返回该策略时需要给使用者回调INTERRUPT的事件并停止接收之前流的数据。
STRATEGY_MIX	混音（第一条流权重 100%，每次增加需要混音的流权重为 50%，之前的每条流按比例共享剩下的 50%）。
STRATEGY_DELAY	延时，当前流延时获取音频焦点。
STRATEGY_REJECT	不播放，当前无法获取音频焦点。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.17 enum AudioInterruptType

【说明】

音频中断类型。

【定义】

```
/**
 * Indicates that the interruption types.
 *
 * @since 1
 */
enum AudioInterruptType {
    /**
     * Indicates that the interruption starts.
     *
     * @since 1
     */
    INTERRUPT_TYPE_BEGIN = 1,
    /**
     * Indicates that the interruption ends.
     *
     * @since 1
     */
    INTERRUPT_TYPE_END
};
```

【成员】

成员名称	描述
INTERRUPT_TYPE_BEGIN	中断开始。
INTERRUPT_TYPE_END	中断结束。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.18 AudioInterruptHintType

【说明】

音频中断提示类型。

【定义】

```
/**
 * Indicates that the interruption hint types.
 *
 * @since 1
 */
enum AudioInterruptHintType {
    /**
     * Indicates a prompt for the application to pause audio playback.
     *
     * @since 1
     */
    INTERRUPT_HINT_PAUSE = 0,
    /**
     * Indicates a prompt for the application to resume audio playback.
     *
     * @since 1
     */
    INTERRUPT_HINT_RESUME,
    /**
     * Indicates a prompt for the application to stop audio playback.
     *
     * @since 1
     */
    INTERRUPT_HINT_STOP
};
```

【成员】

成员名称	描述
INTERRUPT_HINT_PAUSE	提示应用程序暂停音频播放。
INTERRUPT_HINT_RESUME	提示应用程序恢复音频播放。
INTERRUPT_HINT_STOP	提示应用程序停止音频播放。

【注意事项】

音频管理提示恢复播放时，应用层面可根据自身逻辑选择遵从或忽略，例如用户暂停播放时接打电话，待电话结束后，播放应用会收到音频管理恢复提示，当前可根据需要，选择是否恢复，通常保持暂停。

【相关数据类型及接口】

无。

2.4.19 AudioStreamInfo

【说明】

音频流信息。

【定义】

```
/**
 * Provides audio stream information.
 *
 * <p>This class provides initialized parameters of {@code AudioCapturer} and {@code
 AudioRenderer},
 * including the stream encoding format, sampling rate, audio channel, usage type, session ID,
 * buffer size, clearing flag, audio source, and stream flags.
 *
 * @since 1
 */
typedef struct {
    AudioStreamType streamType;
    AudioSession sessionID;
    AudioCodecFormat format;
    uint32_t sampleRate;
    int32_t bitRate;
    /** Number of audio channels */
    int32_t channelCount;
    AudioChannelMode channelMask;
    AudioStreamFlag audioStreamFlag;
    AudioLinkDirection linkDir;
    uint16_t sampleFmt;
} AudioStreamInfo;
```

【成员】

成员名称	描述
streamType	音频流类型，详见“ 2.4.10 AudioStreamType ”小节。
sessionID	音频会话 ID，详见“ 2.4.1 AudioSession ”小节。
format	音频编解码格式，详见“ 2.4.9 AudioCodecFormat ”小节。
sampleRate	采样率。

成员名称	描述
bitRate	码率。
channelCount	声道数。
channelMask	声道模式，详见“2.4.13 AudioChannelMode”小节。
audioStreamFlag	音频流标志位，详见“2.4.12 AudioStreamFlag”小节。
linkDir	音频链路方向，详见“2.4.11 AudioLinkDirection”小节。
sampleFmt	采样格式。1： pcm 16bit； 2： pcm 24bit。

【注意事项】

无。

【相关数据类型及接口】

无。

2.4.20 AudioRendererConfig

【说明】

音频渲染配置。

【定义】

```
/**
 * Provides audio stream out config.
 *
 * <p>This class provides initialized parameters of {@code AudioCapturer} and {@code
 * AudioRenderer},
 * including the stream encoding format, sampling rate, audio channel, usage type, session ID,
 * buffer size, clearing flag, audio source, and stream flags.
 *
 * @since 1
 */
typedef struct {
    AudioCodecFormat audioFormat;
    AudioStreamType streamType;
    uint32_t sampleRate;
    int32_t sessionID;
    uint32_t channelCount;
```

```
AudioLinkDirection linkDir;
uint16_t sampleFmt;
} AudioRendererConfig;
```

【成员】

成员名称	描述
audioFormat	音频编解码格式，详见“2.4.9 AudioCodecFormat”小节。
streamType	音频流类型，详见“2.4.10 AudioStreamType”小节。
sampleRate	采样率。
sessionId	音频会话 ID，详见“2.4.1 AudioSession”小节。
channelCount	声道数。
linkDir	音频链路方向，详见“2.4.11 AudioLinkDirection”小节。
sampleFmt	采样格式。1：pcm 16bit；2：pcm 24bit。

【注意事项】

无。

【相关数据类型及接口】

```
int32_t AudioStreamOut::Init(AudioRendererConfig &config, RendererCallback cbk, const void
*caller);
```

2.4.21 CapturerInputConfig

【说明】

音频采集配置。

【定义】

```
typedef struct {
    AudioCodecFormat audioFormat;
    AudioStreamType streamType;
    uint32_t sampleRate;
    int32_t sessionId;
    uint32_t channelCount;
    AudioLinkDirection linkDir;
} CapturerInputConfig;
```

【成员】

成员名称	描述
audioFormat	音频编解码格式，详见“2.4.9 AudioCodecFormat”小节。
streamType	音频流类型，详见“2.4.10 AudioStreamType”小节。
sampleRate	采样率。
sessionID	音频会话 ID，详见“2.4.1 AudioSession”小节。
channelCount	声道数。
linkDir	音频链路方向，详见“2.4.11 AudioLinkDirection”小节。

【注意事项】

无。

【相关数据类型及接口】

```
int32_t AudioStreamIn::Init(const CapturerInputConfig &config, CapturerCallback cbk, const void *caller);
```

2.4.22 RendererCallback

【说明】

渲染回调函数指针。

【定义】

```
typedef void (*RendererCallback)(void *caller, int32_t event, void *info);
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

```
int32_t AudioStreamOut::Init(AudioRendererConfig &config, RendererCallback cbk, const void *caller);
```

2.4.23 CapturerCallback

【说明】

采集回调函数指针。

【定义】

```
typedef void (*CapturerCallback)(void *caller, int event, void *info);
```

【成员】

无。

【注意事项】

无。

【相关数据类型及接口】

```
int32_t AudioStreamIn::Init(const CapturerInputConfig &config, CapturerCallback cbk, const void *caller);
```

2.4.24 CapturerBuffer

【说明】

采集 Buffer。

【定义】

```
typedef struct {  
    uint32_t frameCount;    // number of sample frames corresponding to size;  
    uint32_t size;          // input/output in bytes == frameCount * frameSize | userSize >=  
                             frameCount * channelCount * BytesPerSample  
    uint8_t *data;          // 调用者分配内存  
} CapturerBuffer;
```

【成员】

成员名称	描述
frameCount	预留参数。
size	采集 Buffer 大小。
data	采集 Buffer 地址。

【注意事项】

无。

【相关数据类型及接口】

```
int32_t AudioStreamIn::ObtainBuffer(CapturerBuffer &buffer, bool isBlocking);
```

2.4.25 InterruptDebugInfo

【说明】

中断信息。

【定义】

```
typedef struct {  
    AudioStreamType streamType;  
    AudioStrategy  currentStrategy;  
    AudioSession sessionID;  
    int64_t streamCreateTime;  
    int64_t streamDestroyTime;  
} InterruptDebugInfo;
```

【成员】

成员名称	描述
streamType	音频流类型，详见“2.4.10 AudioStreamType”小节。
currentStrategy	音频策略，详见“2.4.16 AudioStrategy”小节。
sessionID	音频会话 ID，详见“2.4.1 AudioSession”小节。
streamCreateTime	音频流创建时间。
streamDestroyTime	音频流销毁时间。

【注意事项】

无。

【相关数据类型及接口】

AudioManagerDebugInfo。

2.4.26 DeviceDebugInfo

【说明】

设备信息。

【定义】

```
typedef struct {
    AudioDeviceInfo    deviceInfo;
    AudioDeviceConnectState    deviceState;
} DeviceDebugInfo;
```

【成员】

成员名称	描述
deviceInfo	音频设备信息，详见“2.4.4 AudioDeviceInfo”小节。
deviceState	音频设备状态，详见“2.4.5 AudioDeviceConnectState”小节。

【注意事项】

无。

【相关数据类型及接口】

```
AudioManagerDebugInfo.
```

2.4.27 StreamDebugInfo

【说明】

流信息。

【定义】

```
typedef struct {
    uint32_t state;
    int32_t  streamIndex;
    AudioStreamInfo streamInfo;
    float streamVolume; // stream mgr
    float mixWeight;
    uint32_t curDevice;
} StreamDebugInfo;
```

【成员】

成员名称	描述
state	流状态。
streamIndex	流索引。
streamInfo	音频流信息，详见“ 2.4.19 AudioStreamInfo ”小节。
streamVolume	流音量。
mixWeight	混音权重。
curDevice	当前设备。

【注意事项】

无。

【相关数据类型及接口】

AudioManagerDebugInfo;

2.4.28 AudioInterruptDebugInfo

【说明】

音频中断信息汇总。

【定义】

```
typedef struct {  
    uint32_t inputInterruptCnt;  
    InterruptDebugInfo inputInterruptInfo[MAX_INST_NUM];  
    uint32_t outputInterruptCnt;  
    InterruptDebugInfo outputInterruptInfo[MAX_INST_NUM];  
} AudioInterruptDebugInfo;
```

【成员】

成员名称	描述
inputInterruptCnt	输入中断个数。
inputInterruptInfo	输入中断信息，详见“ 2.4.25 InterruptDebugInfo ”小节。

成员名称	描述
outputInterruptCnt	输出中断个数。
outputInterruptInfo	输出中断信息，详见“ 2.4.25 InterruptDebugInfo ”小节。

【注意事项】

无。

【相关数据类型及接口】

AudioServiceDebugInfo。

2.4.29 AudioStreamDebugInfo

【说明】

音频流信息汇总。

【定义】

```
typedef struct {  
    uint32_t streamInfoCnt;  
    StreamDebugInfo allstreamInfo[MAX_INST_NUM];  
} AudioStreamDebugInfo;
```

【成员】

成员名称	描述
streamInfoCnt	流信息个数。
allstreamInfo	所有的流信息，详见“ 2.4.27 StreamDebugInfo ”小节。

【注意事项】

无。

【相关数据类型及接口】

AudioServiceDebugInfo;

2.4.30 AudioDeviceDebugInfo

【说明】

音频设备信息汇总。

【定义】

```
typedef struct {
    uint32_t deviceInfoCnt;
    DeviceDebugInfo devicesInfo[MAX_INST_NUM];
} AudioDeviceDebugInfo;
```

【成员】

成员名称	描述
deviceInfoCnt	设备信息条数。
devicesInfo	设备信息，详见“2.4.26 DeviceDebugInfo”小节。

【注意事项】

无。

【相关数据类型及接口】

```
AudioServiceDebugInfo;
```

2.4.31 AudioServiceDebugInfo

【说明】

音频服务信息。

【定义】

```
typedef struct {
    AudioInterruptDebugInfo audioInterruptInfo;
    AudioStreamDebugInfo audioStreamInfo;
    AudioDeviceDebugInfo audioDeviceInfo;
} AudioServiceDebugInfo;
```

【成员】

成员名称	描述
------	----

成员名称	描述
audioInterruptInfo	音频中断信息。
audioStreamInfo	音频流信息。
audioDeviceInfo	音频设备信息。

【注意事项】

无。

【相关数据类型及接口】

```
AudioInterruptDebugInfo;  
AudioStreamDebugInfo;  
AudioDeviceDebugInfo;
```

2.4.32 struct AudioInterrupt

【说明】

音频中断。

【定义】

```
/**  
 * @brief An audio interruption management mechanism used to resolve audio playback conflicts  
 * among multiple applications.  
 *  
 * @since 1.0  
 * @version 1.0  
 */  
struct AudioInterrupt {  
    AudioStreamType streamType;  
    AudioSession sessionID;  
    std::shared_ptr<InterruptListener> interruptListener;  
};
```

【成员】

成员名称	描述
streamType	音频流类型，详见“ 2.4.10 AudioStreamType ”小节。

成员名称	描述
sessionID	音频会话 ID, 详见 “ 2.4.1 AudioSession ” 小节。
interruptListener	音频中断观察对象。

【注意事项】

无。

【相关数据类型及接口】

```
enum AudioInterruptType;
AudioInterruptHintType;
int32_t AudioManager::ActivateAudioInterrupt(AudioInterrupt &interrupt);
int32_t AudioManager::DeactivateAudioInterrupt(AudioInterrupt &interrupt);
```

2.4.33 AudioManagerDebugInfo

【说明】

音频管理信息。

【定义】

```
typedef struct {
    /* 所有输入焦点类型 */
    std::vector<InterruptDebugInfo> inputInterruptInfo;
    /* 所有输出焦点类型 */
    std::vector<InterruptDebugInfo> outputInterruptInfo;
    /* 所有流的信息 */
    std::vector<StreamDebugInfo> allstreamInfo;
    /* 所有设备的信息 */
    std::vector<DeviceDebugInfo> allDeviceInfo;
} AudioManagerDebugInfo;
```

【成员】

成员名称	描述
inputInterruptInfo	音频输入中断信息, 详见 “ 2.4.25 InterruptDebugInfo ” 小节。

成员名称	描述
outputInterruptInfo	音频输出中断信息，详见“ 2.4.25 InterruptDebugInfo ”小节。
allstreamInfo	音频流信息，详见“ 2.4.27 StreamDebugInfo ”小节。
allDeviceInfo	音频设备信息，详见“ 2.4.26 DeviceDebugInfo ”小节。

【注意事项】

无。

【相关数据类型及接口】

InterruptDebugInfo;
StreamDebugInfo;
DeviceDebugInfo;

2.5 错误码

表2-1 音频管理错误码

错误代码	宏定义	描述
0	AUDIO_SUCCESS	成功。
-1	AUDIO_ERROR	失败。
-2	AUDIO_INVALID_PARAMS	无效参数。
-3	AUDIO_INVALID_OPERATION	无效操作。
-4	AUDIO_PERMISSION_DENIED	权限拒绝。
-5	AUDIO_NO_INIT	未初始化。
-6	AUDIO_NO_MEM	没有内存。
-7	AUDIO_RETRY_READ	再读取一次。

错误代码	宏定义	描述
-8	AUDIO_DEAD_OBJECT	对象已死。
-9	AUDIO_BAD_STATE	异常状态。

2.6 开发指导

须知

开发前请参考“[5 音频配置](#)”章节修改所需的配置。

2.6.1 本地音乐播放

对应 AT 命令：AT^MEDIAMUSICPLAY=xx,/user/test/xxx.mp3,1,0

该命令是实现将等号后面的参数列表传给媒体 sample 代码入口函数，sample 是媒体对外 API 的封装，用法如下：

步骤 1 应用实现中断观察回调的派生类，用户在回调函数中，根据回调参数，做出相应播控操作。

```
class PlayerInterruptListener : public InterruptListener {
    void OnInterrupt(int32_t type, int32_t hint) override
    {
        if (type == INTERRUPT_TYPE_BEGIN && hint == INTERRUPT_HINT_PAUSE) {
            // 应用暂停播放
        }
        if (type == INTERRUPT_TYPE_END && hint == INTERRUPT_HINT_RESUME) {
            // 应用恢复播放
        }
        if (type == INTERRUPT_TYPE_BEGIN && hint == INTERRUPT_HINT_STOP) {
            // 应用退出播放
        }
    }
};
```

步骤 2 激活音频中断。

```
/* 获取AudioManager实例 */
AudioManager& amIntance = AudioManager::GetInstance();
/* 初始化音频资源*/
amIntance.Initialize();
/* 创建会话ID */
AudioSession sessionId = amIntance.MakeSessionId();
/* 激活音频中断 */

std::shared_ptr<PlayerInterruptListener> playerInterruptListener =
std::make_shared<PlayerInterruptListener>();
AudioInterrupt interrupt = { AUDIO_STREAM_MUSIC, sessionId, playerInterruptListener };
amIntance.ActivateAudioInterrupt(interrupt);
```

步骤 3 播放音乐，详细内容请参见“[4.6.1 本地文件播放示例](#)”小节。

步骤 4 如果播放中途收到中断观察回调，证明有其他应用在激活音频中断，需要做出相应播控响应，因此返回到[步骤 1](#)；正常播放结束则进行[步骤 5](#)，正常播放结束的流程，请参见“[4.6.1 本地文件播放示例](#)”小节。

步骤 5 去激活音频中断。

```
(amIntance.DeactivateAudioInterrupt(interrupt))
```

----结束

2.6.2 本地音乐和蓝牙音乐切换

步骤 1 参考“[2.6.1 本地音乐播放](#)”小节创建本地音乐播放，如果此时又连接了蓝牙耳机则需要进行设备的切换，设置蓝牙设备已经连接状态。

如果在创建本地音乐播放之前设置蓝牙设备已经连接，则会直接从蓝牙设备输出。

如果蓝牙设备断开，设置蓝牙设备已经断连，则自动切换到本地设备输出。

步骤 2 设置蓝牙设备已经连接状态。

```
amIntance.SetDeviceConnectionState(OUT_BLUETOOTH_A2DP,
AUDIO_DEVICE_CONNECTED);
```

步骤 3 设置蓝牙设备已经断连状态。

```
amIntance.SetDeviceConnectionState(OUT_BLUETOOTH_A2DP,
AUDIO_DEVICE_DISCONNECTED);
```

----结束

2.6.3 蓝牙语音通话

步骤 1 参考“2.6.1 本地音乐播放”小节中的步骤 1，通话应用需要响应相应通话控制。

步骤 2 参考“2.6.1 本地音乐播放”小节中的步骤 2，流类型传入 AUDIO_STREAM_VOICE_CALL_BT_SCO。

步骤 3 设置蓝牙 SCO 链路输入/输出设备连接状态。

```
amIntance.SetDeviceConnectionState(IN_BLUETOOTH_SCO, AUDIO_DEVICE_CONNECTED);  
amIntance.SetDeviceConnectionState(OUT_BLUETOOTH_SCO, AUDIO_DEVICE_CONNECTED);
```

步骤 4 初始化通话上下行输入/输出流信息。

```
/* 通话上下行输入流实例 */  
  
std::shared_ptr<AudioStreamIn> upLinkStreamIn = std::make_shared<AudioStreamIn>();  
std::shared_ptr<AudioStreamIn> downLinkStreamIn = std::make_shared<AudioStreamIn>();  
  
/* 通话上下行输出流实例*/  
  
std::shared_ptr<AudioStreamOut> upLinkStreamOut = std::make_shared<AudioStreamOut>();  
std::shared_ptr<AudioStreamOut> downLinkStreamOut = std::make_shared<AudioStreamOut>();  
  
/* 初始化通话上行输入流信息（Codec: mSBC）*/  
CapturerInputConfig upLinkCaptureConfig = {};  
upLinkCaptureConfig.audioFormat = mSBC;  
upLinkCaptureConfig.channelCount = AUDIO_CHANNEL_1;  
upLinkCaptureConfig.sampleRate = AUDIO_SAMPLE_RATE_16K;  
upLinkCaptureConfig.sessionID = sessionId;  
upLinkCaptureConfig.streamType = AUDIO_STREAM_VOICE_CALL_BT_SCO;  
upLinkCaptureConfig.linkDir = AUDIO_UP_LINK;  
upLinkStreamIn->Init(upLinkCaptureConfig);  
  
/* 初始化通话下行输入流信息*/  
  
CapturerInputConfig downLinkCaptureConfig = {};  
downLinkCaptureConfig.sessionID = sessionId;  
downLinkCaptureConfig.streamType = AUDIO_STREAM_VOICE_CALL_BT_SCO;  
downLinkCaptureConfig.linkDir = AUDIO_DOWN_LINK;  
downLinkStreamIn->Init(downLinkCaptureConfig);  
  
/* 初始化通话上行输出流信息*/  
  
AudioRendererConfig upLinkRenderConfig = {};  
upLinkRenderConfig.sessionID = g_sessionId;
```

```
upLinkRenderConfig.streamType = AUDIO_STREAM_VOICE_CALL_BT_SCO;
upLinkRenderConfig.linkDir = AUDIO_UP_LINK;
upLinkStreamOut ->Init(upLinkRenderConfig);

/* 初始化通话下行输出流信息 (Codec: mSBC) */
AudioRenderConfig downLinkRenderConfig = {};
downLinkRenderConfig.audioFormat = mSBC;
downLinkRenderConfig.channelCount = AUDIO_CHANNEL_1;
downLinkRenderConfig.sampleRate = AUDIO_SAMPLE_RATE_16K;
downLinkRenderConfig.sessionID = g_sessionId;
downLinkRenderConfig.streamType = AUDIO_STREAM_VOICE_CALL_BT_SCO;
downLinkRenderConfig.linkDir = linkDir;
downLinkStreamOut ->Init(downLinkRenderConfig);
```

步骤 5 启动通话上下行输入/输出流，建立绑定关系。

```
/* 启动通话上行输入/输出流*/
upLinkStreamIn->Start();
upLinkStreamOut->Start();

/* 建立通话上行绑定关系*/
uint32_t frontendId;
upLinkStreamIn ->GetCurrentChannelId(frontendId);
upLinkStreamOut ->AttachFrontend(frontendId);

/* 启动通话下行输入/输出流*/
downLinkStreamIn->Start();
downLinkStreamOut->Start();

/* 建立通话下行绑定关系*/
uint32_t backendId;
downLinkStreamOut->GetCurrentChannelId(backendId);
downLinkStreamIn ->AttachFrontend(backendId);
```

步骤 6 通话结束，停止并销毁通话上下行输入/输出流。

```
upLinkStreamOut->Stop();
upLinkStreamOut->Release();
downLinkStreamIn->Stop();
downLinkStreamIn->Release();
upLinkStreamIn->Stop();
```

```
upLinkStreamIn->Release();  
downLinkStreamOut->Stop();  
downLinkStreamOut->Release();
```

步骤 7 设置蓝牙 SCO 链路输入/输出设备断开状态。

```
amIntance.SetDeviceConnectionState(IN_BLUETOOTH_SCO,  
AUDIO_DEVICE_DISCONNECTED);  
amIntance.SetDeviceConnectionState(OUT_BLUETOOTH_SCO,  
AUDIO_DEVICE_DISCONNECTED);
```

步骤 8 参考“2.6.1 本地音乐播放”小节中的步骤 5，去激活音频中断。

----结束

2.6.4 音乐播放中途蓝牙语音通话

交互场景介绍：音乐播放和蓝牙语音通话两个场景交互，音频管理会根据应用类型等决策最终行为，上层只需实现中断监听回调函数即可。

步骤 1 参考“2.6.1 本地音乐播放”小节播放音乐，步骤 1 中需要实现中断监听回调函数。

```
class PlayerInerruptListener : public InterruptListener {  
    void OnInterruptbu(int32_t type, int32_t hint) override  
    {  
        if (type == INTERRUPT_TYPE_BEGIN && hint == INTERRUPT_HINT_PAUSE) {  
            // 当前在媒体回调线程，需要回到应用线程响应播放暂停操作  
        }  
        if (type == INTERRUPT_TYPE_END && hint == INTERRUPT_HINT_RESUME) {  
            // 当前在媒体回调线程，需要回到应用线程响应播放恢复操作  
        }  
        if (type == INTERRUPT_TYPE_BEGIN && hint == INTERRUPT_HINT_STOP) {  
            // 当前在媒体回调线程，需要回到应用线程响应播放退出操作  
        }  
    }  
};
```

步骤 2 参考“2.6.3 蓝牙语音通话”小节步骤 1 ~ 步骤 5 进行蓝牙语音通话，此时会触发步骤 1 中的中断响应函数，type: INTERRUPT_TYPE_BEGIN, hint: INTERRUPT_HINT_PAUSE，播放应用需要响应该操作。

步骤 3 参考“2.6.3 蓝牙语音通话”小节步骤 6 和步骤 7 退出蓝牙语音通话，此时会触发步骤 1 中的中断响应函数，type: INTERRUPT_TYPE_END，hint: INTERRUPT_HINT_RESUME，播放应用需要响应该操作。

步骤 4 参考“4.6.1 本地文件播放示例”小节中的步骤 4 和步骤 5 等待播放完成并退出播放。

----结束

2.6.5 4G VOLTE 本地通话

步骤 1 参考“2.6.1 本地音乐播放”小节中的步骤 1，通话应用做相应通话控制响应。

步骤 2 参考“2.6.1 本地音乐播放”小节中的步骤 2，流类型传入 AUDIO_STREAM_VOICE_CALL_VOLTE。

步骤 3 设置 4G VOLTE 通话，本地输入/输出设备连接状态。

```
amIntance.SetDeviceConnectionState(IN_MODEM, AUDIO_DEVICE_CONNECTED);  
amIntance.SetDeviceConnectionState(OUT_MODEM, AUDIO_DEVICE_CONNECTED);
```

步骤 4 初始化通话上下行输入/输出流信息。

```
/* 创建通话上下行流实例 */  
  
std::shared_ptr<AudioStreamIn> downLinkStreamIn = std::make_shared<AudioStreamIn>();  
std::shared_ptr<AudioStreamOut> upLinkStreamOut = std::make_shared<AudioStreamOut>();  
  
/* 初始化通话上下行流实例 */  
  
CapturerInputConfig captureConfig = {};  
captureConfig.audioFormat = PCM;  
captureConfig.channelCount = 1;  
captureConfig.linkDir = AUDIO_DOWN_LINK;  
captureConfig.sampleRate = 16000;  
captureConfig.sessionID = context->sessionID;  
captureConfig.streamType = AUDIO_STREAM_VOICE_CALL_VOLTE;  
downLinkStreamIn->Init(captureConfig);  
  
AudioRendererConfig renderConfig = {};  
renderConfig.audioFormat = PCM;  
renderConfig.channelCount = 1;  
renderConfig.linkDir = AUDIO_UP_LINK;  
renderConfig.sampleRate = 16000;  
renderConfig.sessionID = context->sessionID;  
renderConfig.streamType = AUDIO_STREAM_VOICE_CALL_VOLTE;
```

```
upLinkStreamOut->Init(renderConfig);
```

步骤 5 启动通话上下行输入/输出流。

```
/* 启动通话上下行流 */  
downLinkStreamIn->Start();  
upLinkStreamOut->Play();
```

步骤 6 停止/销毁通话上下行输入/输出流。

```
/* 停止上下行输入/输出流 */  
downLinkStreamIn->Stop();  
upLinkStreamOut->Stop();  
  
/* 销毁上下行输入/输出流 */  
downLinkStreamIn->Release();  
upLinkStreamOut->Release();
```

步骤 7 设置 4G VOLTE 通话，本地输入/输出设备断开状态。

```
amIntance.SetDeviceConnectionState(IN_MODEM, AUDIO_DEVICE_DISCONNECTED);  
amIntance.SetDeviceConnectionState(OUT_MODEM, AUDIO_DEVICE_DISCONNECTED);
```

步骤 8 参考“2.6.1 本地音乐播放”小节中的步骤 5，去激活音频中断。

场景限制：不支持 CAT1 本地通话场景和 CAT1 蓝牙耳机通话场景自动切换；不支持 CAT1 通话场景和其他场景交互。

用法限制：上述场景需要用户先退出当前场景，再进入新场景。

----结束

2.6.6 4G VOLTE 蓝牙通话

步骤 1 参考“2.6.1 本地音乐播放”小节中的步骤 1，通话应用做相应通话控制响应。

步骤 2 参考“2.6.1 本地音乐播放”小节中的步骤 2，流类型传入
AUDIO_STREAM_VOICE_CALL_VOLTE。

步骤 3 设置 4G VOLTE 通话，蓝牙输入/输出设备连接状态。

```
amIntance.SetDeviceConnectionState(IN_BLUETOOTH_SCO, AUDIO_DEVICE_CONNECTED);  
amIntance.SetDeviceConnectionState(OUT_BLUETOOTH_SCO, AUDIO_DEVICE_CONNECTED);  
amIntance.SetDeviceConnectionState(IN_MODEM_HEADSET, AUDIO_DEVICE_CONNECTED);  
amIntance.SetDeviceConnectionState(OUT_MODEM_HEADSET, AUDIO_DEVICE_CONNECTED);
```

步骤 4 初始化通话上下行输入/输出流信息。

```
/* 创建通话上下行流实例 */  
  
std::shared_ptr<AudioStreamIn> downLinkStreamIn = std::make_shared<AudioStreamIn>();  
std::shared_ptr<AudioStreamIn> downLinkStreamOut = std::make_shared<AudioStreamOut>();  
std::shared_ptr<AudioStreamOut> upLinkStreamOut = std::make_shared<AudioStreamOut>();  
std::shared_ptr<AudioStreamIn> upLinkStreamIn = std::make_shared<AudioStreamIn>();  
  
/* 初始化通话上行流实例 */  
  
CapturerInputConfig captureConfig = {};  
captureConfig.audioFormat = mSBC;  
captureConfig.channelCount = 1;  
captureConfig.linkDir = AUDIO_DOWN_LINK;  
captureConfig.sampleRate = 16000;  
captureConfig.sessionID = context->sessionId;  
captureConfig.streamType = AUDIO_STREAM_VOICE_CALL_VOLTE;  
downLinkStreamIn->Init(captureConfig);  
upLinkStreamIn->Init(captureConfig);  
  
/* 初始化通话上下行流实例 */  
  
AudioRendererConfig renderConfig = {};  
renderConfig.audioFormat = mSBC;  
renderConfig.channelCount = 1;  
renderConfig.linkDir = AUDIO_UP_LINK;  
renderConfig.sampleRate = 16000;  
renderConfig.sessionID = context->sessionId;  
renderConfig.streamType = AUDIO_STREAM_VOICE_CALL_VOLTE;  
upLinkStreamOut->Init(renderConfig);  
downLinkStreamOut->Init(renderConfig);
```

步骤 5 启动通话上下行输入/输出流。

```
/* 启动通话上行流 */  
  
upLinkStreamIn->Start();  
upLinkStreamOut->Play();  
  
/* 启动通话下行流 */  
  
downLinkStreamIn->Start();  
downLinkStreamOut->Play();
```

步骤 6 停止/销毁通话上下行输入/输出流。

```
/* 停止上行输入/输出流 */
upLinkStreamIn->Stop();
upLinkStreamOut->Stop();

/* 停止下行输入/输出流 */
downLinkStreamIn->Stop();
downLinkStreamOut->Stop();

/* 销毁上行输入/输出流 */
upLinkStreamIn->Release();
upLinkStreamOut->Release();

/* 销毁下行输入/输出流 */
downLinkStreamIn->Release();
downLinkStreamOut->Release();
```

步骤 7 设置 4G VOLTE 通话，本地输入/输出设备断开状态。

```
amIntance.SetDeviceConnectionState(IN_BLUETOOTH_SCO,
AUDIO_DEVICE_DISCONNECTED);
amIntance.SetDeviceConnectionState(OUT_BLUETOOTH_SCO,
AUDIO_DEVICE_DISCONNECTED);
amIntance.SetDeviceConnectionState(IN_MODEM_HEADSET,
AUDIO_DEVICE_DISCONNECTED);
amIntance.SetDeviceConnectionState(OUT_MODEM_HEADSET,
AUDIO_DEVICE_DISCONNECTED);
```

步骤 8 参考“2.6.1 本地音乐播放”小节中的步骤 5，去激活音频中断。

场景限制：不支持 CAT1 本地通话场景和 CAT1 蓝牙耳机通话场景自动切换；不支持 CAT1 通话场景和其他场景交互。

用法限制：上述场景需要用户先退出当前场景，再进入新场景。

----结束

2.6.7 A2DP Sink 蓝牙播放

步骤 1 参考“2.6.1 本地音乐播放”小节中的步骤 1，A2DP Sink 应用做相应控制响应。

步骤 2 参考“2.6.1 本地音乐播放”小节中的步骤 2，流类型传入 AUDIO_STREAM_A2DP_MUSIC。

步骤 3 设置 A2DP sink 输入设备连接状态：

```
amIntance.SetDeviceConnectionState(IN_BLUETOOTH_A2DP, AUDIO_DEVICE_CONNECTED);
```

步骤 4 初始化 A2DP sink 上下行输入/输出流信息，部分信息可能需要从蓝牙获取。

```
g_a2dpInputConfig.audioFormat = config.audioFormat;
g_a2dpInputConfig.sampleRate = config.sampleRate;
g_a2dpInputConfig.channelCount = config.channelCount;
g_a2dpInputConfig.streamType = AUDIO_STREAM_A2DP_MUSIC;
g_a2dpInputConfig.linkDir = AUDIO_LINK_NONE;
upLinkStreamOut->Init(captureConfig);

g_a2dpRenderConfig.audioFormat = config.audioFormat;
g_a2dpRenderConfig.sampleRate = config.sampleRate;
g_a2dpRenderConfig.channelCount = config.channelCount;
g_a2dpRenderConfig.streamType = AUDIO_STREAM_A2DP_MUSIC;
g_a2dpRenderConfig.linkDir = AUDIO_LINK_NONE;
downLinkStreamIn->Init(captureConfig);
```

步骤 5 启动 A2DP sink 上下行输入/输出流。

```
/* 启动通话上下行流 */
downLinkStreamIn->Start();
upLinkStreamOut->Play();
```

步骤 6 停止/销毁 A2DP sink 上下行输入/输出流。

```
/* 停止上下行输入/输出流 */
downLinkStreamIn->Stop();
upLinkStreamOut->Stop();

/* 销毁上下行输入/输出流 */
downLinkStreamIn->Release();
upLinkStreamOut->Release();
```

步骤 7 设置 A2DP sink 输入设备断开状态。

```
amIntance.SetDeviceConnectionState(IN_BLUETOOTH_A2DP,
AUDIO_DEVICE_DISCONNECTED);
```

步骤 8 参考“2.6.1 本地音乐播放”小节中的步骤 5，去激活音频中断。

场景限制：不支持 A2DP sink 播放的时候，再连接蓝牙耳机进行播放。

-----结束

3 音频采集

- 3.1 概述
- 3.2 功能描述
- 3.3 API 参考
- 3.4 数据类型、数据结构
- 3.5 错误码
- 3.6 典型场景开发指导

3.1 概述

音频采集模块提供完整的声音采集功能，根据用户配置，支持采集音频原始 PCM 和带有压缩的 MP3 数据、OPUS 数据、SILK 数据，仅 OPUS 数据支持 OGG 格式封装。

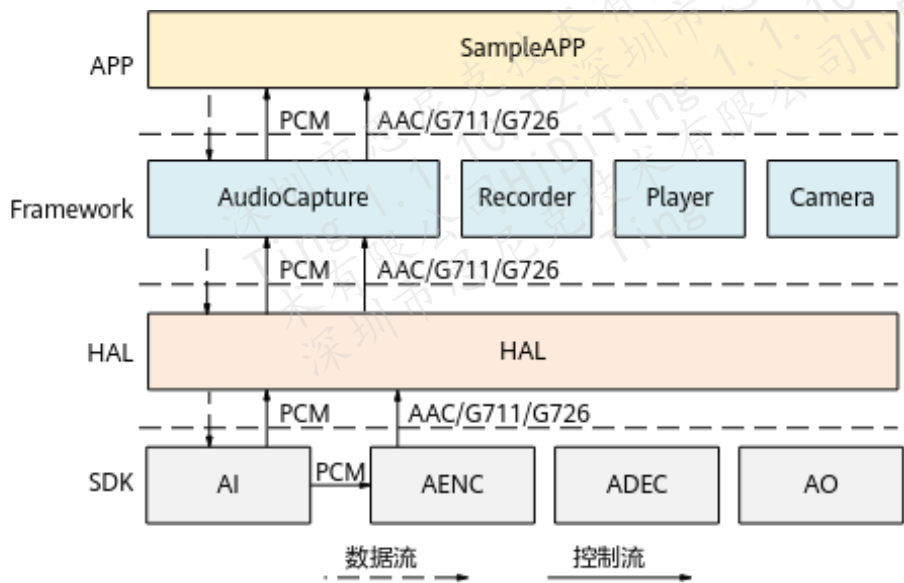
备注：不支持多实例。

3.2 功能描述

3.2.1 机制原理

音频采集模块主要提供参数配置、参数获取、流程控制和状态获取四个功能。具体的流程框图和周边模块交互关系如图 3-1 所示。

图3-1 音频采集模块上下文组件关系图

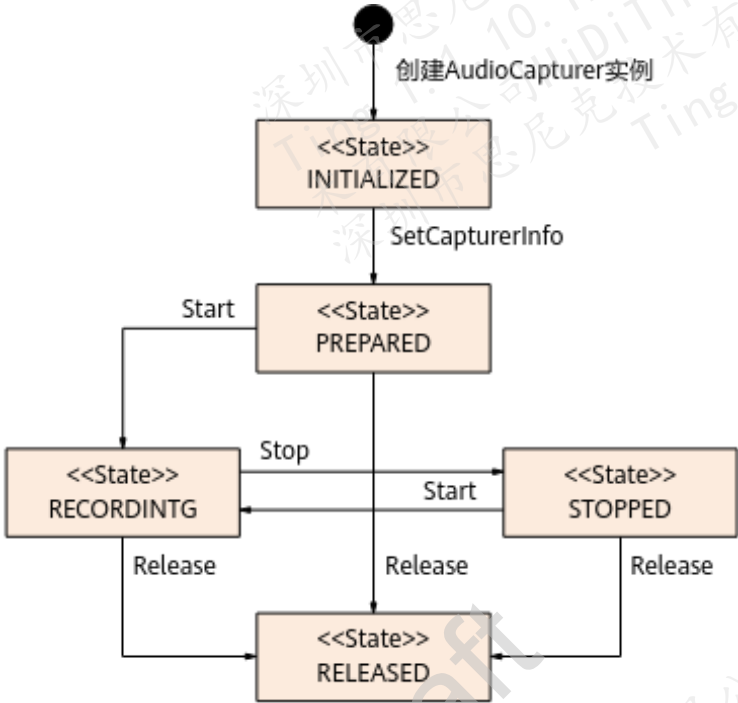


概括来说，音频采集模块主要用来提供以下服务：

- 提供采集参数配置和获取接口。
例如：可配参数包含输入源类型、音频数据格式、采样率、声道数、采样位宽等。
- 提供获取帧数接口。
例如：外部根据该接口得到帧数进而计算存储音频数据缓冲区大小，详见“[3.3.4 Read](#)”接口。
- 提供采集流程接口。
例如：开始采集、开始读数据、停止采集、释放资源等。
- 提供采集位置接口。
例如：获取采集实时位置。
- 提供采集状态获取接口。
例如：采集状态包括初始状态、准备录制、正在录制、录制已停止、资源已释放。

3.2.1.1 音频采集状态机

图3-2 音频采集调用和状态迁移



为保证音频采集状态机正常运转，对上层调用约束如下：

- Start 不允许在 INITIALIZED、RECORDING、RELEASED 状态调用，正常是在 SetCaptureInfo 成功后调用。
- Read 和 Stop 只能在 Start 后调用。
- GetFrameCount 不允许在 INITIALIZED 和 RELEASED 状态调用，返回 0 表示失败。
- GetCaptureInfo 和 GetAudioTime 不允许在 RELEASED 状态调用。

3.2.2 音频采集 DFX

音频采集 DFX 属于非功能属性，例如：性能、可靠性、可服务性、可测试性。细分如下：

- 音频采集参数信息（帧数、采样率、声道数、码率、位宽、音频采集状态），请参见 “3.3.11 DumpInfo” 小节。
- 支持获取采集音频时间。

3.3 API 参考

该功能模块为用户提供以下 API：

- SetCaptureInfo：配置采集参数。
- GetCaptureInfo：获取采集参数。
- Start：开始采集。
- Read：读取采集数据。
- GetStatus：获取采集状态。
- GetAudioTime：获取采集音频时间。
- GetMinFrameCount：获取最小帧数。
- GetFrameCount：获取帧数。
- Stop：停止采集。
- Release：释放采集相关资源。
- DumpInfo：获取运行时的转储日志信息。

3.3.1 SetCapturerInfo

【描述】

配置采集参数。

【定义】

int32_t SetCapturerInfo(const AudioCaptureInfo info);

【参数】

参数名称	描述	输入/输出
info	音频采集参数结构体。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“3.5 错误码”。

【需求】

- 头文件：audio_capturer.h
- 库文件：libaudio_capturer_lite.so

【注意】

BrandyK100 当前音频编码支持单声道、16k 采样率、16bit 位宽采集能力。

【举例】

表3-1 音频采集常用参数配置

格式	源类型	采样率 (Hz)	声道数	码率 (bit/s)	数据位宽	流类型
PCM	AUDIO_MIC	16000	1 或 2	128000	BIT_WIDT H_16	TYPE_MEDIA 或 TYPE_VOICE_C OMMUNICATION
AAC	AUDIO_MIC	48000	1 或 2	128000	BIT_WIDT H_16	TYPE_MEDIA 或 TYPE_VOICE_C OMMUNICATION
G71 1	AUDIO_MIC	8000	1	64000	BIT_WIDT H_16	TYPE_MEDIA 或 TYPE_VOICE_C OMMUNICATION
G72 6	AUDIO_MIC	8000	1	24000	BIT_WIDT H_16	TYPE_MEDIA 或 TYPE_VOICE_C OMMUNICATION

3.3.2 GetCapturerInfo

【描述】

获取采集参数。

【定义】

int32_t GetCapturerInfo(AudioCaptureInfo &info);

【参数】

参数名称	描述	输入/输出
info	音频采集参数结构体引用。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“3.5 错误码”。

【需求】

- 头文件：audio_capturer.h
- 库文件：libaudio_capturer_lite.so

【注意】

SetCapturerInfo 成功后调用该接口有效。

【举例】

无。

3.3.3 Start

【描述】

开始采集。

【定义】

```
bool Start();
```

【参数】

无。

【返回值】

返回值	描述
true	成功。

返回值	描述
false	失败。

【需求】

- 头文件：audio_capturer.h
- 库文件：libaudio_capturer_lite.so

【注意】

SetCapturerInfo 成功后调用该接口有效。

【举例】

无。

3.3.4 Read

【描述】

读取采集数据。

【定义】

int32_t Read(uint8_t *buffer, size_t userSize, bool isBlockingRead);

【参数】

参数名称	描述	输入/输出
buffer	音频数据指针。	输入
userSize	音频数据缓冲区大小，单位 Byte。	输入
isBlockingRead	读取数据是否阻塞。 备注：目前该参数没有使用	输入

【返回值】

返回值	描述
0	成功。

返回值	描述
非 0	失败，详见“3.5 错误码”。

【需求】

- 头文件：audio_capturer.h
- 库文件：libaudio_capturer_lite.so

【注意】

- Start 成功后调用该接口有效。
- 缓冲区大小需要根据 GetFrameCount 返回值确定。

【举例】

确定缓冲区大小方法：

```
AudioCapturer audioCap;  
AudioCapturerInfo info;  
info.bitWidth = BIT_WIDTH_16;  
size_t frameCount = audioCap.GetFrameCount();  
size_t userSize = frameCount * (info.bitWidth / 8) * info.channelCount;  
uint8_t *buffer = new uint8_t[userSize];
```

3.3.5 GetStatus

【描述】

获取采集状态。

【定义】

State GetStatus();

【参数】

无。

【返回值】

返回值	描述
State	当前采集状态。

【需求】

- 头文件: audio_capturer.h
- 库文件: libaudio_capturer_lite.so

【注意】

外部可根据该接口获取当前状态, 同时结合状态机合理使用音频采集 API。

【举例】

无。

3.3.6 GetAudioTime

【描述】

获取采集音频时间。

【定义】

```
bool GetAudioTime(Timestamp &timestamp, ::Timebase base);
```

【参数】

参数名称	描述	输入/输出
timestamp	Timestamp 实例引用。	输入
base	时间基数。	输入

【返回值】

返回值	描述
true	成功。
false	失败。

【需求】

- 头文件: audio_capturer.h
- 库文件: libaudio_capturer_lite.so

【注意】

无。

【举例】

无。

3.3.7 GetMinFrameCount

【描述】

获取最小帧数。

【定义】

```
static bool GetMinFrameCount(int32_t sampleRate, int32_t channelCount,  
AudioCodecFormat audioFormat, size_t &frameCount);
```

【参数】

参数名称	描述	输入/输出
sampleRate	采样率。	输入
channelCount	声道数。	输入
audioFormat	音频数据格式。	输入
frameCount	最小帧数。	输出

【返回值】

返回值	描述
true	成功。
false	失败。

【需求】

- 头文件：audio_capturer.h
- 库文件：libaudio_capturer_lite.so

【注意】

默认返回 0。

【举例】

无。

3.3.8 GetFrameCount

【描述】

获取帧数。

【定义】

uint64_t GetFrameCount();

【参数】

无。

【返回值】

返回值	描述
大于等于 0	成功。
小于 0	失败，详见“3.5 错误码”。

【需求】

- 头文件：audio_capturer.h
- 库文件：libaudio_capturer_lite.so

【注意】

无。

【举例】

本接口用于外部计算存储音频数据缓冲区大小，详见“3.3.4 Read”接口。

【备注】

这里单个音频样点为一帧，调用该接口获取到的值对应单个压缩包包含的样点数。

3.3.9 Stop

【描述】

停止采集。

【定义】

bool Stop();

【参数】

无。

【返回值】

返回值	描述
1	成功。
0	失败。

【需求】

- 头文件：audio_capturer.h
- 库文件：libaudio_capturer_lite.so

【注意】

Start 成功后调用该接口有效。

【举例】

无。

3.3.10 Release

【描述】

释放采集相关资源。

【定义】

bool Release();

【参数】

无。

【返回值】

返回值	描述
1	成功。

返回值	描述
0	失败。

【需求】

- 头文件: audio_capturer.h
- 库文件: libaudio_capturer_lite.so

【注意】

无。

【举例】

无。

3.3.11 DumpInfo

【描述】

Dump 信息。

【语法】

```
int32_t DumpInfo(AudioCapturerDebugInfo &capturerInfo);
```

【参数】

参数名称	描述	输入/输出
capturerInfo	AudioCapturerDebugInfo 基本信息, 详见“ 3.4.10 AudioCapturerDebugInfo ”小节。	输出

【返回值】

返回值	描述	输入/输出
int32_t	• 0: 成功。 • 非 0: 失败, 错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

维测类接口，非功能接口。

【举例】

无。

【相关主题】

无。

3.4 数据类型、数据结构

音频采集模块相关数据类型、数据结构定义如下：

- AudioCapture：定义音频采集类。
- AudioCaptureInfo：定义音频采集属性结构体。
- AudioSourceType：定义音频输入源类型枚举。
- AudioCodecFormat：定义音频数据格式枚举。
- AudioStreamType：定义音频流类型枚举。
- AudioBitWidth：定义采样位宽枚举。
- Timestamp：定义时间戳信息类。
- Timebase：定义时间基数枚举。
- State：定义采集状态枚举。

3.4.1 AudioCapter

【说明】

定义音频采集类。

【定义】

```
class AudioCapter {  
    public:  
        AudioCapter();  
        virtual ~AudioCapter();  
        static bool GetMinFrameCount(int32_t sampleRate, int32_t channelCount,  
        AudioCodecFormat audioFormat, size_t &frameCount);  
};
```

```
uint64_t GetFrameCount();
int32_t SetCapturerInfo(const AudioCapturerInfo info);
int32_t GetCapturerInfo(AudioCapturerInfo &info);
bool Start();
int32_t Read(uint8_t *buffer, size_t userSize, bool isBlockingRead);
State GetStatus();

bool GetAudioTime(Timestamp xamp, Timestamp::Timebase base);

bool Stop();
bool Release();
private:
class AudioCapturerImpl;
std::unique_ptr<AudioCapturerImpl> impl_;
};
```

【注意事项】

无。

【相关数据结构及接口】

- 数据类型：
 - AudioCapturerInfo
 - Timestamp
 - Timebase
 - State
- 成员函数：
 - GetMinFrameCount
 - GetFrameCount
 - SetCapturerInfo
 - GetCapturerInfo
 - Start
 - Read
 - GetStatus
 - GetAudioTime
 - Stop
 - Release

3.4.2 AudioCapturerInfo

【说明】

定义采集属性结构体。

【定义】

```
struct AudioCapterInfo {
    AudioSourceType inputSource = AUDIO_MIC;
    AudioCodecFormat audioFormat = AUDIO_DEFAULT;
    int32_t sampleRate = 0;
    int32_t channelCount = 0;
    int32_t bitRate = 0;
    AudioStreamType streamType = TYPE_MEDIA;
    AudioBitWidth bitWidth = BIT_WIDTH_16;
};
```

【成员】

成员名称	描述
inputSource	音频源类型，详见“3.4.3 AudioSourceType”小节。
audioFormat	音频数据格式，详见“3.4.4 AudioCodecFormat”小节。
sampleRate	采样率。
channelCount	声道数。
bitRate	码率。
streamType	音频流类型，详见“3.4.5 AudioStreamType”小节。
bitWidth	数据位宽，详见“3.4.6 AudioBitWidth”小节。

【注意事项】

无。

【相关数据类型及接口】

- AudioSourceType
- AudioCodecFormat
- AudioStreamType
- AudioBitWidth
- SetCapterInfo
- GetCapterInfo

3.4.3 AudioSourceType

【说明】

定义音频输入源类型枚举。

【定义】

```
typedef enum {
    AUDIO_SOURCE_INVALID = -1,
    AUDIO_SOURCE_DEFAULT = 0,
    AUDIO_MIC = 1,
    AUDIO_VOICE_UPLINK = 2,
    AUDIO_VOICE_DOWNLINK = 3,
    AUDIO_VOICE_CALL = 4,
    AUDIO_CAMCORDER = 5,
    AUDIO_VOICE_RECOGNITION = 6,
    AUDIO_VOICE_COMMUNICATION = 7,
    AUDIO_REMOTE_SUBMIX = 8,
    AUDIO_UNPROCESSED = 9,
    AUDIO_VOICE_PERFORMANCE = 10,
    AUDIO_ECHO_REFERENCE = 1997,
    AUDIO_RADIO_TUNER = 1998,
    AUDIO_HOTWORD = 1999,
    AUDIO_REMOTE_SUBMIX_EXTEND = 10007,
} AudioSourceType;
```

【成员】

成员名称	描述
AUDIO_SOURCE_INVALID	无效类型。
AUDIO_SOURCE_DEFAULT	默认源类型，对应麦克风。
AUDIO_MIC	麦克风。
AUDIO_VOICE_UPLINK	不支持。
AUDIO_VOICE_DOWNLINK	不支持。
AUDIO_VOICE_CALL	不支持。
AUDIO_CAMCORDER	不支持。
AUDIO_VOICE_RECOGNITION	不支持。
AUDIO_VOICE_COMMUNICATION	不支持。
AUDIO_REMOTE_SUBMIX	不支持。

成员名称	描述
AUDIO_UNPROCESSED	不支持。
AUDIO_VOICE_PERFORMANCE	不支持。
AUDIO_ECHO_REFERENCE	不支持。
AUDIO_RADIO_TUNER	不支持。
AUDIO_HOTWORD	不支持。
AUDIO_REMOTE_SUBMIX_EXTEND	不支持。

【注意事项】

开发板和开源板音频采集只支持麦克风输入。

【相关数据类型及接口】

- AudioCapturerInfo

3.4.4 AudioCodecFormat

【说明】

定义音频数据格式枚举。

【定义】

```
typedef enum {  
    /** Default format */  
    AUDIO_DEFAULT      = 0,  
    /** PCM */  
    PCM                = 1,  
    /** Advanced Audio Coding Low Complexity (AAC-LC) */  
    AAC_LC              = 2,  
    /** High-Efficiency Advanced Audio Coding (AAC-HE), previously known as  
    AAC+ or aacPlus v1 */  
    AAC_HE_V1          = 3,  
    /** AAC++ or aacPlus v2 */  
    AAC_HE_V2          = 4,  
    /** Advanced Audio Coding Low Delay (AAC-LD) */  
    AAC_LD              = 5,  
    /** Advanced Audio Coding Enhanced Low Delay (AAC-ELD) */  
    AAC_ELD             = 6,  
}
```

```
/** G711 a-law*/  
G711A          = 7,  
/** G711 u-law */  
G711U          = 8,  
/** G726 */  
G726           = 9,  
/** Invalid value */  
FORMAT_BUTT,  
} AudioCodecFormat;
```

【成员】

成员名称	描述
AUDIO_DEFAULT	默认格式，对应 PCM。
PCM	脉冲编码调制。
AAC_LC	高级音频编码低复杂度，目前不支持。
AAC_HE_V1	高效高级音频编码以前称为 AAC+或 aacPlus v1，目前不支持。
AAC_HE_V2	高效高级音频编码以前称为 AAC+或 aacPlus v2，目前不支持。
AAC_LD	高级音频编码低延迟，目前不支持。
AAC_ELD	高级音频编码增强低延迟，目前不支持。
G711A	G711 a-law，目前不支持。
G711U	G711 u-law，目前不支持。
G726	G726，目前不支持。
FORMAT_BUTT	无效格式。

【注意事项】

无。

【相关数据类型及接口】

- AudioCapterInfo

3.4.5 AudioStreamType

【说明】

定义音频流类型枚举。

【定义】

```
typedef enum {  
    /** Default audio stream type */  
    TYPE_DEFAULT = -1,  
    /** Media */  
    TYPE_MEDIA = 0,  
    /** Voice call */  
    TYPE_VOICE_COMMUNICATION = 1,  
    /** System sound */  
    TYPE_SYSTEM = 2,  
    /** Ringtone */  
    TYPE_RING = 3,  
    /** Music */  
    TYPE_MUSIC = 4,  
    /** Alarm */  
    TYPE_ALARM = 5,  
    /** Notification */  
    TYPE_NOTIFICATION = 6,  
    /** Bluetooth Synchronous Connection-Oriented (SCO) */  
    TYPE_BLUETOOTH_SCO = 7,  
    /** Enforced audible */  
    TYPE_ENFORCED_AUDIBLE = 8,  
    /** Dual-tone multi-frequency (DTMF) */  
    TYPE_DTMF = 9,  
    /** Text-To-Speech (TTS) */  
    TYPE_TTS = 10,  
    /** Accessibility */  
    TYPE_ACCESSIBILITY = 11,  
} AudioStreamType;
```

【成员】

成员名称	描述
TYPE_DEFAULT	默认类型。默认使用 TYPE_MEDIA。
TYPE_MEDIA	媒体声，采集场景对应从麦克风接收到的声

成员名称	描述
	音。
TYPE_VOICE_COMMUNICATION	语音通话声音。
TYPE_SYSTEM	不涉及。
TYPE_RING	不涉及。
TYPE_MUSIC	不涉及。
TYPE_ALARM	不涉及。
TYPE_NOTIFICATION	不涉及。
TYPE_BLUETOOTH_SCO	不涉及。
TYPE_ENFORCED_AUDIBLE	不涉及。
TYPE_DTMF	不涉及。
TYPE_ACCESSIBILITY	不涉及。

【注意事项】

只支持媒体和语音通话类型。

【相关数据类型及接口】

- AudioCapturerInfo

3.4.6 AudioBitWidth

【说明】

定义采样位宽枚举。

【定义】

```
typedef enum {  
    /** 8-bit width */  
    BIT_WIDTH_8    = 8,  
    /** 16-bit width */  
    BIT_WIDTH_16   = 16,  
    /** 24-bit width */  
    BIT_WIDTH_24   = 24,  
    /** 32-bit width */  
    BIT_WIDTH_32   = 32,  
};
```

```
BIT_WIDTH_32 = 32,  
/** Invalid value */  
BIT_WIDTH_BUTT,  
} AudioBitWidth;
```

【成员】

成员名称	描述
BIT_WIDTH_8	8bit 数据位宽。当前不支持。
BIT_WIDTH_16	16bit 数据位宽。
BIT_WIDTH_24	24bit 数据位宽。当前不支持。
BIT_WIDTH_32	32bit 数据位宽。当前不支持。
BIT_WIDTH_BUTT	无效位宽。

【注意事项】

只支持 16bit 数据位宽。

【相关数据类型及接口】

- AudioCapturerInfo

3.4.7 Timestamp

【说明】

定义时间戳信息类。

【定义】

```
class Timestamp {  
public:  
    Timestamp() : framePosition(0)  
    {  
        time.tv_sec = 0;  
        time.tv_nsec = 0;  
    }  
    ~Timestamp() = default;  
    uint32_t framePosition;  
    struct timespec time;  
    /**
```

```
* @brief Enumerates the time base of this <b>Timestamp</b>. Different timing methods are
supported.
*
*/
enum class Timebase : int32_t {
    /** Monotonically increasing time, excluding the system sleep time */
    MONOTONIC = 0,
    /** Monotonically increasing time, including the system sleep time */
    BOOTTIME = 1
};
```

【成员】

成员名称	描述
framePosition	不支持。
time	Linux 时间结构 timespec 变量。
Timebase	时间基数枚举，详见“3.4.8 Timebase”小节。

【注意事项】

无。

【相关数据类型及接口】

- Timebase
- GetAudioTime

3.4.8 Timebase

【描述】

定义时间基数枚举。

【定义】

```
enum class Timebase : int32_t {
    /** Monotonically increasing time, excluding the system sleep time */
    MONOTONIC = 0,
    /** Monotonically increasing time, including the system sleep time */
    BOOTTIME = 1
};
```

【成员】

成员名称	描述
MONOTONIC	单调递增时间，不包括系统休眠时间。
BOOTTIME	单调递增时间，包括系统休眠时间。

【注意事项】

建议使用 MONOTONIC。实际效果与芯片相关。

【相关数据类型及接口】

Timestamp

3.4.9 State

【描述】

定义采集状态枚举。

【定义】

```
enum State : uint32_t {  
    /** Initialized */  
    INITIALIZED,  
    /** Prepared */  
    PREPARED,  
    /** Recording */  
    RECORDING,  
    /** Stopped */  
    STOPPED,  
    /** Released */  
    RELEASED  
};
```

【成员】

成员名称	描述
INITIALIZED	初始状态。
PREPARED	准备采集。
RECORDING	正在采集。

成员名称	描述
STOPPED	采集已停止。
RELEASED	采集资源已释放。

【注意事项】

无。

【相关数据类型及接口】

- `GetStatus`

3.4.10 AudioCapturerDebugInfo

【说明】

音频采集基本调试信息。

【定义】

```
struct AudioCapturerDebugInfo {
    int64_t frameCount;
    int32_t sampleRate;
    int32_t channelCount;
    int32_t bitRate;
    AudioBitWidth bitWidth;
    State state;
};
```

【成员】

成员名称	描述
frameCount	当前条件下所需的帧数。
sampleRate	采样率。
channelCount	声道数。
bitRate	码率。
bitWidth	采样位宽，详见 “ 3.4.6 AudioBitWidth ” 小节。
state	音频采集的状态，详见 “ 3.4.9 State ” 小节。

【注意事项】

无。

【相关数据类型及接口】

- AudioBitWidth
- State
- GetFrameCount
- GetStatus
- GetCapturerInfo

3.5 错误码

表3-2 音频采集 API 错误码

错误代码	宏定义	描述
0xFFFFFFFF	ERR_INVALID_READ	读数据失败。
0x0	SUCCESS	成功。
0x3C10000	ERROR	失败。
0x3C10001	ERR_ILLEGAL_STATE	非法状态。
0x2A05	ERR_INVALID_PARAM	无效参数。
0x3C10003	ERR_EARLY_PREPARE	不涉及。
0x3C10004	ERR_SOURCE_NOT_SET	不涉及。
0x3C10005	ERR_INVALID_OPERATION	无效操作。
0x2A09	ERR_NOFREE_CHANNEL	不涉及。
0x3C10007	ERR_READ_BUFFER	不涉及。
0x3C10008	ERR_NOT_STARTED	不涉及。
0x3C100C8	ERR_UNKNOWN	未知错误。

3.6 典型场景开发指导

3.6.1 音频采集

下面 sample 描述了音频采集模块的创建、提取数据到销毁的过程。

对应 AT 命令：

MP3 录制：AT^MEDIAAUDIORECORD=xx,mp3,16000,1,16,/user/test/xxx.mp3

SILK 录制：AT^MEDIAAUDIORECORD=xx,silk,16000,1,16,/user/test/audio_cap.silk

PCM 录制：

AT^MEDIAAUDIORECORD=xx,pcm,16000,1,16,/user/test/audio_cap.pcm

OPUS 录制：

AT^MEDIAAUDIORECORD=xx,opus,16000,1,16,/user/test/audio_cap.ogg

说明

每种格式的采样率/声道数/位宽是固定的，不支持修改。

上述命令会将等号后面的参数列表传给媒体 sample 代码入口函数，sample 是媒体对外 API 的封装，用法如下。

3.6.1.1 音频采集准备工作

```
/******  
请求音频焦点、创建实例、配置参数  
/******  
class CaptureInterruptListener : public InterruptListener {  
public:  
    CaptureInterruptListener();  
    ~CaptureInterruptListener();  
  
    void OnInterrupt(int32_t type, int32_t hint) override  
    {  
        if (type == INTERRUPT_TYPE_BEGIN && hint == INTERRUPT_HINT_PAUSE) {  
            printf("{%s():%d} need app pause audio capture\n", __FUNCTION__, __LINE__);  
        }  
        if (type == INTERRUPT_TYPE_END && hint == INTERRUPT_HINT_RESUME) {  
            printf("{%s():%d} need app resume audio capture\n", __FUNCTION__, __LINE__);  
        }  
        if (type == INTERRUPT_TYPE_BEGIN && hint == INTERRUPT_HINT_STOP) {  
            printf("{%s():%d} need app stop audio capture\n", __FUNCTION__, __LINE__);  
        }  
    }  
};
```

```

    }
    if (type == INTERRUPT_TYPE_END && hint == INTERRUPT_HINT_STOP) {
        printf("{%s():%d} reserved\n", __FUNCTION__, __LINE__);
    }
}

};
char filePath[PATH_MAX];
AudioCodecFormat audioFormat;
if (strncasecmp(argv[1], "mp3", strlen("mp3")) == 0) {
    audioFormat = MP3;
} else if (strncasecmp(argv[1], "silk", strlen("silk")) == 0) {
    audioFormat = SILK;
} else if (strncasecmp(argv[1], "pcm", strlen("pcm")) == 0) {
    audioFormat = PCM;
} else if (strncasecmp(argv[1], "opus", strlen("opus")) == 0) {
    audioFormat = OPUS;
}
int32_t sampleRate = atoi(argv[2]);
int32_t channelCount = atoi(argv[3]);
AudioBitWidth bitWidth = (AudioBitWidth)atoi(argv[4]);
if (strlen(argv[5]) < PATH_MAX) {
    if (memcpy_s(filePath, PATH_MAX, argv[5], strlen(argv[5]) + 1) != EOK) {
        printf("{%s():%d} memcpy_s failed\n", __FUNCTION__, __LINE__);
        return -1;
    }
} else {
    printf("{%s():%d} file path too long\n", __FUNCTION__, __LINE__);
    return -1;
}

FILE *pfd = fopen(filePath, "w+");
if (pfd == NULL) {
    printf("{%s():%d} fopen failed\n", __FUNCTION__, __LINE__);
    return -1;
}

/* 获取AudioManager实例 */
AudioManager& amIntance = AudioManager::GetInstance();
/* 初始化音频资源*/
amIntance.Initialize();
/* 创建会话ID */
AudioSession sessionId = amIntance.MakeSessionId();

```

```

/* 激活音频中断 */

std::shared_ptr<CaptureInterruptListener> captureInterruptListener=
std::make_shared<CaptureInterruptListener>();
AudioInterrupt interrupt = { AUDIO_STREAM_VOICE_RECORD, sessionId,
captureInterruptListener};
amIntance.ActivateAudioInterrupt(interrupt);

AudioCapturer audioCapturer = std::make_shared<AudioCapturer>();
if (audioCapturer == nullptr || audioCapturer.get() == nullptr) {
    printf("{%s,%d} audio capturer is nullptr\n", __func__, __LINE__);
    (void)amIntance.DeactivateAudioInterrupt(interrupt);
    fclose(pfd);
    return -1;
}
AudioCapturerInfo info = {};
info.inputSource = AUDIO_MIC;
info.audioFormat = audioFormat;
info.sampleRate = sampleRate;
info.channelCount = channelCount;
info.streamType = AUDIO_STREAM_VOICE_RECORD;
info.bitWidth = bitWidth;
info.sessionID = sessionId;

if (audioCapturer->SetCapturerInfo(info) != 0) {
    printf("{%s,%d} set capturer info failed\n", __func__, __LINE__);
    (void)amIntance.DeactivateAudioInterrupt(interrupt);
    fclose(pfd);
    return -1;
}

size_t frameCount = audioCapturer->GetFrameCount();
if (!frameCount) {
    printf("{%s,%d} frame count is invalid\n", __func__, __LINE__);
    (void)audioCapturer->Release()
    (void)amIntance.DeactivateAudioInterrupt(interrupt);
    fclose(pfd);
    return -1;
}
if (audioFormat == OPUS) {
    AudioCapturerOggCtxInit(pfd, info);
}

```

3.6.1.2 音频采集启动

```

/*****/
启动处理
/*****/

size_t framesize = frameCount * info.channelCount * info.bitWidth / sizeof(uint8_t);
uint8_t *buffer = new uint8_t[framesize];
if (buffer == nullptr) {
    printf("{%s,%d} buffer is nullptr\n", __func__, __LINE__);
    return -1;
}
if (!audioCapturer->Start()) {
    printf("{%s,%d} start failed\n", __func__, __LINE__);
    delete g_audioSourceProcessInput.buffer;
    g_audioSourceProcessInput.buffer = nullptr;
    return -1;
}

```

3.6.1.3 音频采集数据提取

```

/*****/
数据提取处理
/*****/

int readCnt = 0;
bool needStop = false;
while (true) {
    int ret = audioCapturer->Read(buffer, framesize, false);
    if (ret == -1) {
        printf("{%s():%d} read failed: 0x%x\n", __FUNCTION__, __LINE__, ret);
        continue;
    }
    if (ret == ERR_RETRY_READ) {
        usleep(10000);
        continue;
    }
    if (audioFormat == OPUS) {
        AudioCapturerWriteInOGG(buffer, ret, pfd, needStop); //传入needStop, OGG封装格式最
        后一个page需要特殊处理
    } else {
        fwrite(buffer, 1, ret, pfd);
    }
}

```

```

        readCnt++;

        printf("{%s}:%d} read count: %d, size: %d\n", __FUNCTION__, __LINE__, readCnt, ret);
        if (needStop) {
            break;
        }
    }
}

```

3.6.1.4 音频采集停止

```

/*****
停止处理
*****/

if (!audioCapturer->Stop()) {
    printf("{%s,%d} stop failed\n", __func__, __LINE__);
}

```

3.6.1.5 音频采集退出工作

```

/*****
退出处理
*****/

if (!audioCapturer->Release()) {
    printf("{%s,%d} release failed\n", __func__, __LINE__);
}

if (audioFormat == OPUS) {
    AudioCapturerOggCtxDelnit();
}

delete g_audioSourceProcessInput.buffer;
g_audioSourceProcessInput.buffer = nullptr;

if (amIntance.DeactivateAudioInterrupt(interrupt) != 0) {
    printf("{%s,%d} deactivate audio interrupt failed\n", __func__, __LINE__);
}

```

4 播放器模块

- 4.1 概述
- 4.2 功能描述
- 4.3 API 参考
- 4.4 数据类型及数据结构
- 4.5 错误码
- 4.6 使用指导

4.1 概述

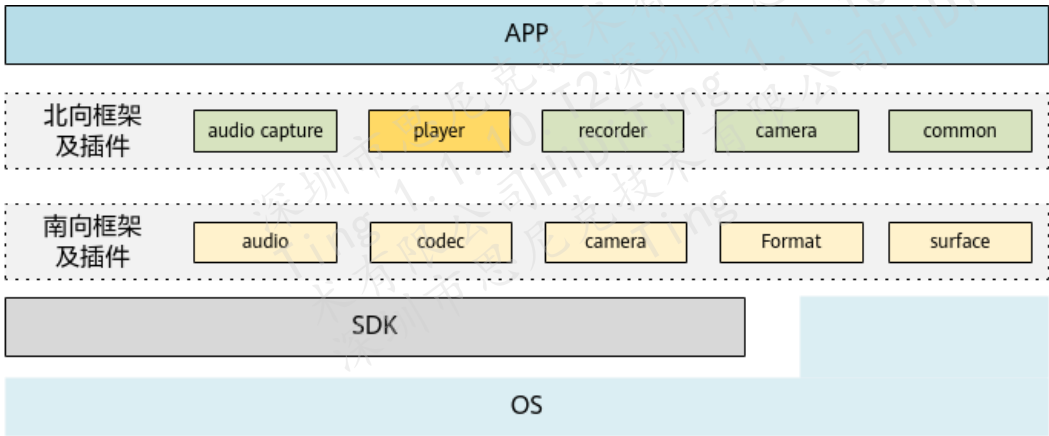
播放器模块为用户提供本地音频文件播放功能。可以播放单音频 MP3/AAC/FLAC/WAV/OGG/M4A。同时也提供 MP4 封装的视频播放功能，当前仅支持 MP4 封装的 jpeg 片源。

播放器模块包含解封装功能与播控功能，解封装功能使用对媒体文件的解析，例如将 MP3 解析成 ES 流；播控功能包括播放控制、内部数据管理等。用户可以根据 demuxer_interface 插件库头文件实现自己的解封装库。

此外，本模块提供了回调注册接口，让用户必须注册回调接口以接收文件播放完成和拖动完成的消息。可以接收播放进度回调消息以及播放错误消息。

播放器模块在整个系统中的位置如图 4-1 所示。

图4-1 播放器模块上下文



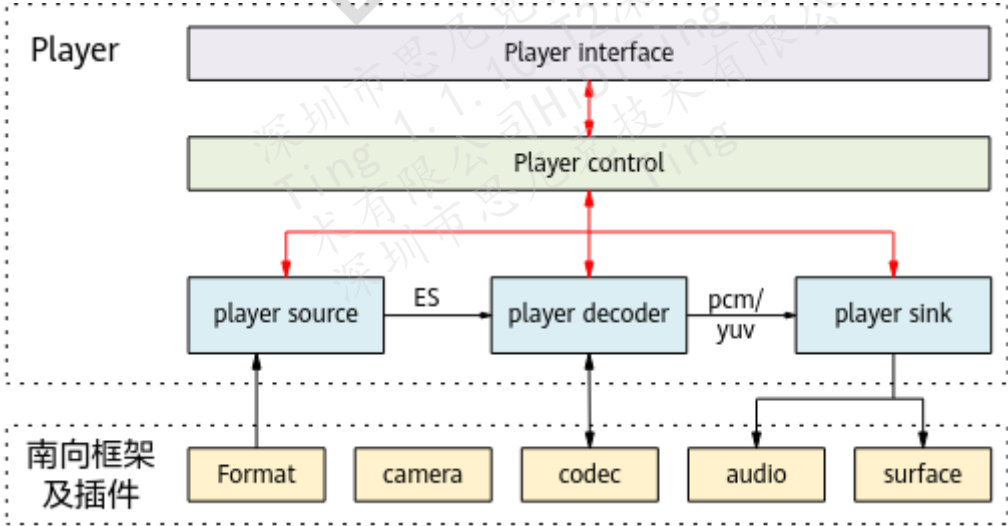
须知

本播放器不支持网络播放。

4.1.1 播放器模块层次结构

播放器模块的层次结构如图 4-2 所示。

图4-2 播放器模块层次结构图



Player 主要由 4 大模块组成：player control（播放器控制模块）、player source（播放源处理模块）、player decoder（解码模块）、player sink（输出模块）。

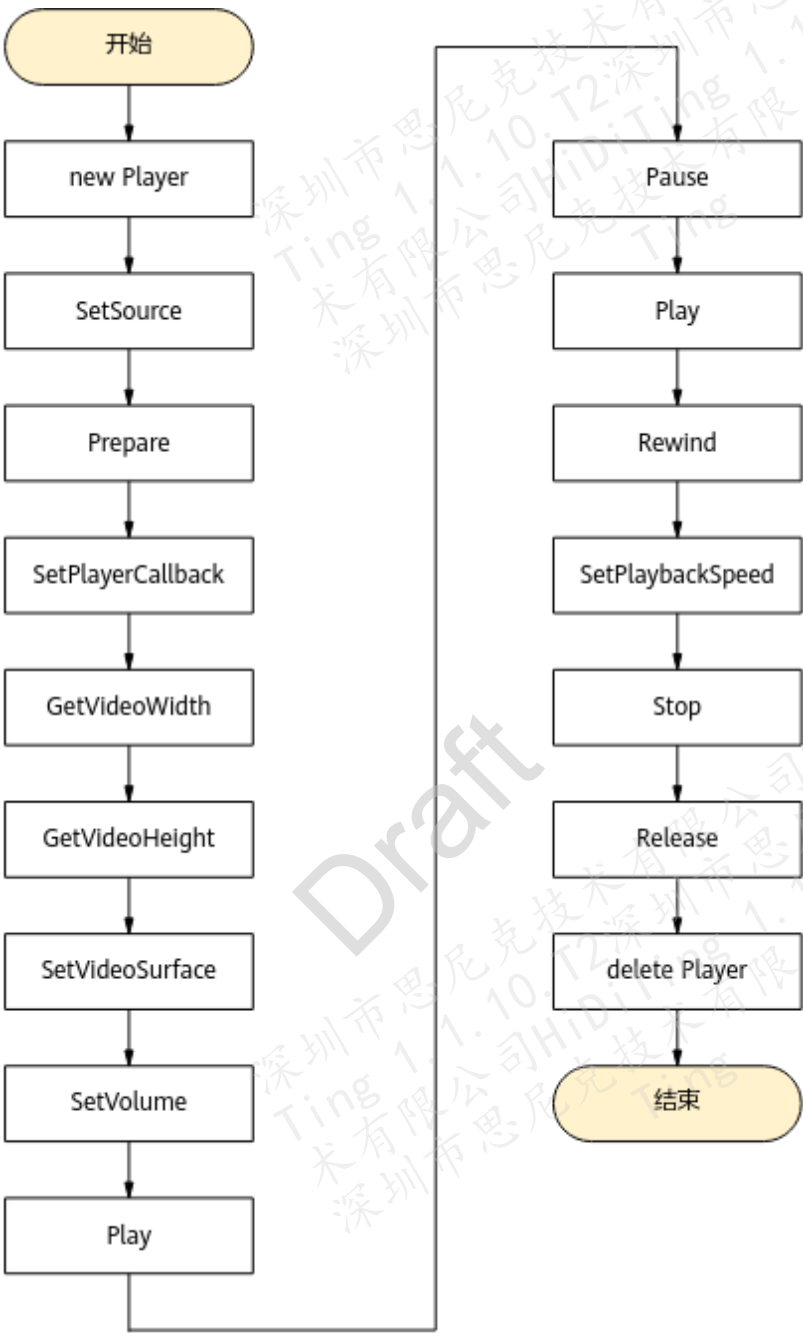
- player control 模块主要负责接收响应 APP 侧命令、模块管理和上报事件处理。

- player source 对接 Format 模块，负责片源解析处理及输出数据帧处理。
- player decoder 模块对接 codec 模块，负责音视频帧解码处理。
- player sink 对接 audio 和显示模块，负责音视频流输出处理及同步处理。

4.1.2 播放器模块处理流程

播放器模块的处理流程如图 4-3 所示。

图4-3 播放器模块处理流程



从图 4-3 可以看出整个播放器的处理流程：

- 步骤 1 创建播放器实例。
- 步骤 2 设置播放源（文件路径 uri 或 stream source）。
- 步骤 3 播放准备处理。

步骤 4 注册回调函数。

步骤 5 获取宽高属性（可选）。

步骤 6 设置视频显示区域。

步骤 7 设置音量（可选）。

步骤 8 执行播放直到文件结束。

步骤 9 播放过程中执行 Pause/Play。

步骤 10 播放过程中执行 Rewind。

步骤 11 播放过程中执行倍速播放。

步骤 12 停止播放器。

步骤 13 释放播放器资源。

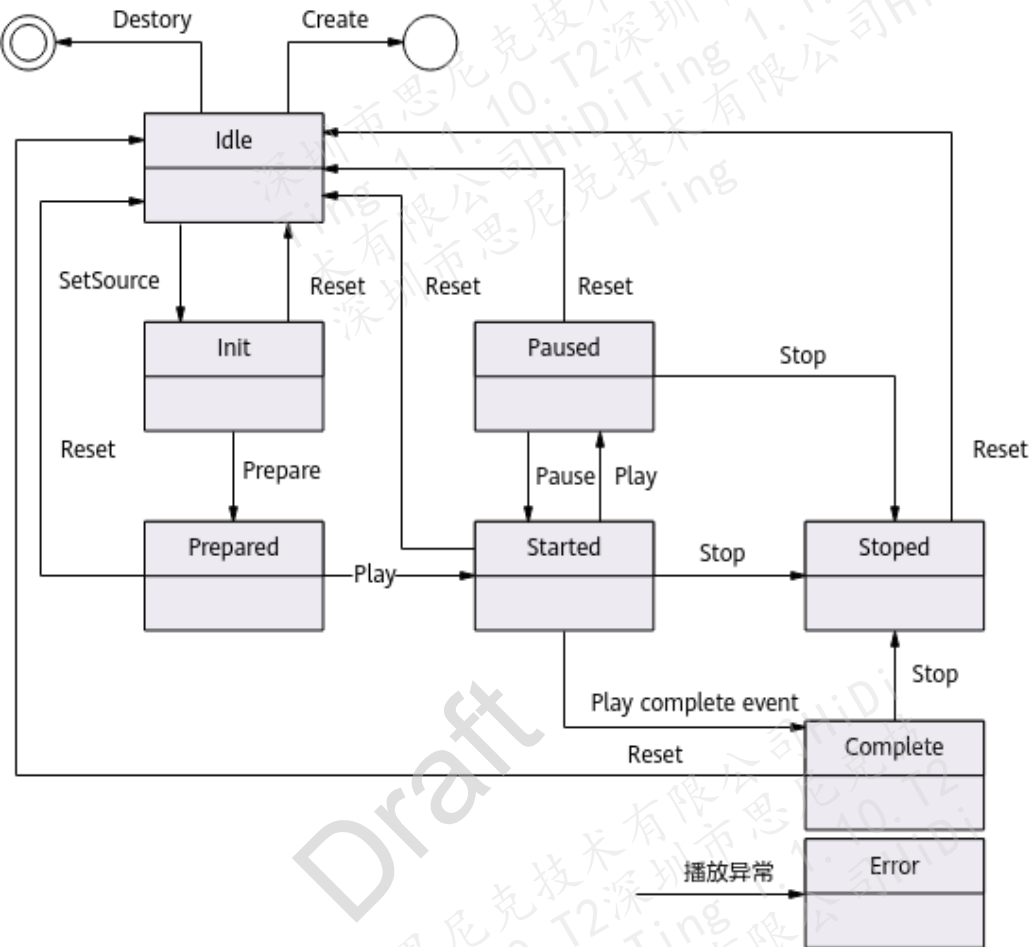
步骤 14 删除播放器实例。

----结束

4.1.3 播放器状态图

播放器状态如[图 4-4](#) 所示。

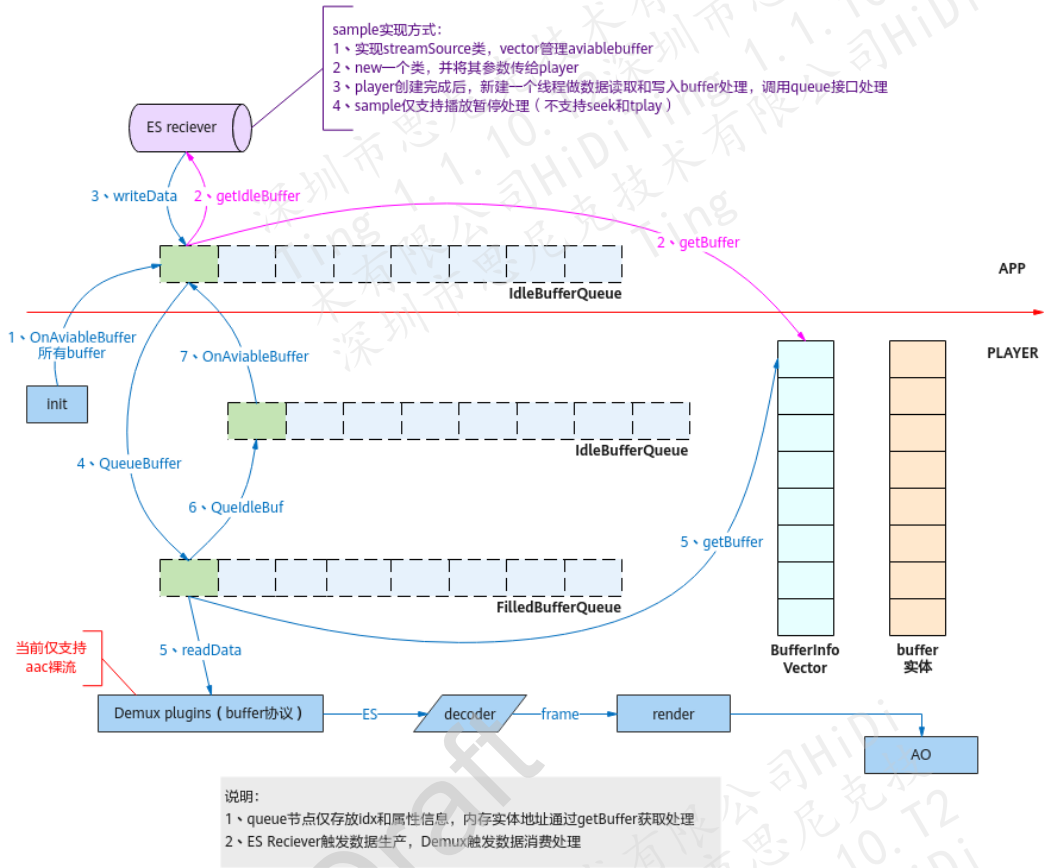
图4-4 播放器模块状态图



4.1.4 StreamSource 源类型 Buffer 流转

针对 StreamSource 场景，Buffer 运转方式示意图如图 4-5 所示，player 内部持有 Buffer 实体，外部 APP 侧仅使用 Available Buffer 并填充后送到 player 侧，player 从 Buffer 读取数据解析后送解码处理。

图4-5 StreamSource buffer 流转图



4.2 功能描述

4.2.1 播控模块

播控模块提供媒体播放、暂停、停止功能。用户在回调函数中不能有阻塞操作, 也不能调用播放器接口。

播放器模块的播放状态对应接口如下:

- 实例创建后对应 PLAYER_IDLE 状态。
- SetSource 对应 PLAYER_INITIALIZED 状态。
- Prepare 对应 PLAYER_PREPARED 状态。
- Play 对应 PLAYER_STARTED 状态。
- Pause 对应 PLAYER_PAUSED 状态。

- Stop 对应 PLAYER_STOPPED 状态。
- Reset 对应 PLAYER_IDLE 状态。

4.2.2 播放器模块 DFX

播放器模块 DFX 属于非功能属性，例如：性能、可靠性、可服务性、可测试性。细分如下：

- 播放文件信息（流类型、文件路径、文件大小、开始播放时间、总播放时间、视频流索引、音频流索引等），请参见“4.3.1.23-DumpInfo”小节。
- 播放视频流信息（视频流索引号、视频解码类型、视频宽、视频高等），请参见“4.3.1.23-DumpInfo”小节。
- 播放音频流信息（音频流索引号、音频解码类型、采样率、声道数、位宽等），请参见“4.3.1.23-DumpInfo”小节。
- 播放控制信息（会话 ID、输出设备、当前播放位置、倍速、播放状态、是否循环等），请参见“4.3.1.23-DumpInfo”小节。
- 支持获取播放状态、播放倍速、当前播放位置。
- 支持获取视频宽、高信息。

4.3 API 参考

4.3.1 Player 类

该功能模块提供以下 API：

- SetSource：设置 player 要播放的数据源。
- Prepare：播放器预处理，同时完成片源的解析处理。
- Play：开始片源播放处理。
- IsPlaying：查询当前播放器状态是否处于播放状态。
- Pause：暂停播放。
- Stop：停止播放。
- Rewind：回退到指定播放位置。
- SetVolume：设置播放器音量。
- SetVideoSurface：设置视频显示区域。

- EnableSingleLooping: 设置循环播放标记。
- IsSingleLooping: 查询循环播放标记。
- GetPlayerState: 查询当前播放器状态。
- GetCurrentTime: 查询当前播放器位置。
- GetDuration: 查询片源时长。
- GetVideoWidth: 获取播放视频流宽度。
- GetVideoHeight: 获取播放视频流高度。
- SetPlaybackSpeed: 设置播放速度。
- GetPlaybackSpeed: 获取播放速度。
- Reset: 重置播放器到 IDLE 状态。
- Release: 销毁播放器资源。
- SetPlayerCallback: 设置播放器回调函数。
- SetParameter: 扩展接口, 设置播放器参数 (如设置首帧暂停模式)。
- DumpInfo: 获取运行时的转储日志信息。

4.3.1.1 SetSource

【描述】

设置 player 要播放的数据源, 可以是文件路径 URI 和 AAC 裸流 StreamSource。

【语法】

```
int32_t SetSource(const Source &source);
```

【参数】

参数名称	描述	输入/输出
source	播放片源信息, 详见“4.3.3 Source 类”。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败, 详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

使用前 media_server 和 wms_server 已正常启动。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.2 Prepare

【描述】

播放器预处理，同时完成片源的解析处理。

【语法】

int32_t Prepare();

【参数】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

该接口需 SetSource 接口调用成功后使用。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.3 Play

【描述】

开始片源播放处理。

【语法】

int32_t Play();

【参数】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

该接口需 Prepare 接口调用成功后使用。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.4 IsPlaying

【描述】

查询当前播放器状态是否处于播放状态。

【语法】

```
bool IsPlaying();
```

【参数】

无。

【返回值】

返回值	描述
true	处于播放状态，对应 PLAYER_STARTED 状态。
false	处于非播放状态，除 PLAYER_STARTED 之外的状态。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.5 Pause

【描述】

暂停播放。

【语法】

```
int32_t Pause();
```

【参数】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

- 该接口一般在 PLAYER_STARTED 状态下调用。
- 如果是首帧暂停播放模式，则在 PLAYER_PREPARED 状态下调用本接口达到首帧暂停的效果。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.6 Stop

【描述】

停止播放。

【语法】

int32_t Stop();

【参数】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

该接口需在 PLAYER_STARTED、PLAYER_PAUSED、PLAYER_PLAYBACK_COMPLETE 状态下调用。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.7 Rewind

【描述】

回退到指定播放位置，当前版本不支持该接口。

【语法】

int32_t Rewind(int64_t mSeconds, int32_t mode);

【参数】

参数名称	描述	输入/输出
mSeconds	回退位置，单位：ms。	输入
mode	回退处理模式，详见“4.4.1 PlayerSeekMode”。	输入

【返回值】

返回值	描述
-----	----

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

- 该接口是异步接口，Rewind 完成后会播放器根据注册的回调函数调用 OnRewindToComplete 上报事件处理。
- 多 Rewind 操作需要在前次 Rewind 操作收到 OnRewindToComplete 回调消息后执行，否则可能出现中间的 Rewind 操作失效。
- 该接口只能在 PLAYER_STARTED、PLAYER_PAUSED 两种播放状态下调用。
- 在调用 Rewind 接口与接收到 Rewind end 回调消息之间：
 - 调用 Play 或 Pause 会执行失败。
 - 调用 SetPlaybackSpeed 则 Rewind 操作可能会执行失败。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.8 SetVolume

【描述】

设置播放器音量。

【语法】

```
int32_t SetVolume(float leftVolume, float rightVolume);
```

【参数】

参数名称	描述	输入/输出
------	----	-------

参数名称	描述	输入/输出
leftVolume	左声道音量。	输入
rightVolume	右声道音量。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

- 芯片差异不支持单独控制左声道和右声道，内部取左右声道均值配置 AO 音量处理。
- 左右声道都配置为 0，表示设置静音模式。

【相关主题】

无。

4.3.1.9 SetVideoSurface

【描述】

设置视频显示区域。

【语法】

int32_t SetVideoSurface(Surface *surface);

【参数】

参数名称	描述	输入/输出
surface	Surface 指针，来源详见 4.6.1 步骤 3。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

设置的显示区域不能超过显示器或 LCD 可显示区域。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.10 EnableSingleLooping

【描述】

设置循环播放标记。

【语法】

```
int32_t EnableSingleLooping(bool loop);
```

【参数】

参数名称	描述	输入/输出
loop	是否循环播放。	输入

【返回值】

返回值	描述
-----	----

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

针对不支持循环播放的场景，设置失败或没有循环播放效果。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.11 IsSingleLooping

【描述】

查询循环播放标记。

【语法】

bool IsSingleLooping();

【参数】

无。

【返回值】

返回值	描述
true	循环播放模式。
false	非循环播放模式。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.12 GetPlayerState

【描述】

查询当前播放器状态。

【语法】

```
int32_t GetPlayerState(int32_t &state);
```

【参数】

参数名称	描述	输入/输出
state	播放状态。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.13 GetCurrentTime

【描述】

查询当前播放时间。

【语法】

```
int32_t GetCurrentTime(int64_t &time);
```

【参数】

参数名称	描述	输入/输出
time	当前播放时间，单位 ms。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.14 GetDuration**【描述】**

获取片源时长。

【语法】

```
int32_t GetDuration(int64_t &duration);
```

【参数】

参数名称	描述	输入/输出
duration	片源时长，单位：ms。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

针对 StreamSource 类型源，获取到的时长为-1，直到播放完成，可获取到片源时长。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.15 GetVideoWidth

【描述】

获取播放视频流宽度。

【语法】

```
int32_t GetVideoWidth(int32_t &videoWidth);
```

【参数】

参数名称	描述	输入/输出
videoWidth	视频宽度。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

该接口需 Prepare 执行成功后才能拿到有效值。

【举例】

无。

【相关主题】

无。

4.3.1.16 GetVideoHeight

【描述】

获取播放视频流高度。

【语法】

int32_t GetVideoHeight(int32_t &videoHeight);

【参数】

参数名称	描述	输入/输出
videoHeight	视频高度。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

该接口需 Prepare 执行成功后被调用。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.17 SetPlaybackSpeed

【描述】

设置播放速度，当前版本不支持该接口。

【语法】

int32_t SetPlaybackSpeed(float speed);

【参数】

参数名称	描述	输入/输出
speed	播放速度。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

- 该接口需要 Play 执行后操作设置，非正常速度下只有视频输出。
- 设置播放速度为 1.0 恢复正常速度。
- 纯音频文件播放无倍速效果，且返回失败。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.18 GetPlaybackSpeed

【描述】

获取播放速度。

【语法】

```
int32_t GetPlaybackSpeed(float &speed);
```

【参数】

参数名称	描述	输入/输出
------	----	-------

参数名称	描述	输入/输出
speed	播放速度。	输出

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

无。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.19 Reset

【描述】

重置播放器到 IDLE 状态。

【语法】

int32_t Reset();

【参数】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

Reset 操作成功后可调用 SetSource 继续播放下一个片源处理。

【举例】

无。

【相关主题】

无。

4.3.1.20 Release

【描述】

释放播放器资源。

【语法】

int32_t Release();

【参数】

无。

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“4.5 错误码”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

- 该接口主要用做释放播放器资源，但是实例本身还没有析构处理。
- 调用该接口后，不能再执行其他操作处理。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.21 SetPlayerCallback

【描述】

设置播放器回调函数。

【语法】

```
void SetPlayerCallback(const std::shared_ptr<PlayerCallback> &cb);
```

【参数】

参数名称	描述	输入/输出
cb	播放器回调函数类 PlayerCallback。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

- 如果不设置播放器回调函数或设置回调函数 cb 为 NULL，则不会触发事件上报到 APP 侧。
- APP 需在释放播放器资源后析构回调函数类。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.1.22 SetParameter

【描述】

扩展接口，设置播放器参数（如设置首帧暂停模式）。

【语法】

```
int32_t SetParameter(const Format &params);
```

【参数】

参数名称	描述	输入/输出
params	待设置的参数 Format 类，如 PAUSE_AFTER_PLAY。	输入

【返回值】

返回值	描述
0	成功。
非 0	失败，详见“ 4.5 错误码 ”。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：libplayer_lite.so

【注意】

可以在 PLAYER_INITIALIZED 和 PLAYER_PREPARED 状态下，设置首帧暂停播放模式；并在 Prepare 后执行 Pause 操作，达到首帧暂停效果。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.1.23 DumpInfo

【描述】

Dump 信息。

【语法】

int32_t DumpInfo(PlayerDebugInfo *playerInfo);

【参数】

参数名称	描述	输入/输出
playerInfo	PlayerDebugInfo 基本信息，详见“4.4.7-PlayerDebugInfo”小节。	输出

【返回值】

返回值	描述	输入/输出
int32_t	0：成功。 - 非 0：失败，错误码定义在 audio_errors.h。	输出

【芯片差异】

无。

【注意】

维测类接口，非功能接口。

【举例】

无。

【相关主题】

无。

4.3.2 PlayerCallback 类

该功能模块提供以下 API，PlayerCallback 类对应接口由 APP 侧实现，且不能在回调事件中处理事务。

- OnPlaybackComplete：播放完成事件上报。
- OnError：错误事件上报。
- OnInfo：信息事件上报。
- OnVideoSizeChanged：视频分辨率发生变化事件上报。
- OnRewindToComplete：拖动完成事件上报。

4.3.2.1 OnPlaybackComplete

【描述】

播放完成事件上报。

【语法】

```
void OnPlaybackComplete()
```

【参数】

无。

【返回值】

无。

【需求】

- 头文件：player.h、source.h、format.h、surface.h。
- 库文件：上层实现库。

【注意】

播放完成时上报该事件，循环播放状态无事件上报。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.2.2 OnError

【描述】

错误事件上报。

【语法】

```
void OnError(int32_t errorType, int32_t errorCode);
```

【参数】

参数名称	描述	输入/输出
errorType	错误类型，详见“ 4.4.4 PlayerErrorType ”小节。	输出
errorCode	错误码。	输出

【返回值】

无

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：上层实现库。

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.2.3 OnInfo

【描述】

信息事件上报。

【语法】

void OnInfo(int type, int extra);

【参数】

参数名称	描述	输入/输出
type	信息事件类型，详见“4.4.3 PlayerInfoType”小节。	输出
extra	可选项，type 携带参数。	输出

【返回值】

无

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：上层实现库。

【注意】

无。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.2.4 OnVideoSizeChanged

【描述】

视频分辨率发生变化事件上报。

【语法】

void OnVideoSizeChanged(int width, int height);

【参数】

参数名称	描述	输入/输出
width	视频宽度。	输出
height	视频高度。	输出

【返回值】

无

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：上层实现库。

【注意】

- Prepare 后主动上报默认流视频分辨率信息。
- 播放过程中发现分辨率发生变化，触发该事件上报。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.2.5 OnRewindToComplete

【描述】

拖动完成事件上报。

【语法】

void OnRewindToComplete();

【参数】

无。

【返回值】

无。

【需求】

- 头文件：player.h、source.h、format.h、surface.h
- 库文件：上层实现库。

【注意】

Rewind 操作为异步操作，收到该事件才表示 Rewind 真正完成。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.3 Source 类

该功能模块提供以下 API。

- Source：构造函数。
- GetSourceFd：获取 SOURCE_TYPE_FD 源类型文件描述符。
- GetSourceType：获取源类型。
- GetSourceFdOffset：获取 SOURCE_TYPE_FD 源类型文件偏移量。
- GetSourceFdLength：获取 SOURCE_TYPE_FD 源类型文件大小。
- GetSourceUri：获取 SOURCE_TYPE_URI 源类型路径。
- GetSourceHeader：获取 SOURCE_TYPE_URI 源类型 HEADER 信息。
- GetSourceStream：获取 SOURCE_TYPE_STREAM 源类型 StreamSource 信息。
- GetSourceStreamFormat：获取 SOURCE_TYPE_STREAM 源类型扩展信息。

4.3.3.1 Source

【描述】

构造函数。

【语法】

```
Source(int32_t fd);
```

```
Source(const std::string& uri);
```

```
Source(int32_t fd, uint64_t offset, uint64_t len);
```

```
Source(const std::string &uri, const std::map<std::string, std::string> &header);
```

```
Source(const std::shared_ptr<StreamSource> &stream, const Format &formats);
```

【参数】

参数名称	描述	输入/输出
fd	SOURCE_TYPE_FD 源类型，文件描述符。	输入
uri	SOURCE_TYPE_URI 源类型，文件路径。	输入
offset	SOURCE_TYPE_FD 源类型，文件偏移量。	输入
len	SOURCE_TYPE_FD 源类型，文件大小。	输入
header	SOURCE_TYPE_URI 源类型，网络路径 header 信息。	输入
stream	SOURCE_TYPE_STREAM 源类型，StreamSource 类实体。	输入
formats	SOURCE_TYPE_STREAM 源类型，扩展信息，可用于承载； 流格式信息，如 CODEC_MIME、 MIME_AUDIO_AAC。	输入

【返回值】

无。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

- SOURCE_TYPE_FD 源类型当前不支持。
- SOURCE_TYPE_URI 源类型当前仅支持播放本地文件。
- SOURCE_TYPE_STREAM 源类型当前仅支持播放 AACLC 裸流。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.3.2 GetSourceFd

【描述】

获取 SOURCE_TYPE_FD 源类型文件描述符。

【语法】

int32_t GetSourceFd();

【参数】

无。

【返回值】

返回值	描述
-1	文件描述符不存在。
大于 0	文件描述符。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

需先调用 GetSourceType 确认是 SOURCE_TYPE_FD 源类型后，再使用该接口。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.3.3 GetSourceType

【描述】

获取源类型。

【语法】

SourceType GetSourceType();

【参数】

无。

【返回值】

返回值	描述
SOURCE_TYPE_URI	承载文件路径类型。
SOURCE_TYPE_FD	承载文件描述符类型。
SOURCE_TYPE_STREAM	承载流类型。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.3.4 GetSourceFdOffset

【描述】

获取 SOURCE_TYPE_FD 源类型文件偏移量。

【语法】

uint64_t GetSourceFdOffset();

【参数】

无。

【返回值】

返回值	描述
0	从文件起始位置开始播放处理。
非 0	从文件偏移量位置开始播放处理。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

需先调用 GetSourceType 确认是 SOURCE_TYPE_FD 源类型后，再使用该接口。

【举例】

无。

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.3.5 GetSourceFdLength

【描述】

获取 SOURCE_TYPE_FD 源类型文件大小。

【语法】

```
uint64_t GetSourceFdLength();
```

【参数】

无。

【返回值】

返回值	描述
0x7fffffffffffffff	无效值。
其他值	文件大小。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

需先调用 GetSourceType 确认是 SOURCE_TYPE_FD 源类型后，再使用该接口。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.3.6 GetSourceUri

【描述】

获取 SOURCE_TYPE_URI 源类型路径。

【语法】

```
const std::string &GetSourceUri();
```

【参数】

无。

【返回值】

URI 地址。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

需先调用 GetSourceType 确认是 SOURCE_TYPE_URI 源类型后，再使用该接口。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.3.7 GetSourceHeader

【描述】

获取 SOURCE_TYPE_URI 源类型 HEADER 信息。

【语法】

```
const std::map<std::string, std::string> &Source::GetSourceHeader();
```

【参数】

无。

【返回值】

HEADER 信息，通过 map 存放。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

- 需先调用 GetSourceType 确认是 SOURCE_TYPE_URI 源类型后，再使用该接口。
- 目前不支持网络片源播放，未真正使用。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.3.8 GetSourceStream

【描述】

获取 SOURCE_TYPE_STREAM 源类型 StreamSource 信息。

【语法】

```
const std::shared_ptr<StreamSource> &GetSourceStream();
```

【参数】

无。

【返回值】

返回值	描述
NULL	使用方法存在问题。
非 NULL	StreamSource 信息。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

该需先调用 GetSourceType 确认是 SOURCE_TYPE_STREAM 源类型后，再使用该接口。

【相关主题】

无。

4.3.3.9 GetSourceStreamFormat

【描述】

获取 SOURCE_TYPE_STREAM 源类型扩展信息。

【语法】

const Format &GetSourceStreamFormat();

【参数】

无。

【返回值】

扩展信息，如流格式。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

该需先调用 `GetSourceType` 确认是 `SOURCE_TYPE_STREAM` 源类型后, 再使用该接口。

【相关主题】

无。

4.3.4 StreamSource 类

该功能模块提供以下 API。

- `OnBufferAvailable`: 传递某个外部可使用 Buffer 信息回调函数, APP 侧实现。
- `SetStreamCallback`: 设置 `StreamSource` 回调函数到 APP 侧, APP 侧实现。

4.3.4.1 OnBufferAvailable

【描述】

传递某个外部可使用 buffer 信息回调函数, APP 侧实现。

【语法】

```
void OnBufferAvailable(size_t index, size_t offset, size_t size);
```

【参数】

参数名称	描述	输入/输出
index	Buffer 索引值。	输入
offset	相对 Buffer 起始地址, 可用段的偏移量。	输入
size	Buffer 可用段长度。	输入

【返回值】

无。

【需求】

- 头文件: `source.h`、`format.h`
- 库文件: `libmedia_common.so`

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.4.2 SetStreamCallback

【描述】

设置 StreamSource 回调函数到 APP 侧，APP 侧实现。

【语法】

```
void SetStreamCallback(const std::shared_ptr<StreamCallback> &callback);
```

【参数】

参数名称	描述	输入/输出
callback	回调方法类，详见“ 4.3.5 StreamCallback 类 ”。	输入

【返回值】

无

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.5 StreamCallback 类

该功能模块提供以下 API。

- GetBuffer：获取指定 Buffer 地址。
- QueueBuffer：传输装载数据的 Buffer 到播放器内部。
- SetParameters：扩展接口。

4.3.5.1 GetBuffer

【描述】

获取指定 Buffer 地址。

【语法】

uint8_t *GetBuffer(size_t index);

【参数】

参数名称	描述	输入/输出
index	Buffer 索引值。	输入

【返回值】

指定 Buffer 的虚拟地址。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.3.5.2 QueueBuffer

【描述】

传输装载数据的 Buffer 到播放器内部。

【语法】

```
void QueueBuffer(size_t index, size_t offset, size_t size, int64_t timestampUs,
uint32_t flags);
```

【参数】

参数名称	描述	输入/输出
index	Buffer 索引值。	输入
offset	承载数据地址相对 Buffer 起始地址的偏移量。	输入
size	承载数据长度。	输入
timestampUs	承载数据时戳（以帧为单位场景有效）。	输入
flags	Buffer 数据标识，详见“4.4.6 BufferFlags”小节。	输入

【返回值】

无。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

无。

【举例】

具体示例内容请参见“4.6.1 本地文件播放示例”小节。

【相关主题】

无。

4.3.5.3 SetParameters

【描述】

扩展接口。

【语法】

```
void SetParameters(const Format &params);
```

【参数】

参数名称	描述	输入/输出
params	扩展参数信息，详见 format.h 中的 Format 类。	输入

【返回值】

无。

【需求】

- 头文件：source.h、format.h
- 库文件：libmedia_common.so

【注意】

无。

【举例】

具体示例内容请参见“[4.6.1 本地文件播放示例](#)”小节。

【相关主题】

无。

4.4 数据类型及数据结构

- PlayerSeekMode：播放器 Rewind 模式。
- PlayerStates：定义播放状态。
- PlayerInfoType：播放器信息时间类型。
- PlayerErrorType：播放器错误时间类型。

- SourceType：播放片源源类型。
- BufferFlags：Buffer 承载数据标识。

4.4.1 PlayerSeekMode

【说明】

播放器 Rewind 模式。

【定义】

```
enum PlayerSeekMode : int32_t {  
    PLAYER_SEEK_PREVIOUS_SYNC = 0,  
    PLAYER_SEEK_NEXT_SYNC,  
    PLAYER_SEEK_CLOSEST_SYNC,  
    PLAYER_SEEK_CLOSEST  
};
```

【成员】

成员名称	描述
PLAYER_SEEK_PREVIOUS_SYNC	将位置移动到给定时间之前的 IDR 帧。
PLAYER_SEEK_NEXT_SYNC	将位置移动到给定时间之后的 IDR 帧。
PLAYER_SEEK_CLOSEST_SYNC	Rewind 到最近 IDR 帧位置。
PLAYER_SEEK_CLOSEST	Rewind 到最近数据帧位置。

【注意事项】

当前只支持 PLAYER_SEEK_PREVIOUS_SYNC 模式，其他模式在播放器内部会自动转换处理。

【相关数据类型及接口】

无。

4.4.2 PlayerStates

【说明】

定义播放状态。

【定义】

```
enum PlayerStates : uint32_t {  
    PLAYER_STATE_ERROR = 0,  
    PLAYER_IDLE = 1 << 0,  
    PLAYER_INITIALIZED = 1 << 1,  
    PLAYER_PREPARING = 1 << 2,  
    PLAYER_PREPARED = 1 << 3,  
    PLAYER_STARTED = 1 << 4,  
    PLAYER_PAUSED = 1 << 5,  
    PLAYER_STOPPED = 1 << 6,  
    PLAYER_PLAYBACK_COMPLETE = 1 << 7  
};
```

【成员】

成员名称	描述
PLAYER_STATE_ERROR	播放错误状态。
PLAYER_IDLE	播放空闲状态。
PLAYER_INITIALIZED	播放初始化状态。
PLAYER_PREPARING	播放准备过程中状态。
PLAYER_PREPARED	播放准备完成状态。
PLAYER_STARTED	播放状态。
PLAYER_PAUSED	播放暂停状态。
PLAYER_STOPPED	播放停止状态。
PLAYER_PLAYBACK_COMPLETE	播放完成状态。

【注意事项】

目前 Prepare 仅支持同步处理，故外部实际查询不到 PLAYER_PREPARING 状态。

【相关数据类型及接口】

无。

4.4.3 PlayerInfoType

【说明】

播放器信息事件类型。

【定义】

```
enum PlayerInfoType : int32_t {  
    PLAYER_INFO_RENDER_START = 0,  
};
```

【成员】

成员名称	描述
PLAYER_INFO_RENDER_START	首帧视频帧送显事件。

【注意事项】

无

【相关数据结构及接口】

无。

4.4.4 PlayerErrorType

【说明】

播放器错误事件类型。

【定义】

```
enum PlayerErrorType : int32_t {  
    PLAYER_ERROR_UNKNOWN = 0,  
};
```

【成员】

成员名称	描述
PLAYER_ERROR_UNKNOWN	未知错误。

【注意事项】

无。

【相关数据结构及接口】

无。

4.4.5 SourceType

【说明】

播放片源源类型。

【定义】

```
enum class SourceType : int32_t {  
    SOURCE_TYPE_URI = 0,  
    SOURCE_TYPE_FD,  
    SOURCE_TYPE_STREAM,  
};
```

【成员】

成员名称	描述
SOURCE_TYPE_URI	文件路径类型。
SOURCE_TYPE_FD	文件描述符类型。
SOURCE_TYPE_STREAM	流类型。

【注意事项】

无。

【相关数据类型及接口】

无。

4.4.6 BufferFlags

【说明】

Buffer 承载数据标识。

【定义】

```
enum BufferFlags : uint32_t {  
    STREAM_FLAG_SYNCFRAME = 1,  
    STREAM_FLAG_CODECCONFIG = 2,  
    STREAM_FLAG_EOS = 4,  
    STREAM_FLAG_PARTIAL_FRAME = 8,  
};
```

```
STREAM_FLAG_ENDOFFRAME = 16,  
STREAM_FLAG_MUXER_DATA = 32,  
};
```

【成员】

成员名称	描述
STREAM_FLAG_SYNCFRAME	单独的一帧，同步帧。
STREAM_FLAG_CODECCONFIG	编解码配置信息，如 SPS、PPS、VPS 等。
STREAM_FLAG_EOS	流结束。
STREAM_FLAG_PARTIAL_FRAME	数据帧的一部分。
STREAM_FLAG_ENDOFFRAME	帧结束。
STREAM_FLAG_MUXER_DATA	容器数据。

【注意事项】

无。

【相关数据结构及接口】

无。

4.4.7 CodecFormat

【说明】

音视频编解码格式。

【定义】

```
/**  
 * @brief Enumerates audio, video, and image encoding formats.  
 *  
 * @since 1.0  
 * @version 1.0  
 */  
typedef enum {  
    CODEC_H264 = 0, /**< H264 or AVC */  
    CODEC_H265,      /**< H265 or HEVC */  
};
```

```
CODEC_JPEG,      /**< JPEG */
CODEC_MJPEG,     /**< MJPEG */
CODEC_AAC,       /**< AAC */
CODEC_G711A,     /**< G711A */
CODEC_G711U,     /**< G711u */
CODEC_PCM,       /**< PCM */
CODEC_MP3,       /**< MP3 */
CODEC_G726,      /**< G726 */
CODEC_OPUS,      /**< OPUS */
CODEC_FLAC,      /**< FLAC */
CODEC_VORBIS,    /**< VORBIS */
CODEC_APE,       /**< APE */
CODEC_SILK,      /**< SILK */
CODEC_SBC,       /**< SBC */
CODEC_BUT,       /**< Undefined format */
} CodecFormat;
```

【成员】

成员名称	描述
CODEC_H264	对应 H264 或 AVC 格式。
CODEC_H265	对应 H265 或 HEVC 格式。
CODEC_JPEG	JPEG。
CODEC_MJPEG	MJPEG。
CODEC_AAC	高级音频编码低复杂度。
CODEC_G711A	G711A, 目前不支持。
CODEC_G711U	G711U, 目前不支持。
CODEC_PCM	脉冲编码调制。
CODEC_MP3	MP3。
CODEC_G726	G726。
CODEC_OPUS	Opus。
CODEC_FLAC	Flac。
CODEC_VORBIS	Vorbis。

成员名称	描述
CODEC_APE	Ape。
CODEC_SILK	Silk。
CODEC_SBC	msbc。
CODEC_BUT	未定义格式。

【注意事项】

无。

【相关数据类型及接口】

无。

4.4.8 PlayerDebugInfo

【说明】

音频管理信息。

【定义】

```
typedef struct {  
    /* 播放文件信息 */  
    PlayerFileDebugInfo fileDebugInfo;  
    /* 视频流信息 */  
    PlayerVideoDebugInfo videoDebugInfo;  
    /* 音频流信息 */  
    PlayerAudioDebugInfo audioDebugInfo;  
    /* 播放控制信息 */  
    PlayerControlDebugInfo playControllInfo;  
} PlayerDebugInfo;
```

【成员】

成员名称	描述
fileDebugInfo	播放文件信息，详见“4.4.9-PlayerFileDebugInfo”小节。

成员名称	描述
videoDebugInfo	播放视频流信息，详见“4.4.10-PlayerVideoDebugInfo”小节。
audioDebugInfo	播放音频流信息，详见“4.4.11-PlayerAudioDebugInfo”小节。
playControllInfo	播放控制信息，详见“4.4.12-PlayerControlDebugInfo”小节。

【注意事项】

无。

【相关数据类型及接口】

- PlayerFileDebugInfo
- PlayerVideoDebugInfo
- PlayerAudioDebugInfo
- PlayerControlDebugInfo

4.4.9 PlayerFileDebugInfo

【说明】

播放文件信息汇总。

【定义】

```
struct PlayerFileDebugInfo {
    uint32_t streamType;
    std::string filePath;
    int32_t fd;
    uint64_t offset;
    int64_t s64FileSize;
    int64_t s64StartTime;
    int64_t s64Duration;
    int32_t s32UsedVideoStreamIndex;
    int32_t s32UsedAudioStreamIndex;
};
```

【成员】

成员名称	描述
------	----

成员名称	描述
streamType	音频流类型，详见“2.4.10 AudioStreamType”小节。
filePath	文件路径，默认为空。
fd	文件描述符，-1 表示没有传入文件描述符。
offset	文件从头开始的偏移量，默认为 0。
s64FileSize	文件大小，默认为 0，不一定所有的格式能获取文件大小。
s64StartTime	播放开始时间，默认为 0。
s64Duration	总的播放时间，默认为 0。
s32UsedVideoStreamIndex	视频流的索引，-1 表示不存在该流。
s32UsedAudioStreamIndex	音频流的索引，-1 表示不存在该流。

【注意事项】

无。

【相关数据类型及接口】

- AudioStreamType
- GetAudioStreamType
- GetDuration

4.4.10 PlayerVideoDebugInfo

【说明】

播放视频流信息汇总。

【定义】

```
struct PlayerVideoDebugInfo {  
    int32_t s32UsedVideoStreamIndex;  
    int32_t enVideoType;  
    uint32_t u32Width;  
    uint32_t u32Height;  
    uint32_t frameBufCnt;
```

```
};
```

【成员】

成员名称	描述
s32UsedVideoStreamIndex	视频流的索引, -1 表示不存在该流。
enVideoType	视频解码器类型, 详见“ 4.4.7-CodecFormat ”小节。
u32Width	视频宽。
u32Height	视频高。
frameBufCnt	视频帧缓存大小。

【注意事项】

无。

【相关数据类型及接口】

- CodecFormat

4.4.11 PlayerAudioDebugInfo

【说明】

播放音频流信息汇总。

【定义】

```
struct PlayerAudioDebugInfo {  
    int32_t s32UsedAudioStreamIndex;  
    int32_t enAudioType;  
    uint32_t u32AudioChannelCnt;  
    uint16_t sampleFmt;  
    uint32_t u32SampleRate;  
    uint16_t bitWidth;  
};
```

【成员】

成员名称	描述
------	----

成员名称	描述
s32UsedAudioStreamIndex	音频流的索引，-1 表示不存在该流。。
enAudioType	音频解码器类型，详见“4.4.7-CodecFormat”小节。
u32AudioChannelCnt	声道数。
sampleFmt	采样格式。
u32SampleRate	采样率。
bitWidth	位宽。

【注意事项】

无。

【相关数据类型及接口】

- CodecFormat

4.4.12 PlayerControlDebugInfo

【说明】

播放控制信息汇总。

【定义】

```
struct PlayerControlDebugInfo {
    uint32_t sessionId;
    int32_t outDevice;
    int64_t currentPosition;
    int64_t rewindPosition;
    float speed;
    int32_t playerControlState;
    bool isSingleLoop;
    bool isVideoStarted;
    bool isAudioStarted;
    bool hasRenderAudioEos;
    bool hasRenderVideoEos;
    bool offloadAudio;
};
```

【成员】

成员名称	描述
sessionId	音频会话 ID，详见“ 2.4.1 AudioSession ”小节。
outDevice	音频设备类型，详见“ 2.4.3 AudioDeviceType ”小节。
currentPosition	当前播放位置。
rewindPosition	回退位置，默认是 0。
speed	播放倍速，默认是 1 倍数。
playerControlState	当前播放状态，详见“ 4.4.2-PlayerStates ”小节。
isSingleLoop	是否是循环播放，默认为 false。
isVideoStarted	是否开始解码视频流。
isAudioStarted	是否开始解码音频流。
hasRenderAudioEos	是否已经解码完音频流。
hasRenderVideoEos	是否已经解码完视频流。
offloadAudio	音频是否是 Offload 模式。

【注意事项】

无。

【相关数据类型及接口】

- AudioSession
- AudioDeviceType
- PlayerStates
- GetPlaybackSpeed
- IsSingleLooping
- GetCurrentTime
- GetAudioSessionId
- GetRoutedDeviceId

4.5 错误码

当前播放接口返回参数有明确注释，无需错误码。

4.6 使用指导

4.6.1 本地文件播放示例

文件播放一个媒体文件到结尾，用户接口调用说明如下：

步骤 1 初始化播放器模块

```
/*-----*/  
播放器模块进行初始化并创建一个播放实例。  
/*-----*/  
std::shared_ptr<Player> player = std::make_shared<Player>();  
if (player == nullptr || player.get() == nullptr) {  
    printf("{%s):%d} player is nullptr\n", __func__, __LINE__);  
    return -1;  
}
```

步骤 2 设置播放源，准备播放

```
/*-----*/  
设置播放器的数据源，并执行Prepare处理  
/*-----*/  
  
std::string uri(xxx.xxx); // xxx.xxx: 文件路径  
std::map<std::string, std::string> header;  
Source source(uri, header);  
player->SetSource(source);  
  
player->SetAudioSessionId(sessionId); // 音频文件需要  
player->Prepare();
```

步骤 3 设置播放信息

```
bool isPlaybackCompleted = false;  
MediaMutexHandle playerMutex = MediaMutexCreate(NULL);  
MediaThreadCondHandle playerCond = MediaThreadCondCreate();  
class PlayerCallbackImpl : public PlayerCallback {
```

```

public:
    PlayerCallbackImpl() {}
    ~PlayerCallbackImpl() {}
    void OnPlaybackComplete()
    {
        MediaMutexLock(playerMutex);
        isPlaybackCompleted = true;
        MediaThreadCondSignal(playerCond);
        MediaMutexUnLock(playerMutex);
    }
}

// 视频文件需要
bool surfaceInitExit = false;
bool surfaceInitSuccess = false;
MediaMutexHandle surfaceMutex = MediaMutexCreate(NULL);
MediaThreadCondHandle surfaceCond = MediaThreadCondCreate();
static void SurfaceInitEvent(void)
{
    RootView::GetInstance()->SetWidth(Screen::GetInstance().GetWidth());
    RootView::GetInstance()->SetHeight(Screen::GetInstance().GetHeight());
    RootView::GetInstance()->SetPosition(0, 0);
    RootView::GetInstance()->SetStyle(STYLE_BACKGROUND_OPA, 0);

    UILiteSurfaceView *surfaceView = new UILiteSurfaceView();
    if (surfaceView == nullptr) {
        printf("{%s():%d} surfaceView is nullptr\n", __FUNCTION__, __LINE__);
        MediaMutexLock(surfaceMutex);
        surfaceInitExit = true;
        surfaceInitSuccess = false;
        MediaThreadCondSignal(surfaceCond);
        MediaMutexUnLock(surfaceMutex);
        return;
    }
    surfaceView->SetViewId("sur");
    RootView::GetInstance()->Add(surfaceView);

    Surface *surface = surfaceView->GetSurface();
    surface->SetStrideAlignment(64);
    surface->SetWidthAndHeight(454, 454); // 视频宽高

    surface->SetQueueSize(2);
    surface->SetFormat(PIXEL_FMT_YCBCR_420_SP);
    surface->SetSize(454 * 454 * 3 / 2);

```

```

        MediaMutexLock(surfaceMutex);
        surfaceInitExit = true;
        surfaceInitSuccess = true;
        MediaThreadCondSignal(surfaceCond);
        MediaMutexUnLock(surfaceMutex);
    }
    GraphicService::GetInstance()->PostGraphicEvent([] {
        SurfaceInitEvent();
    })

    // 阻塞式等待 Surface 初始化结束
    MediaMutexLock(surfaceMutex);
    while (!surfaceInitExit) {
        MediaThreadCondWait(surfaceCond, surfaceMutex);
    }
    MediaMutexUnLock(surfaceMutex);
    player->SetVideoSurface(surface);
    std::shared_ptr<PlayerCallback> pcb = std::make_shared<PlayerCallbackImpl>();
    player->SetPlayerCallback(pcb);
    player->SetVolume(100,100);

```

步骤 4 播放直到收到播放到文件尾的消息

```

ret = player->Play();

printf("{%s():%d} play ret: %d\n", __FUNCTION__, __LINE__, ret);
MediaMutexLock(playerMutex);
while (!isPlaybackCompleted) {
    MediaThreadCondWait(playerCond, playerMutex);
}
MediaMutexUnLock(playerMutex);

```

步骤 5 停止播放器并销毁播放器

```

bool surfaceDeinitExit = false;
static void SurfaceDeinitEvent(void)
{
    RootView::GetInstance()->Remove(surfaceView);
    MediaMutexLock(surfaceMutex);
    surfaceDeinitExit = true;
    MediaThreadCondSignal(surfaceCond);
    MediaMutexUnLock(surfaceMutex);
}

```

```

}
int32_t ret = player->Stop();

printf("{%s():%d} stop ret: %d\n", __FUNCTION__, __LINE__, ret);
ret = player->Reset();
printf("{%s():%d} reset ret: %d\n", __FUNCTION__, __LINE__, ret);
ret = player->Release();

printf("{%s():%d} release ret: %d\n", __FUNCTION__, __LINE__, ret);

GraphicService::GetInstance()->PostGraphicEvent([] {
    SurfaceDeinitEvent();
})

// 阻塞式等待 Surface 去初始化结束
MediaMutexLock(surfaceMutex);
while (!g_videoPlayContext->surfaceDeinitExit) {
    MediaThreadCondWait(surfaceCond, surfaceMutex);
}
MediaMutexUnLock(surfaceMutex);

if (playerMutex != NULL) {
    MediaMutexDestroy(&playerMutex);
    playerMutex = NULL;
}
if (playerCond != NULL) {
    MediaThreadCondDestroy(&playerCond);
    playerCond = NULL;
}
if (surfaceMutex != NULL) {
    MediaMutexDestroy(&surfaceMutex);
    surfaceMutex = NULL;
}
if (surfaceCond != NULL) {
    MediaThreadCondDestroy(&surfaceCond);
    surfaceCond = NULL;
}
if (surfaceView != NULL) {
    delete surfaceView;
    surfaceView = NULL;
}

```

----结束

4.6.2 StreamSource 播放示例

StreamSource 用于音频 AACLC 裸流播放，后续步骤请参考“[4.6.1 本地文件播放示例](#)”中本地文件播放流程中[步骤 3](#)~[步骤 5](#)。

步骤 1 初始化设置播放器模块

```

/*****/

播放器模块进行初始化并创建一个播放实例。

/*****/
sample.adapter = std::make_shared<Player>();

```

步骤 2 设置并准备播放数据源

```

/*****/

设置播放器的数据源，并执行Prepare处理

/*****/
sample.streamSample = std::make_shared<StreamSourceSample>();
Format formats;
formats.PutStringValue(CODEC_MIME, MIME_AUDIO_AAC);
Source source(sample.streamSample, formats);
sample.adapter->SetSource(source);
sample.isThreadRunning = true;
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_attr_setstacksize(&attr, 0x10000);
int32_t ret = pthread_create(&sample.process, &attr, streamProcess, &sample);
if (ret != 0) {
    printf("pthread_create failed %d\n", ret);
    sample.isThreadRunning = false;
    return -1;
}
sample.adapter->Prepare();

```

streamProcess 线程处理代码片段

```

while (true) {
    pthread_mutex_lock(&sample->mutex);
    if (!sample->isThreadRunning) {
        printf("[%s,%d] thread should exit\n", __func__, __LINE__);
        pthread_mutex_unlock(&sample->mutex);
        break;
    }
}

```

```
}
pthread_mutex_unlock(&sample->mutex);
ret = sample->streamSample->GetAvailableBuffer(&buffer);
if (ret != 0) {
    usleep(STEAM_PROCESS_SLEEP_TIME_US);
    continue;
}
data = sample->streamSample->GetBufferAddress(buffer.idx);
if (data == nullptr) {
    printf("[%s, %d] get buffer null", __func__, __LINE__);
    break;
}
len = (buffer.size < READ_LEN) ? buffer.size : READ_LEN;
readLen = fread(data + buffer.offset, 1, len, pFile);
if (readLen <= len && readLen > 0) {
    sample->streamSample->QueueBuffer(buffer.idx, buffer.offset, readLen, 0, 8);
} else {
    sample->streamSample->QueueBuffer(buffer.idx, buffer.offset, readLen, 0, 4);
    printf("[%s, %d] have render eos", __func__, __LINE__);
    break;
}
}
```

----结束

4.6.3 注意事项

- 如果使用了 launcher，则播放应用应该通过 launcher 进入。
- 存在 launcher 场景，播放应用从非 launcher 进入，视频画面会被 launcher 遮盖。

5 音频配置

音频配置文件路径为：drivers\boards\brandy_evb\product\product_evb4_standard.h

5.1 内置输入设备

5.2 内置输出设备

5.3 外挂输入设备

5.4 外挂输出设备

5.5 TRACK MODE

5.6 AEF

5.7 编码码率

5.8 上行音量

5.1 内置输入设备

说明

通常为麦克风。

- BUILTIN_AI_PORT：枚举值对应的端口需与硬件上内置输入设备连接的端口一致（请参考“drivers\drivers\driver\audio\include\soc_uapi_ai.h”中的 uapi_ai_port 枚举）。
- BUILTIN_AI_PORT_ATTR_ADC_RX_TYPE：当前不可配置，请使用默认值。
- BUILTIN_AI_PCM_ATTR_AUDIO_CH：当前不可配置，请使用默认值。
- BUILTIN_AI_PCM_ATTR_BIT_DEPTH：当前不可配置，请使用默认值。
- BUILTIN_AI_PCM_ATTR_SAMPLE_RATE：当前不可配置，请使用默认值。

- BUILTIN_AI_PCM_ATTR_FRAME_PER_SEC: 当前不可配置, 请使用默认值。

5.2 内置输出设备

说明

通常为 SmartPA。

12.288M 为 codec 模块的总时钟, 用于计算分频比。

- BUILTIN_SND_OUT_PORT: 枚举值对应的端口需与硬件上内置输出设备连接的端口一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_sound.h” 中的 uapi_snd_out_port 枚举)。
- BUILTIN_SND_OUT_PORT_NUM: 输出端口个数。
- BUILTIN_SND_OUT_AUDIO_CH: 需与内置输出设备的声道数一致。
- BUILTIN_SND_OUT_BIT_DEPTH: 需与内置输出设备的位深一致。
- BUILTIN_SND_OUT_SAMPLE_RATE: 需与内置输出设备的采样率一致。

当使用 I2S 连接内置输出设备时, 需要修改以下配置:

- BUILTIN_SND_OUT_I2S_ATTR_I2S_BCLK: 12.288M 时钟经过此枚举对应的比值分频后, 需与内置输出设备的 I2S BCLK 一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 uapi_audio_i2s_bclk_sel 枚举)。
例如 BCLK 为 1.536M, 计算得分频比应为 8 ($12.288\text{M}/1.536\text{M}=8$), 查询枚举 uapi_audio_i2s_bclk_sel, 应选择 UAPI_AUDIO_I2S_BCLK_8_DIV, 对应枚举值为 8。
- BUILTIN_SND_OUT_I2S_ATTR_BIT_DEPTH: 需与内置输出设备的 I2S 位深一致。
- BUILTIN_SND_OUT_I2S_ATTR_AUDIO_CH: 需与内置输出设备的 I2S 声道数一致。
- BUILTIN_SND_OUT_I2S_ATTR_I2S_STD_MODE: 枚举值对应的模式需与内置输出设备的 I2S 模式一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 uapi_audio_i2s_mode 枚举)。
- BUILTIN_SND_OUT_I2S_ATTR_MASTER: 需与内置输出设备的 I2S 主/从模式相反 (1 表示做主设备, 0 表示做从设备)。

- BUILTIN_SND_OUT_I2S_ATTR_I2S_MCLK: 12.288M 时钟经过此枚举对应的比值分频后, 需与内置输出设备的 I2S FS 一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 uapi_audio_i2s_mclk_sel 枚举)。
例如 FS 为 48K, 计算得分频比应为 256 ($12.288\text{M}/48\text{K}=256$), 查询枚举 uapi_audio_i2s_mclk_sel, 应选择 UAPI_AUDIO_I2S_MCLK_256_FS, 对应枚举值为 1。

当 I2S 模式选择 PCM 时, 需要修改以下配置:

- BUILTIN_SND_OUT_I2S_ATTR_I2S_PCM_DELAY: BCLK 时延 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 uapi_audio_i2s_pcm_delay 枚举)。
- BUILTIN_SND_OUT_I2S_ATTR_SAMPLE_RISE_EDGE: 采样沿选择 (1 表示在 BCLK 的上升沿采样数据, 0 表示在 BCLK 的下降沿采样数据)。

5.3 外挂输入设备

说明

通常为 CAT1 模组。

12.288M 为 codec 模块的总时钟, 用于计算分频比。

- EXTERNAL_AI_PORT: 枚举值对应的端口需与硬件上外挂输入设备连接的端口一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_ai.h” 中的 uapi_ai_port 枚举)。
- EXTERNAL_AI_I2S_ATTR_MASTER: 需与外挂输入设备的 I2S 主/从模式相反 (1 表示做主设备, 0 表示做从设备)。
- EXTERNAL_AI_I2S_ATTR_I2S_MODE: 枚举值对应的模式需与外挂输入设备的 I2S 模式一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 uapi_audio_i2s_mode 枚举)。
- EXTERNAL_AI_I2S_ATTR_MCLK: 12.288M 时钟经过此枚举对应的比值分频后, 需与外挂输入设备的 I2S FS 一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 uapi_audio_i2s_mclk_sel 枚举)。

例如 FS 为 16K，计算得分频比应为 768 ($12.288\text{M}/16\text{K}=768$)，查询枚举 `uapi_audio_i2s_mclk_sel`，应选择 `UAPI_AUDIO_I2S_MCLK_768_FS`，对应枚举值为 4。

- `EXTERNAL_AI_I2S_ATTR_BCLK`: 12.288M 时钟经过此枚举对应的比值分频后，需与外挂输入设备的 I2S BCLK 一致（请参考“`drivers\drivers\driver\audio\include\soc_uapi_audio.h`”中的 `uapi_audio_i2s_bclk_sel` 枚举）。

例如 BCLK 为 512K，计算得分频比应为 24 ($12.288\text{M}/512\text{K}=24$)，查询枚举 `uapi_audio_i2s_bclk_sel`，应选择 `UAPI_AUDIO_I2S_BCLK_24_DIV`，对应枚举值为 24。

- `EXTERNAL_AI_I2S_ATTR_BIT_DEPTH`: 需与外挂输入设备的 I2S 位深一致。
- `EXTERNAL_AI_I2S_ATTR_AUDIO_CH`: 需与外挂输入设备的 I2S 声道数一致。

当 I2S 模式选择 PCM 时，需要修改以下配置：

- `EXTERNAL_AI_I2S_ATTR_SAMPLE_RISE_EDGE`: 采样沿选择（1 表示在 BCLK 的上升沿采样数据，0 表示在 BCLK 的下降沿采样数据）。
- `EXTERNAL_AI_I2S_ATTR_PCM_DELAY_CYCLE`: BCLK 时延（请参考“`drivers\drivers\driver\audio\include\soc_uapi_audio.h`”中的 `uapi_audio_i2s_pcm_delay` 枚举）。
- `EXTERNAL_AI_PCM_ATTR_CHANNELS`: 需与外挂输入设备的 PCM 声道数一致。
- `EXTERNAL_AI_PCM_ATTR_BIT_DEPTH`: 需与外挂输入设备的 PCM 位深一致。
- `EXTERNAL_AI_PCM_ATTR_SAMPLE_RATE`: 需与外挂输入设备的 PCM 采样率一致。
- `EXTERNAL_AI_PCM_ATTR_FRAME_PER_SEC`: 当前不可配置，请使用默认值。

5.4 外挂输出设备

□ 说明

通常为 CAT1 模组。

12.288M 为 codec 模块的总时钟，用于计算分频比。

- `EXTERNAL_SND_OUT_PORT`: 枚举值对应的端口需与硬件上外挂输出设备连接的端口一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_sound.h” 中的 `uapi_snd_out_port` 枚举)。
- `EXTERNAL_SND_OUT_AUDIO_CHANNEL`: 需与外挂输出设备的声道数一致。
- `EXTERNAL_SND_OUT_AUDIO_BIT_DEPTH`: 需与外挂输出设备的位深一致。
- `EXTERNAL_SND_OUT_AUDIO_SAMPLE_RATE`: 需与外挂输出设备的采样率一致。
- `EXTERNAL_SND_OUT_PORT_NUM`: 输出端口个数。
- `EXTERNAL_SND_OUT_I2S_ATTR_BIT_DEPTH`: 需与外挂输出设备的 I2S 位深一致。
- `EXTERNAL_SND_OUT_I2S_ATTR_CHANNEL`: 需与外挂输出设备的 I2S 声道数一致。
- `EXTERNAL_SND_OUT_I2S_ATTR_I2S_MODE`: 枚举值对应的模式需与外挂输出设备的 I2S 模式一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 `uapi_audio_i2s_mode` 枚举)。
- `EXTERNAL_SND_OUT_I2S_ATTR_MCLK`: 12.288M 时钟经过此枚举对应的比值分频后, 需与外挂输出设备的 I2S FS 一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 `uapi_audio_i2s_mclk_sel` 枚举)。
例如 FS 为 16K, 计算得分频比应为 768 ($12.288\text{M}/16\text{K}=768$), 查询枚举 `uapi_audio_i2s_mclk_sel`, 应选择 `UAPI_AUDIO_I2S_MCLK_768_FS`, 对应枚举值为 4。
- `EXTERNAL_SND_OUT_I2S_ATTR_MASTER`: 需与外挂输出设备的 I2S 主/从模式相反 (1 表示做主设备, 0 表示做从设备)。
- `EXTERNAL_SND_OUT_I2S_ATTR_SAMPLE_RISE_EDGE`: 采样沿选择 (仅当 I2S 模式选择 PCM 时需要配置, 1 表示在 BCLK 的上升沿采样数据, 0 表示在 BCLK 的下降沿采样数据)。
- `EXTERNAL_SND_OUT_I2S_ATTR_BCLK`: 12.288M 时钟经过此枚举对应的比值分频后, 需与外挂输出设备的 I2S BCLK 一致 (请参考 “drivers\drivers\driver\audio\include\soc_uapi_audio.h” 中的 `uapi_audio_i2s_bclk_sel` 枚举)。

例如 BCLK 为 512K，计算得分频比应为 24 ($12.288\text{M}/512\text{K}=24$)，查询枚举 `uapi_audio_i2s_bclk_sel`，应选择 `UAPI_AUDIO_I2S_BCLK_24_DIV`，对应枚举值为 24。

5.5 TRACK MODE

参考 “drivers\drivers\driver\audio\include\soc_uapi_sound.h” 中的 `uapi_snd_track_mode` 枚举：

- `SND_TRACK_MODE_BT_MUSIC`：蓝牙音乐场景的 TRACK MODE。
- `SND_TRACK_MODE_BT_SCO`：蓝牙通话场景的 TRACK MODE。
- `SND_TRACK_MODE_LOCAL_CAT1`：4G VOLTE 本地通话场景的 TRACK MODE。
- `SND_TRACK_MODE_BT_CAT1`：4G VOLTE 蓝牙通话场景的 TRACK MODE。
- `SND_TRACK_MODE_LOCAL_MUSIC`：本地音乐场景的 TRACK MODE。

5.6 AEF

- `SND_PORT_AEF_TYPE`：当前不可配置，请使用默认值。
- `SND_PORT_AEF_ENABLE`：AEF 算法开关（true 代表开启 AEF 算法，false 代表关闭 AEF 算法）。

5.7 编解码率

- `OPUS_ENC_BIT_RATE`：OPUS 编解码率。
- `MP3_ENC_BIT_RATE`：MP3 编解码率。

5.8 上行音量

- `DEFAULTE_AI_VOL_DB_INTERGER`：AI 音量。

6 AT 命令使用

- 所有自定义命令必须以“AT^”为前缀。
- AT 命令的字符串不区分大小写，建议统一为大写字符。
- AT 指令以<CR><LF>为结尾，表明一条指令结束。
- AT 指令的返回值包括两部分：响应信息和结果码。
- 响应信息以“^”开头，冒号之后有一个空格，响应信息可以多行，并以<CR><LF>为结尾。
- 结果码“OK/ERROR”独立一行输出。

- 6.1 本地播放
- 6.2 本地采集
- 6.3 音频管理

6.1 本地播放

6.1.1 设置指令

表6-1 本地播放设置指令一览表

类型	指令	示例	说明
开始音乐播放	AT^MEDIAMUSICPLAY=xx,xxxx xxx,x,x	AT^MEDIAMUSICPLAY=xx,/user/test/audio_cap.mp3,1,0 执行成功：OK 执行失败：ERROR	xx：开始播放标记，固定参数。 /user/test/audio_cap.mp3：音乐播放路径。

类型	指令	示例	说明
			0: STREAM 方式播放, 目前不支持 STREAM, 固定参数。 1: URI 方式播放, 目前只支持 URI, 固定参数。
暂停音乐播放	AT^MEDIAMUSICPAUSE=pause	AT^MEDIAMUSICPAUSE=pause 执行成功: OK 执行失败: ERROR	pause: 暂停播放动作标记。
恢复音乐播放	AT^MEDIAMUSICRESUME=resume	AT^MEDIAMUSICRESUME=resume 执行成功: OK 执行失败: ERROR	resume: 恢复播放动作标记。
停止播放音乐	AT^MEDIAMUSICSTOP=stop	AT^MEDIAMUSICSTOP=stop 执行成功: OK 执行失败: ERROR	stop: 停止播放动作标记。
设置音乐音量	AT^MEDIASET MUSICVOLUME=setvolume,xxx	AT^MEDIASET MUSICVOLUME=setvolume,100 执行成功: OK 执行失败: ERROR	setvolume: 设置音频音量的动作标记。 100: 音量的挡位值 (0~100)。
设置音乐循环播放	AT^MEDIAMUSICSINGLELOOP=xxx	AT^MEDIAMUSICSINGLELOOP=loopen 执行成功: OK 执行失败: ERROR	loopen: 开启音频循环播放。 loopoff: 关闭音频循环播放。
切换上一曲音乐	AT^MEDIAMUSICSWITCHSONG=xxx	AT^MEDIAMUSICSWITCHSONG=prevsong 执行成功: OK 执行失败: ERROR	prevsong: 切换到上一首音乐的动作标记。 nextsong: 切换到下一首音乐的动作标记。
开始播放	AT^MEDIANOTIFYPLAY=xx,/us	AT^MEDIANOTIFYPLAY=xx,/user/test/naozho	xx: 开始播放标记, 固

类型	指令	示例	说明
提示音	er/test/naozhong.sbc,1,0	ng.sbc,1,0 执行成功：OK 执行失败：ERROR	定参数。 /user/test/naozhong.sbc ：提示音播放路径。 0：STREAM 方式播放，目前不支持 STREAM，固定参数。 1：URI 方式播放，目前只支持 URI，固定参数。
停止播放提示音	AT^MEDIANOTIFYPLAY=stop	AT^MEDIANOTIFYPLAY=stop 执行成功：OK 执行失败：ERROR	stop：停止播放动作标记。

6.1.2 查询指令

表6-2 本地播放查询指令一览表

类型	指令	示例	说明
查询音乐音量	AT^MEDIAGETMUSICVOLUME=getvolume	AT^MEDIAGETMUSICVOLUME=getvolume 执行成功：OK 执行失败：ERROR	getvolume：获取音量的动作标记。
查询播放文件信息	AT^MEDIAMUSICABLUMINFO=abluminfo	AT^MEDIAMUSICABLUMINFO=abluminfo 执行成功：OK 执行失败：ERROR	abluminfo：获取播放文件信息动作标记。
查询当前播放状态	AT^MEDIAGETCURRENTMUSICSTATE	AT^MEDIAGETCURRENTMUSICSTATE 执行成功：OK	无。

类型	指令	示例	说明
		执行失败：ERROR	
查询当前播放的 dump 信息	AT^MEDIAMUSICPLAY=dumpinfo	AT^MEDIAMUSICPLAY=dumpinfo 执行成功：OK 执行失败：ERROR	dumpinfo：获取当前播放 dump 信息动作标记。

6.2 本地采集

6.2.1 设置指令

表6-3 本地录制设置指令一览表

类型	指令	示例	说明
开始录制音频	AT^MEDIAAUDIORECORD=xx,mp3,16000,1,16,/user/test/audio_cap.mp3	AT^MEDIAAUDIORECORD=xx,mp3,16000,1,16,/user/test/audio_cap.mp3 执行成功：OK 执行失败：ERROR	xx：开始录制标记。 mp3：录制音频格式。 16000：录音音频采样率。 1：录制音频通道数。 16：录制音频位宽。 /user/test/audio_cap.mp3：录音数据存放文件。
停止录制音频	AT^MEDIAAUDIORECORD=stop	AT^MEDIAAUDIORECORD=stop 执行成功：OK 执行失败：ERROR	stop：停止录制动作标记。

6.2.2 查询指令

表6-4 本地录制查询指令一览表

类型	指令	示例	说明
查询录制音频信息	AT^MEDIAAUDIORECORD=dumpinfo	AT^MEDIAAUDIORECORD=dumpinfo 执行成功：OK 执行失败：ERROR	dumpinfo：获取录制音频信息动作标记。

6.3 音频管理

6.3.1 设置指令

表6-5 音频管理设置指令一览表

类型	指令	示例	说明
开始视频预览	AT^MEDIACAMERAPREVIEWSTART=start,/user/test/454x454_10s.data	AT^MEDIACAMERAPREVIEWSTART=start,/user/test/454x454_10s.data 执行成功：OK 执行失败：ERROR	start：开始动作标记。 /user/test/454x454_10s.data：预览文件路径。
停止视频预览	AT^MEDIACAMERAPREVIEWSTART=stop	AT^MEDIACAMERAPREVIEWSTART=stop 执行成功：OK 执行失败：ERROR	stop：停止动作标记。
内部测试开始本地CAT1通话	AT^MEDIAVOICECALLVOLTESTART	AT^MEDIACAMERAPREVIEWSTART=stop 执行成功：OK 执行失败：ERROR	只是拉起音频通路，无法实现数据的轮转。
内部测试停止	AT^MEDIAVOICECALLVOLTESTOP	AT^MEDIACAMERAPREVIEWSTART=stop	只是销毁拉起的音频通路。

类型	指令	示例	说明
本地 CAT1 通话		执行成功: OK 执行失败: ERROR	
设置 CAT1 通话 3A 算法的 开关	AT^MEDIAVOIC ECALLVOLTE=s eaenable,1	AT^MEDIAVOICECALL VOLTE=seaenable,1 执行成功: OK 执行失败: ERROR	seaenable: 设置 3A 算 法动作开关标记。 0: 关闭 3A 算法。 1: 打开 3A 算法。
设置 CAT1 通话上 行是否 静音	AT^MEDIAVOIC ECALLVOLTE= mute,0	AT^MEDIAVOICECALL VOLTE=mute,0 执行成功: OK 执行失败: ERROR	mute: 设置上行静音的 标记。 0: 解除上行静音。 1: 上行静音。
设置 CAT1 下行通 话音量	AT^MEDIAVOIC ECALLVOLTE=s etvolume,80	AT^MEDIAVOICECALL VOLTE=setvolume,80 执行成功: OK 执行失败: ERROR	setvolume: 设置音频音 量的动作标记。 80: 音量的挡位值 (0 ~ 100)。
设置 A2DP sink 的 AEF 算 法开关 和参数	AT^MEDIA2DPSI NKAEF=1,29	AT^MEDIA2DPSINKAE F=1,29 执行成功: OK 执行失败: ERROR	注意: 此命令需要在 sink 音乐播放中执行。 1: AEF 音效是否使能 (0: 不使能; 1: 使 能)。 29: 音效算法参数 (可 以缺省), 参数范围 8 ~ 29。
设置本 地音乐 播放的 ARF 算 法开关	AT^MEDIAMUSI CPLAY=aefenabl e,400,1	AT^MEDIAMUSICPLAY =aefenable,400,1 执行成功: OK 执行失败: ERROR	注意: 此命令需要在本地 音乐播放中执行。 400: AEF 的算法类型, 目前仅支持当前类型。 1: AEF 音效是否使能 (0: 不使能; 1: 使 能)。

类型	指令	示例	说明
设置本地音乐播放的 AEF 算法参数	AT^MEDIAMUSICPLAY=setaefparam,400,10,4	AT^MEDIAMUSICPLAY=setaefparam,400,10,4 执行成功：OK 执行失败：ERROR	注意：此命令需要在本地音乐播放中执行。 400：AEF 的算法类型，目前仅支持当前类型。 10：算法参数，范围 8~29。 4：参数长度。